Welcome back to CSC148!

Class will begin at 10 minutes after the hour!

Announcements

- A0 was due on Monday (up to Friday 6pm with grace tokens!)
  - **Reminder: Do not reveal any part of your solutions!**
- Midterm next week T/W during class time
  - Material is cumulative (everything from the first week to the end of this week, including A0)
  - **Please see Quercus for room locations!**
- A1 will come out *after* the midterm!
- A0 marks will not be released until *after* the midterm
  (but *will* be released before the drop deadline!)

Announcements

- There *will* be a lab this week **and** next week
  - This week: Linked Lists (mock test)
  - Next week: Code Review
- No class in the third lecture hour after the midterm

Communicating about running time

We want a way of talking about running time that doesn't depend on timing experiments or exact step counts.

What we care about is the *type of growth*:

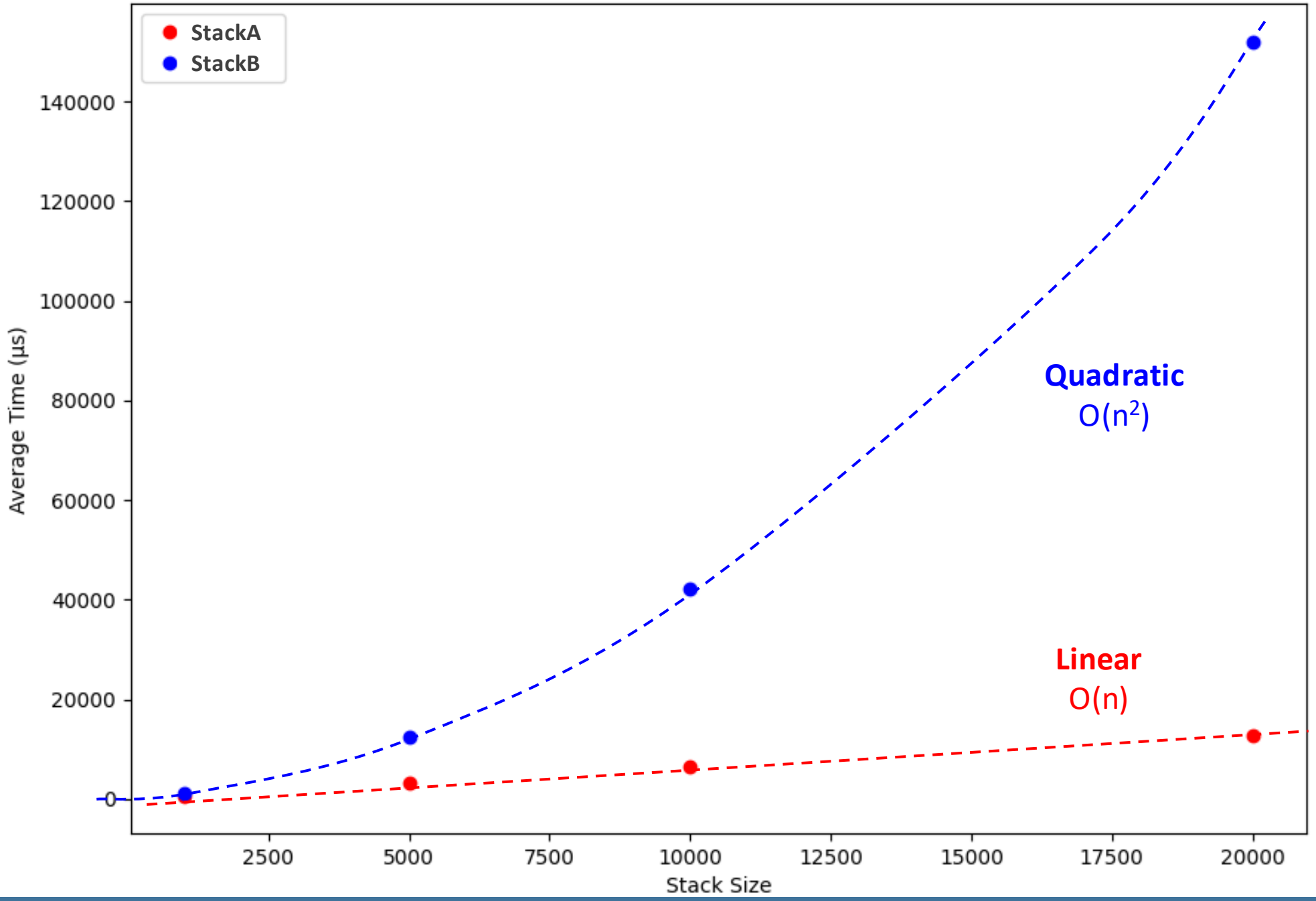
logarithmic, linear, quadratic, exponential, etc.

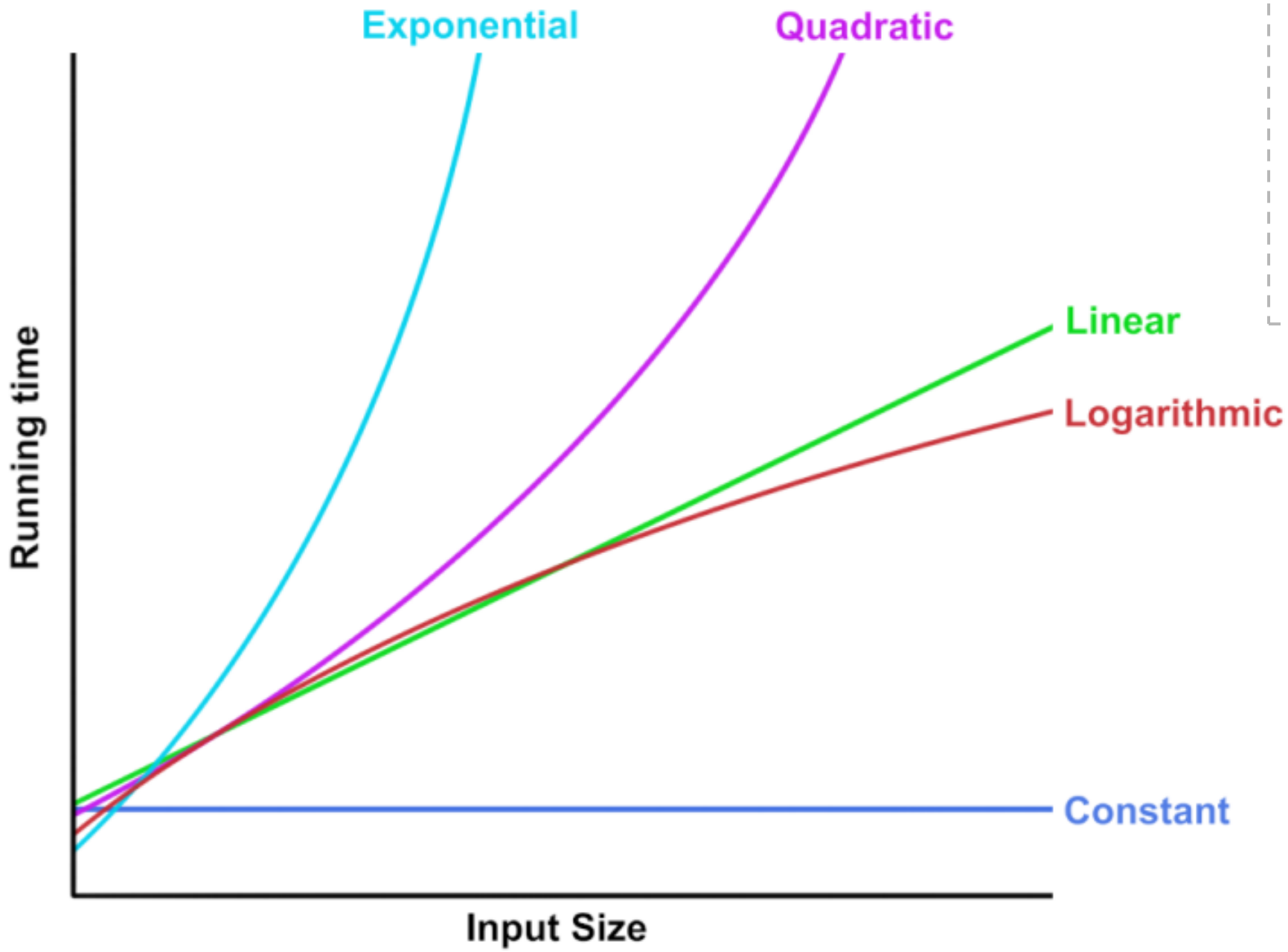
Related Reading: Analysing Algorithm Running Time

Big-Oh notation

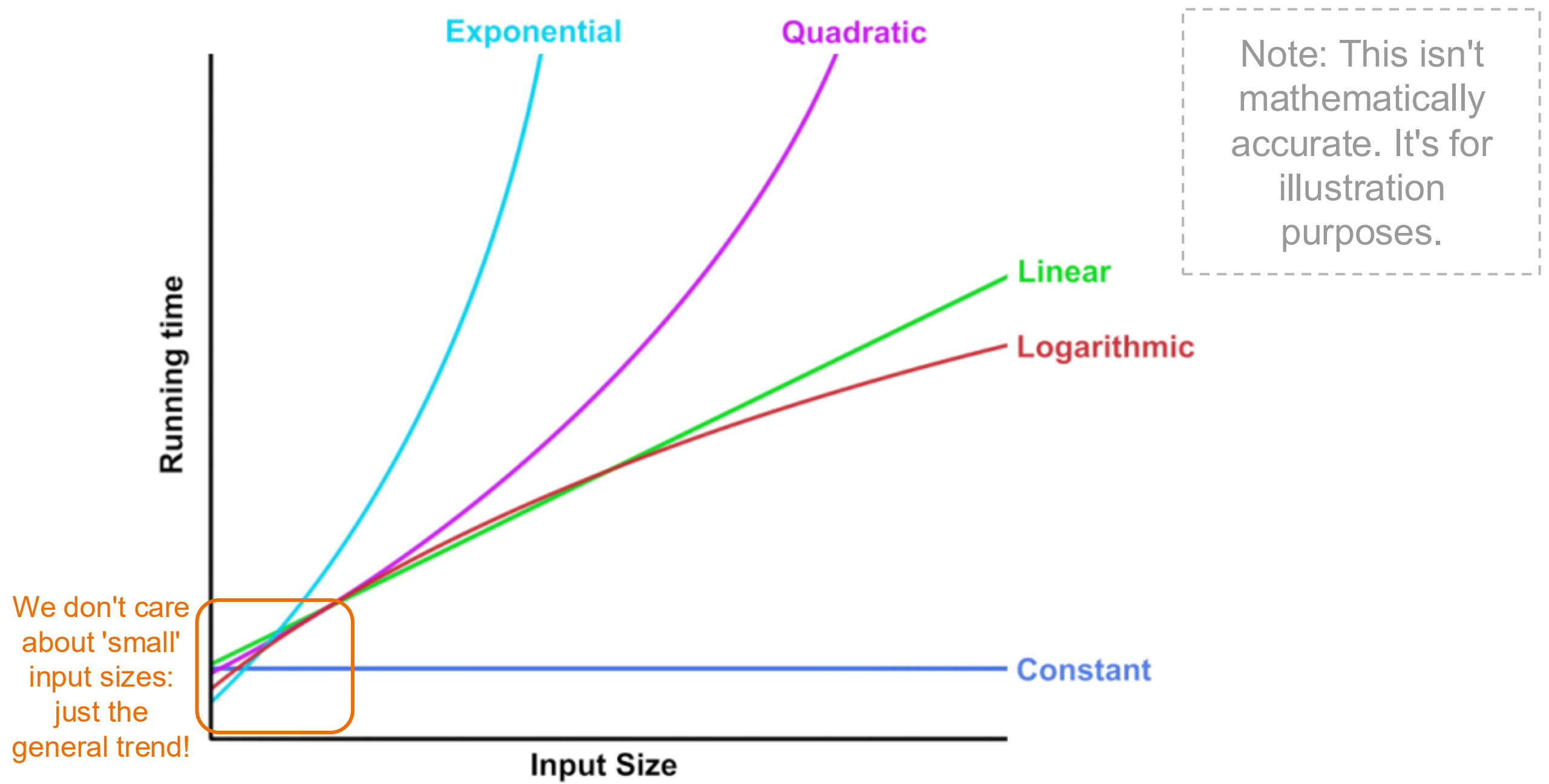
$O(n)$, $O(n^2)$, $O(\log n)$, $O(2^n)$, …

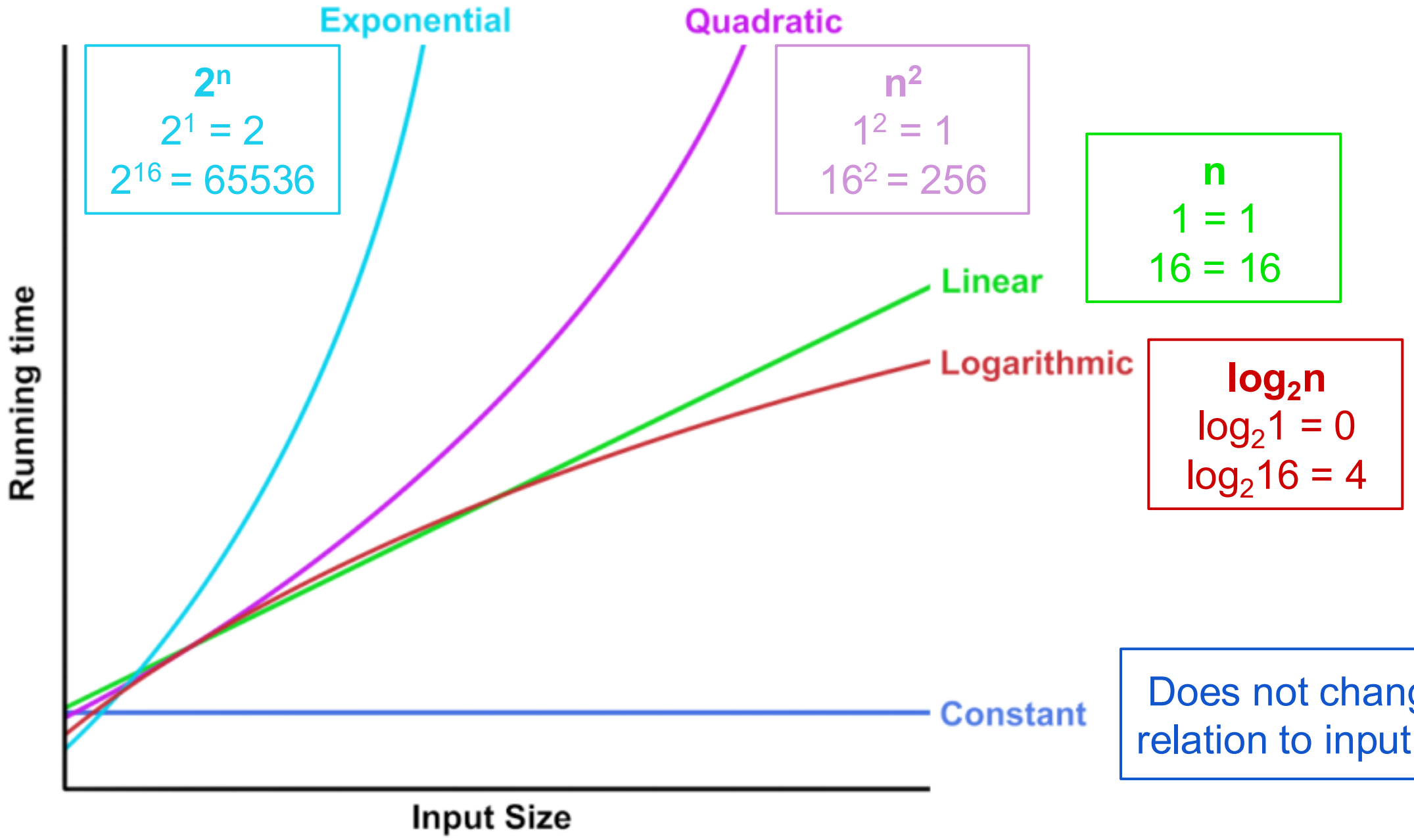
Homework: read section 4.4 of the Lecture Notes.


StackA
StackB
140000
120000
100000
80000
60000
40000
20000
0
Average Time (µs)
2500
5000
7500
10000
12500
15000
17500
20000
Stack Size
Quadratic
O(n²)
Linear
O(n)



Note: This isn't mathematically accurate. It's for illustration purposes.


Exponential
Quadratic
Linear
Logarithmic
Constant
Running time
Input Size
We don't care about 'small' input sizes: just the general trend!
Note: This isn't mathematically accurate. It's for illustration purposes.


Exponential
Quadratic
2^n
$2^1 = 2$
$2^{16} = 65536$
n^2
$1^2 = 1$
$16^2 = 256$
n
1 = 1
16 = 16
Linear
Logarithmic
$\log_2 n$
$\log_2 1 = 0$
$\log_2 16 = 4$
Running time
Input Size
Constant
Does not change in relation to input size.

Linked Lists

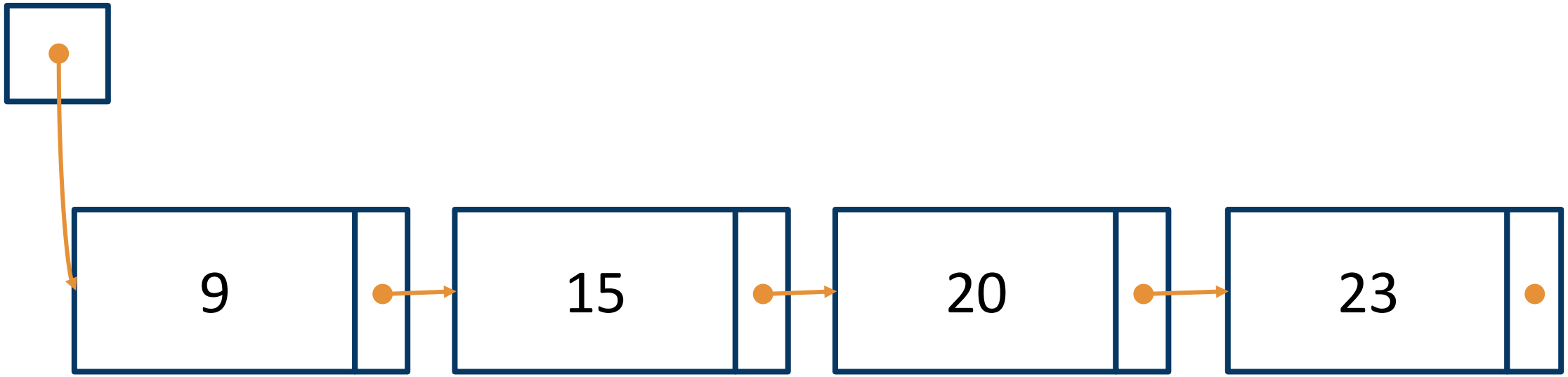
CSC148: INTRODUCTION TO COMPUTER SCIENCE
Related Readings: 6.1–6.4

This Lecture: Linked Lists!

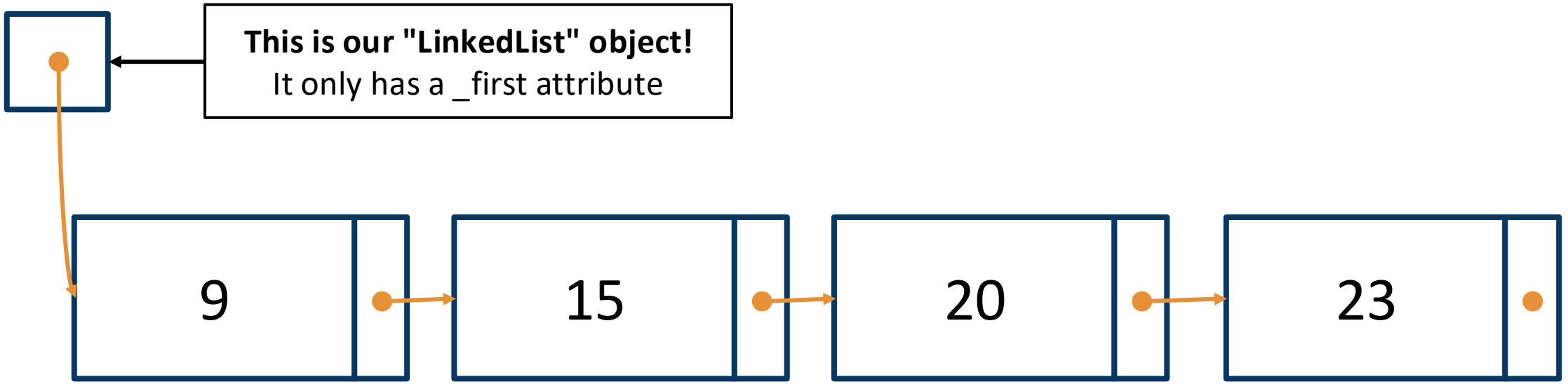
By the end of this week's lectures, you should be able to:

- Define a **Linked List**
- Write new Linked List methods & functions
  - Create examples and draw corresponding Linked List diagrams when planning or explaining a Linked List method
  - Recall common mistakes and edge cases
  - Identify and implement relevant stopping conditions
  - Traverse and mutate a Linked List
- Efficiency of Linked Lists
  - Recall the Big-Oh complexity of insertion and deletion in Linked Lists
  - Determine the Big-Oh complexity of a given Linked List operation

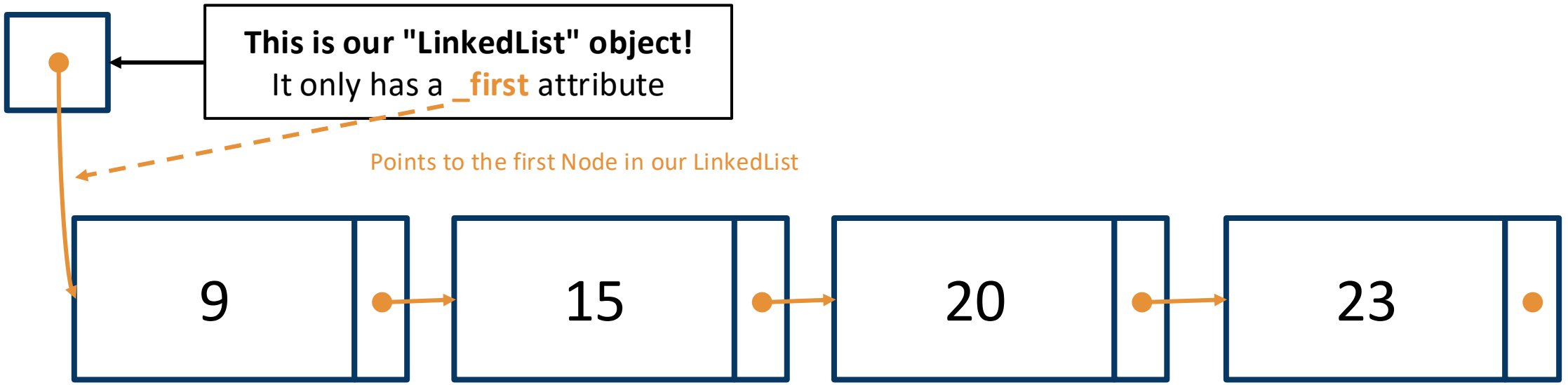
Recap: LinkedLists



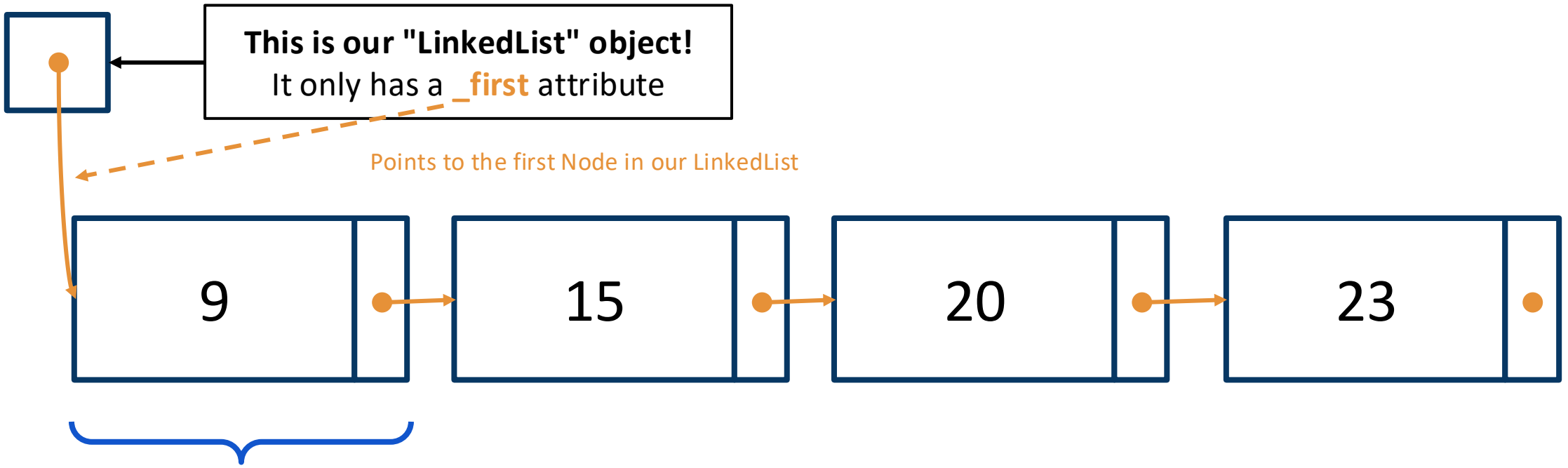
Recap: LinkedLists



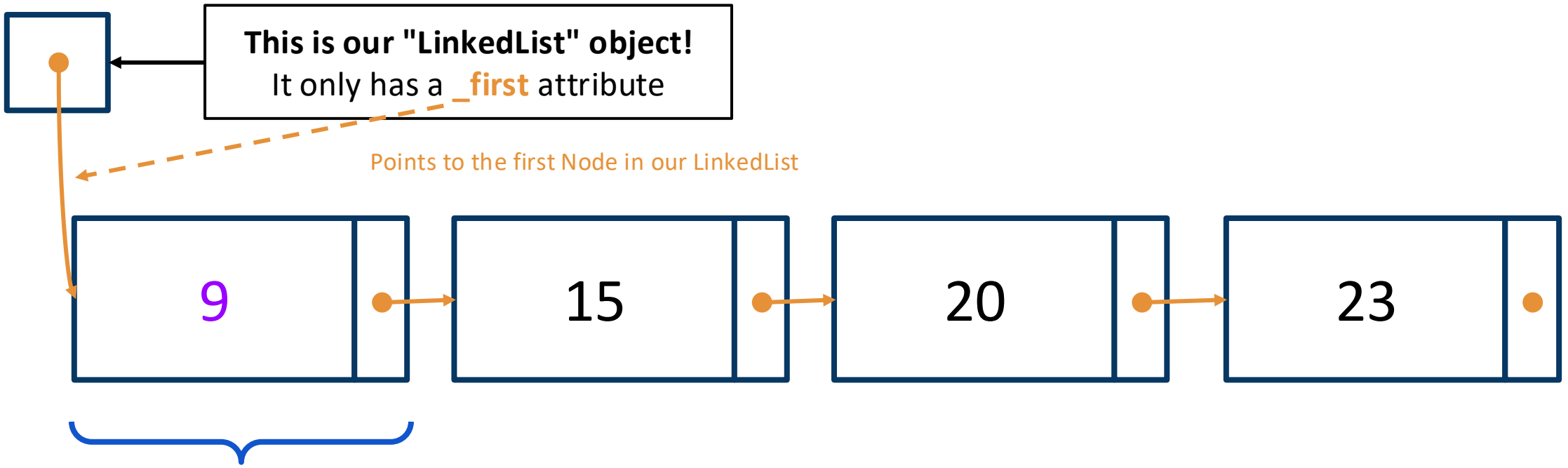
Recap: LinkedLists



Recap: LinkedLists



Recap: LinkedLists

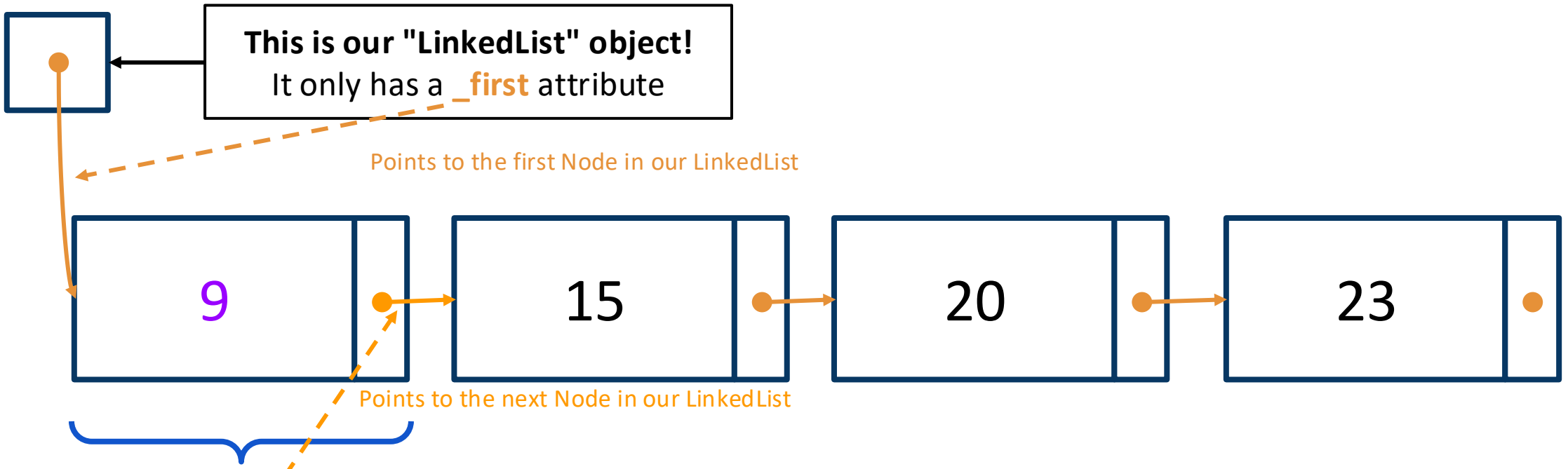


This is a "_Node" object!
It has an **item** attribute and next attribute

You use .item to get the *value* of the node.

A node is *not* its item – it's more than that!

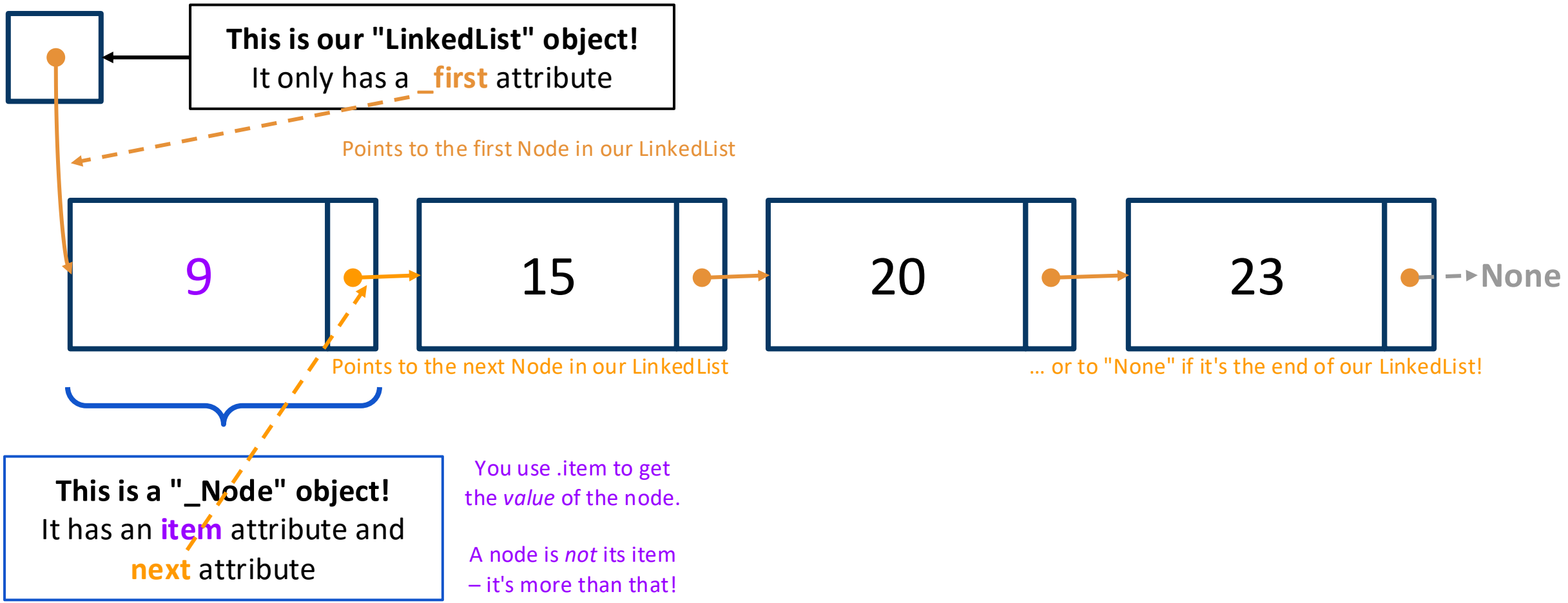
Recap: LinkedLists



You use .item to get the *value* of the node.

A node is *not* its item – it's more than that!

Recap: LinkedLists



Be careful!

A _Node whose *item* is None

```
node.item = None
```

For comparison, think of the list:
[1, 2, None, 4]

None

The value None itself.
Not a _Node at all.

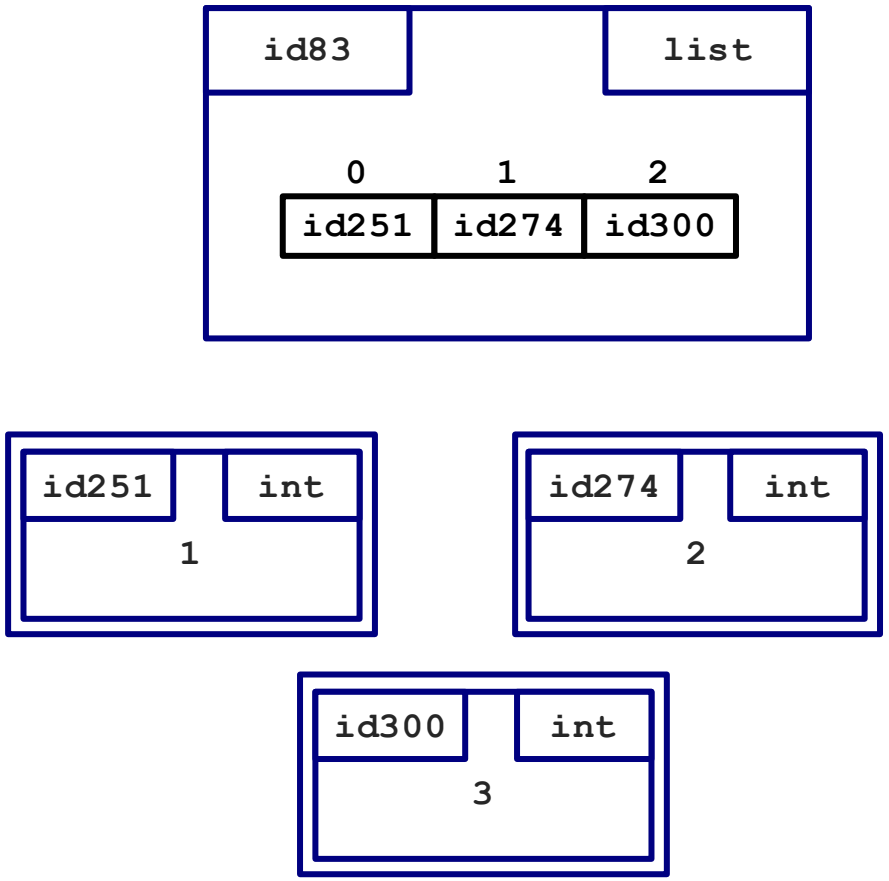
```
node = None
```

Two implementations of the list ADT

- **Array-based** lists store references to elements in contiguous blocks of memory.

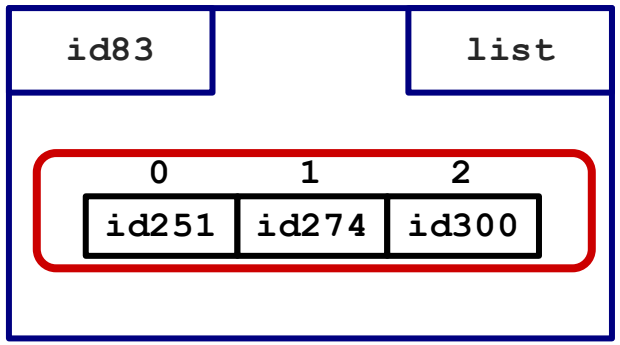
- **Linked** lists can store elements anywhere, but each element must store a reference to the *next* element in the list.

Lists in Memory

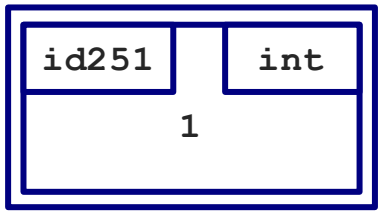


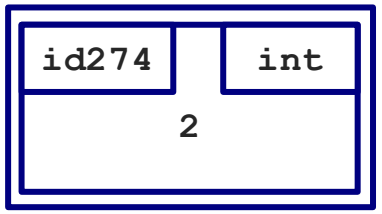
83	id251
84	id274
85	id300
	extra space
	. . .
251	1
	. . .
274	2
	. . .
300	3

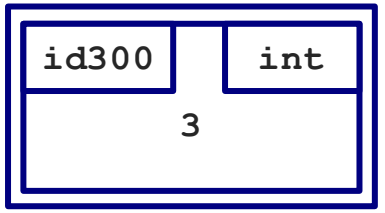
Lists in Memory

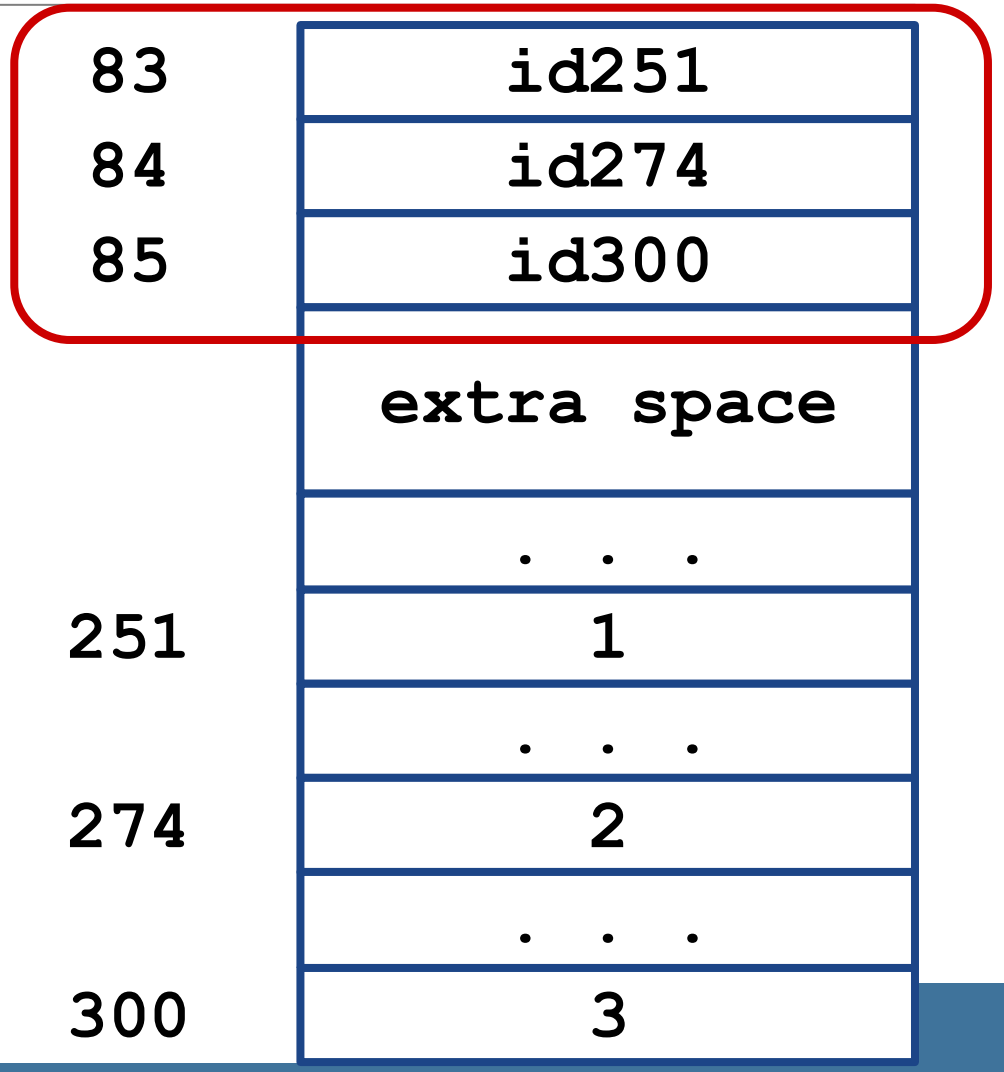


Stored in consecutive locations in memory!

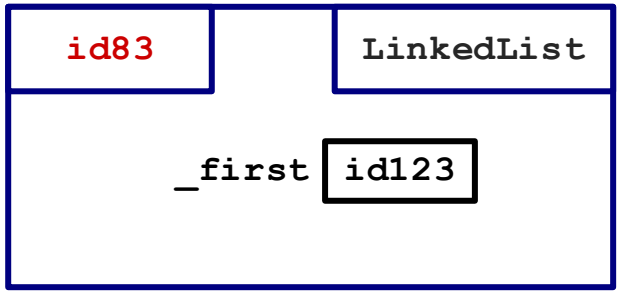




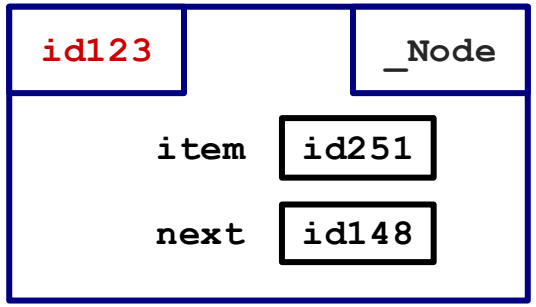


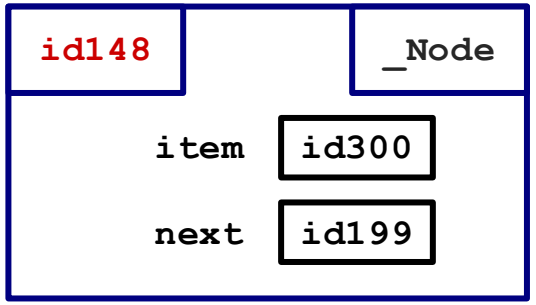


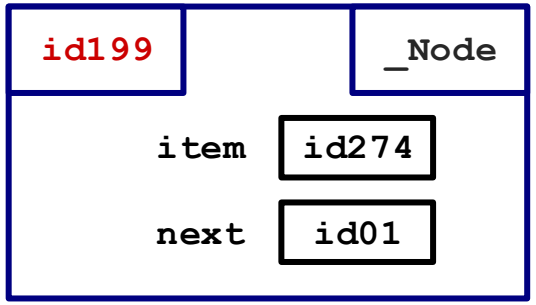
Linked Lists in Memory (Model)

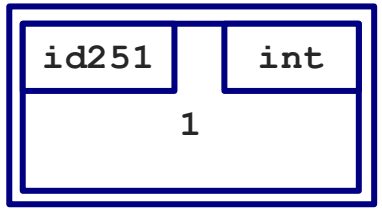


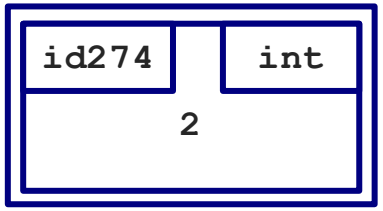
LinkedList and _Nodes do not have to be consecutive in memory!

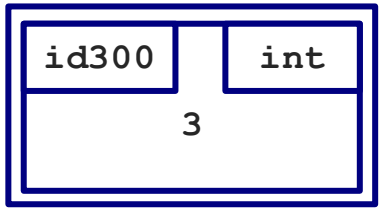


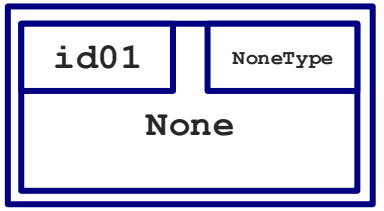












Our goals this week

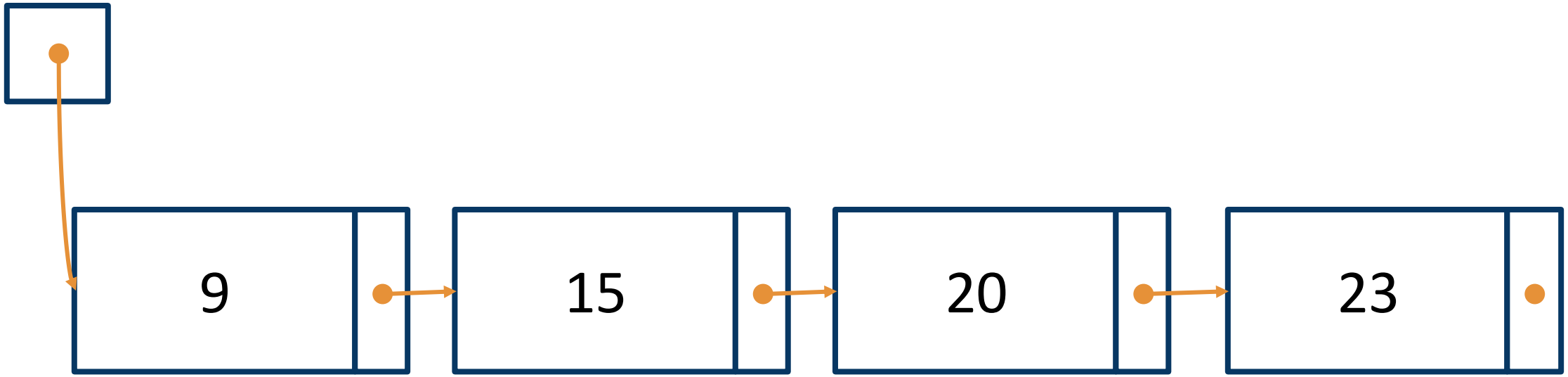
1. Work with linked lists by implementing the same operations as Python's built-in `list`.
2. Analyze the running time of our linked list methods and compare them to the array-based `list`.

Code summary

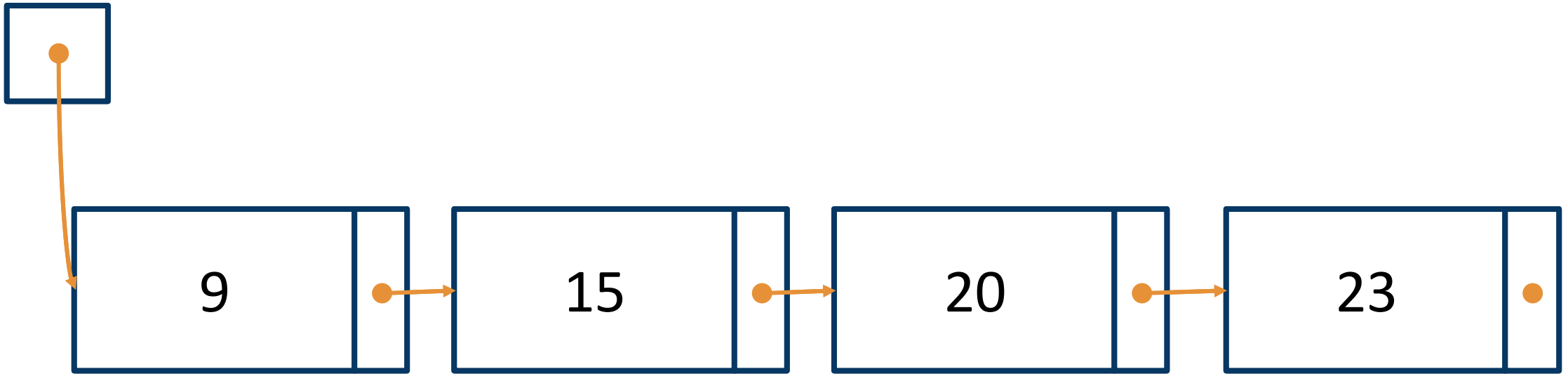
```
class _Node:
    item: Any
    next: _Node | None

class LinkedList:
    _first: _Node | None
```

```
curr = self._first
while curr is not None:
    ... curr.item ...
    curr = curr.next
```

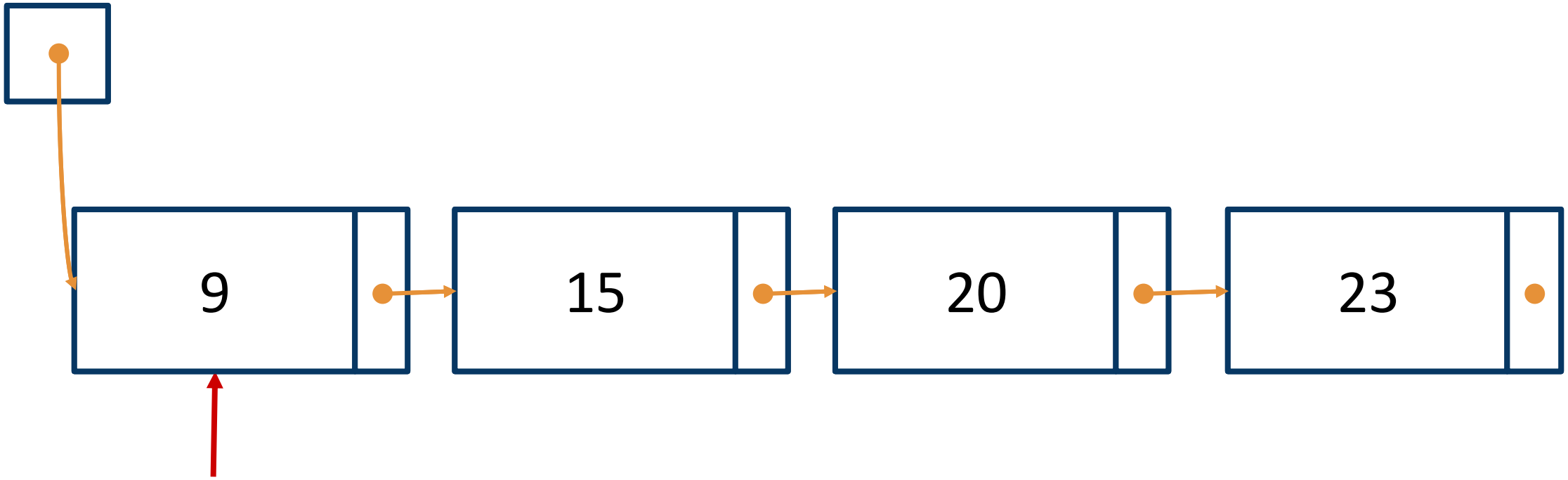
Recap: __contains__



Recap: __contains__(**20**)



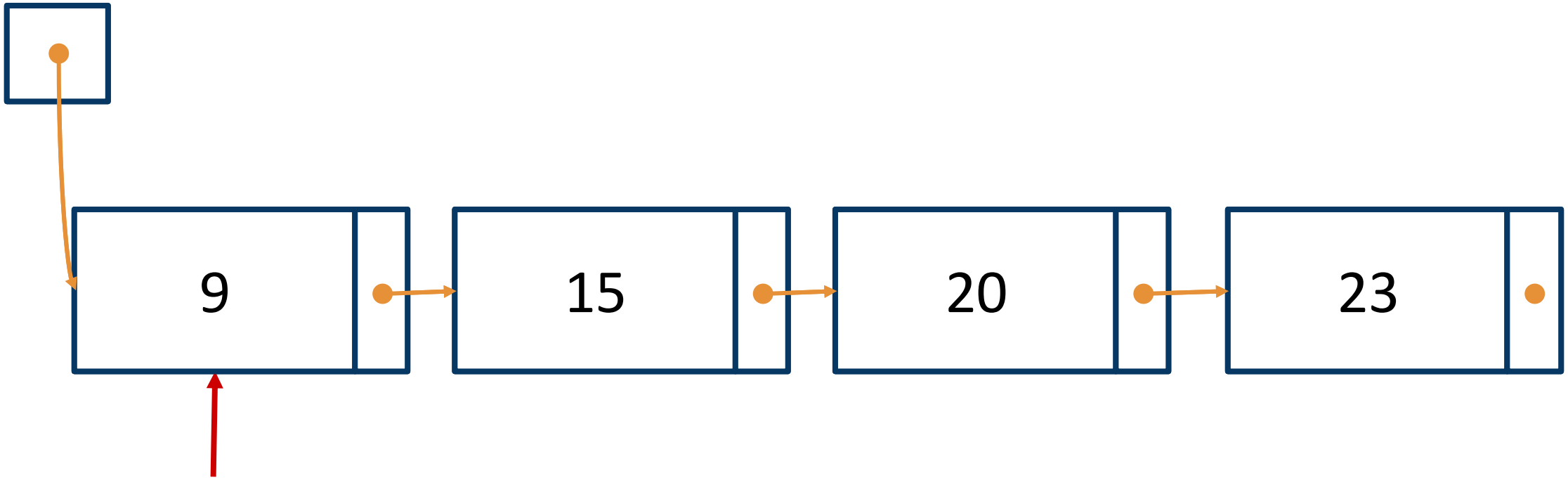
Recap: __contains__(**20**)




```
curr = self._first
```

Start at the first _Node

Recap: __contains__(**20**)

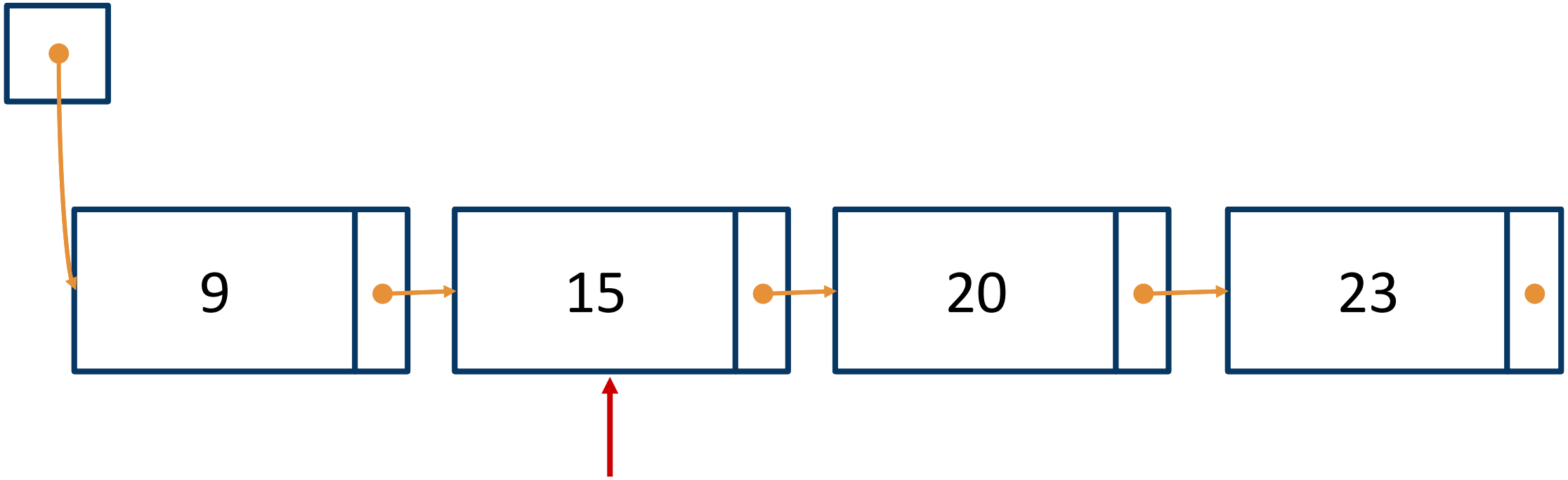


`curr`

`curr.item != 20`

So, move on to the next item

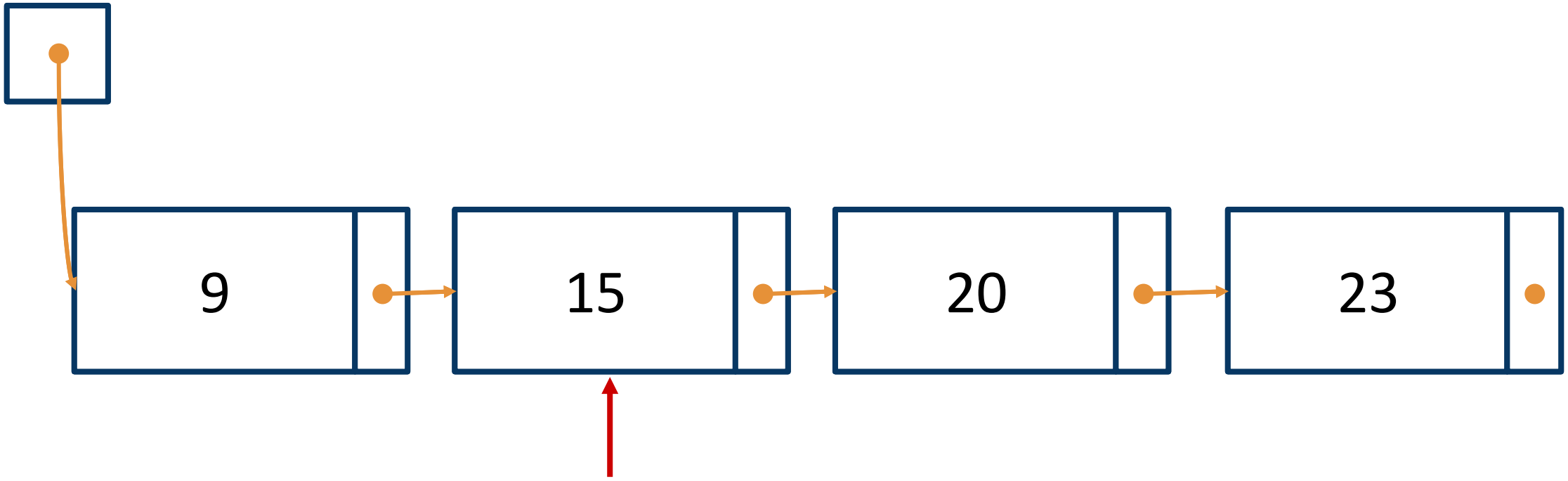
Recap: __contains__(**20**)




```
curr = curr.next
```

This changes curr to our next _Node

Recap: __contains__(**20**)

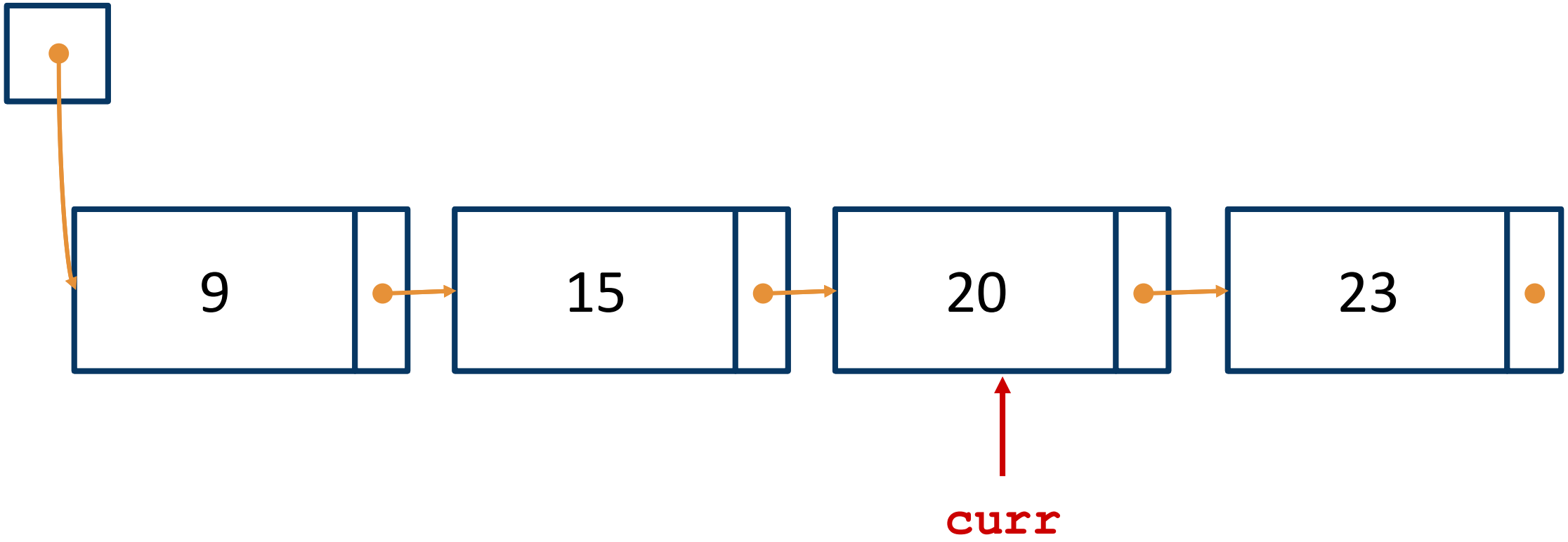


`curr`

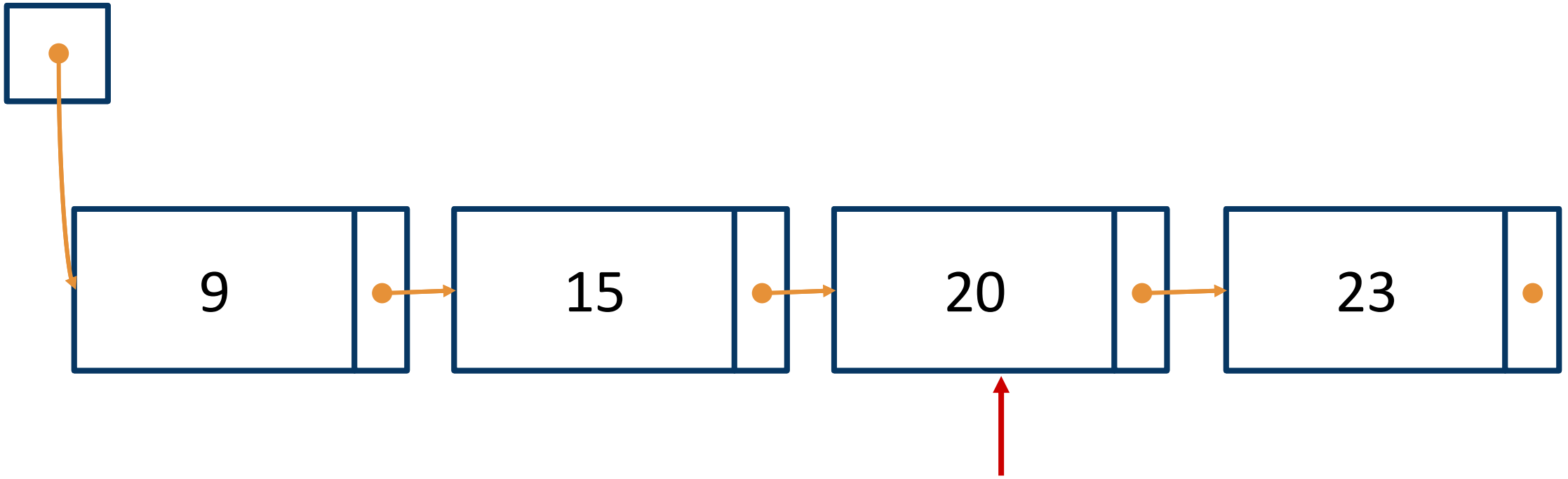
`curr.item != 20`

So, move on to the next item

Recap: __contains__(**20**)



Recap: __contains__(**20**)

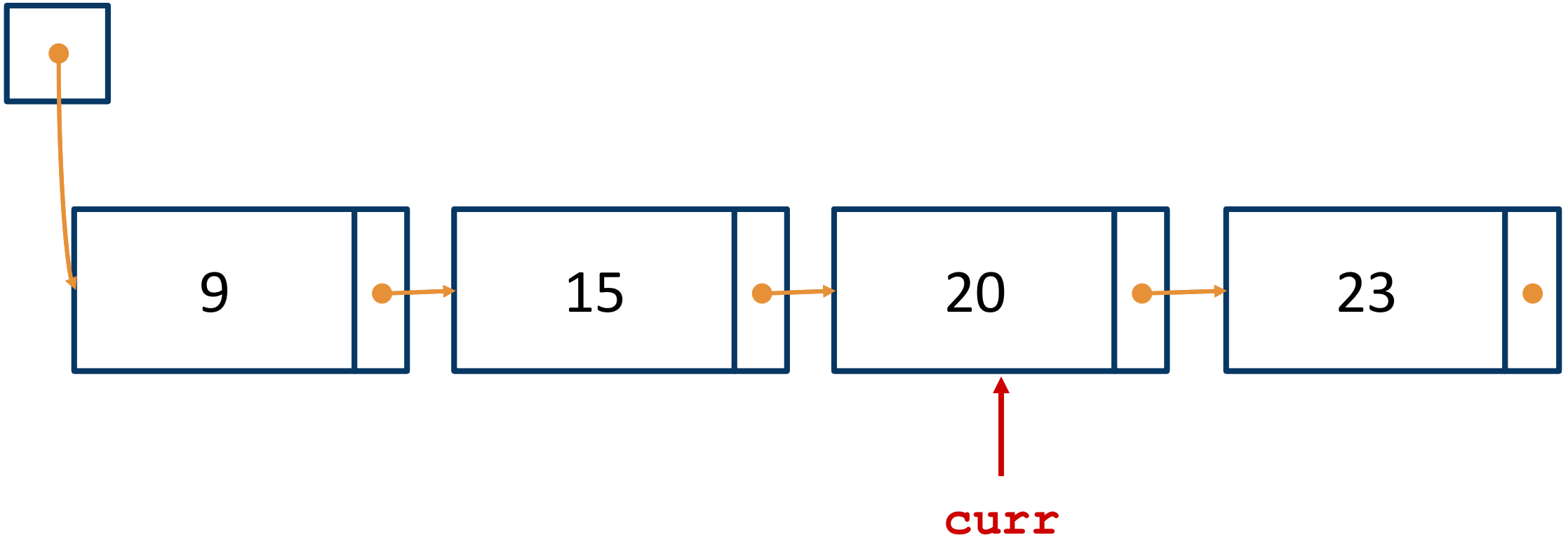


`curr.item == 20`

So we can stop looking and return True

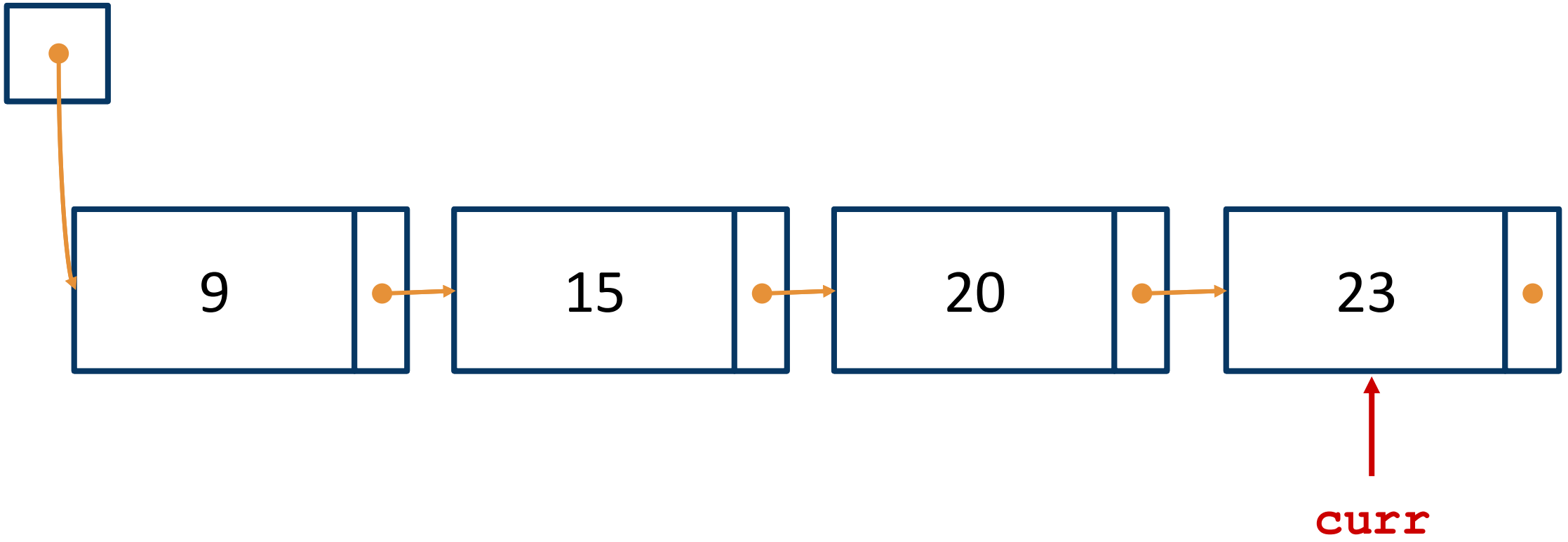
What if the item isn't in our list?

Recap: __contains__(**5**)



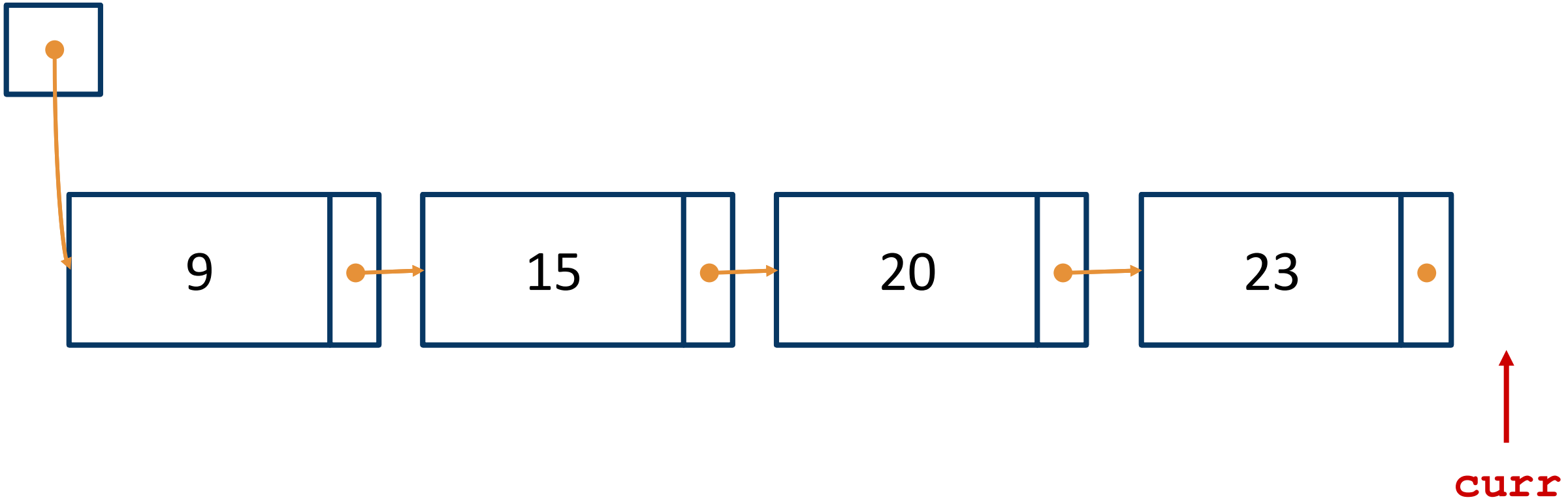
What if the item isn't in our list?

Recap: __contains__(**5**)



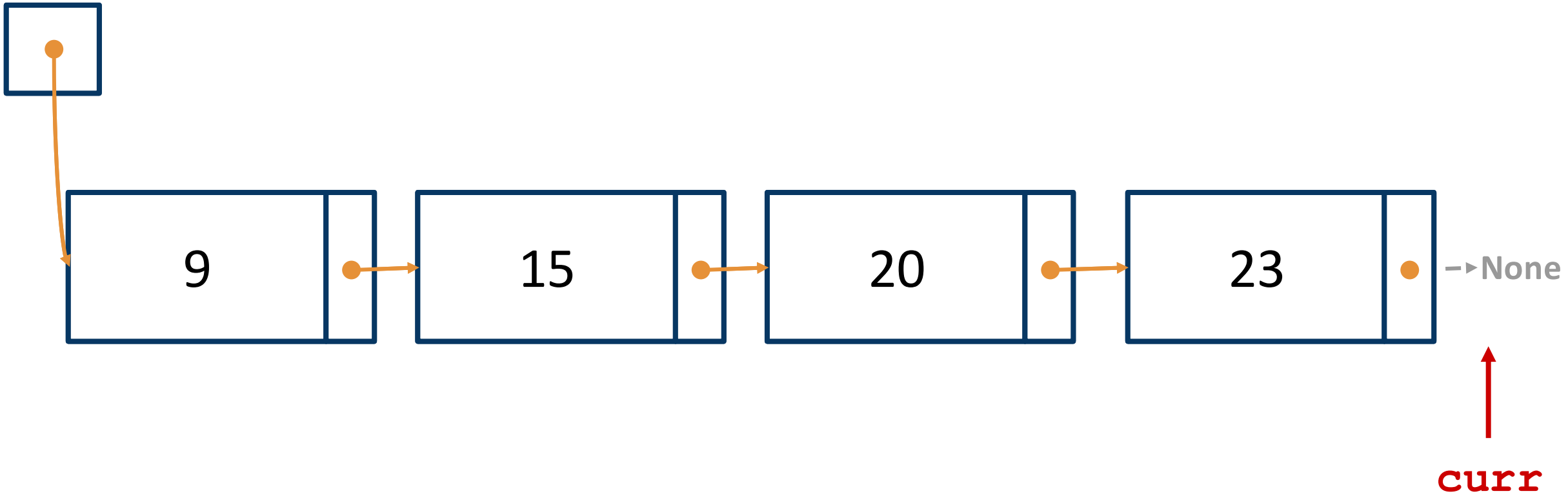
What if the item isn't in our list?

Recap: __contains__(**5**)



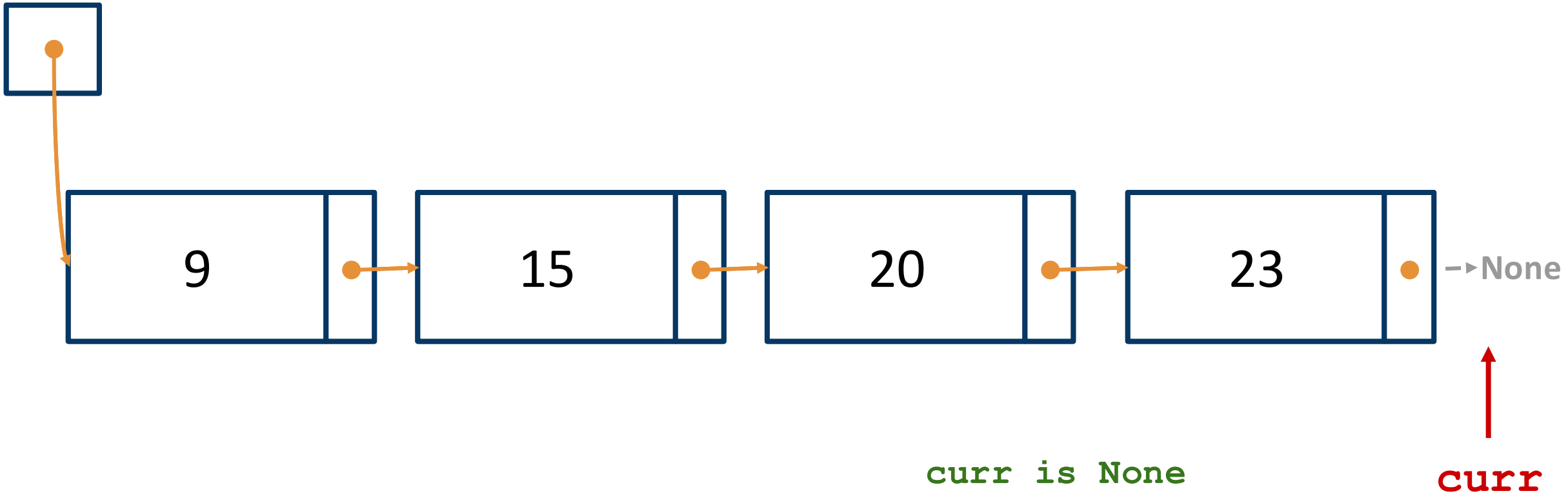
What if the item isn't in our list?

Recap: __contains__(**5**)



What if the item isn't in our list?

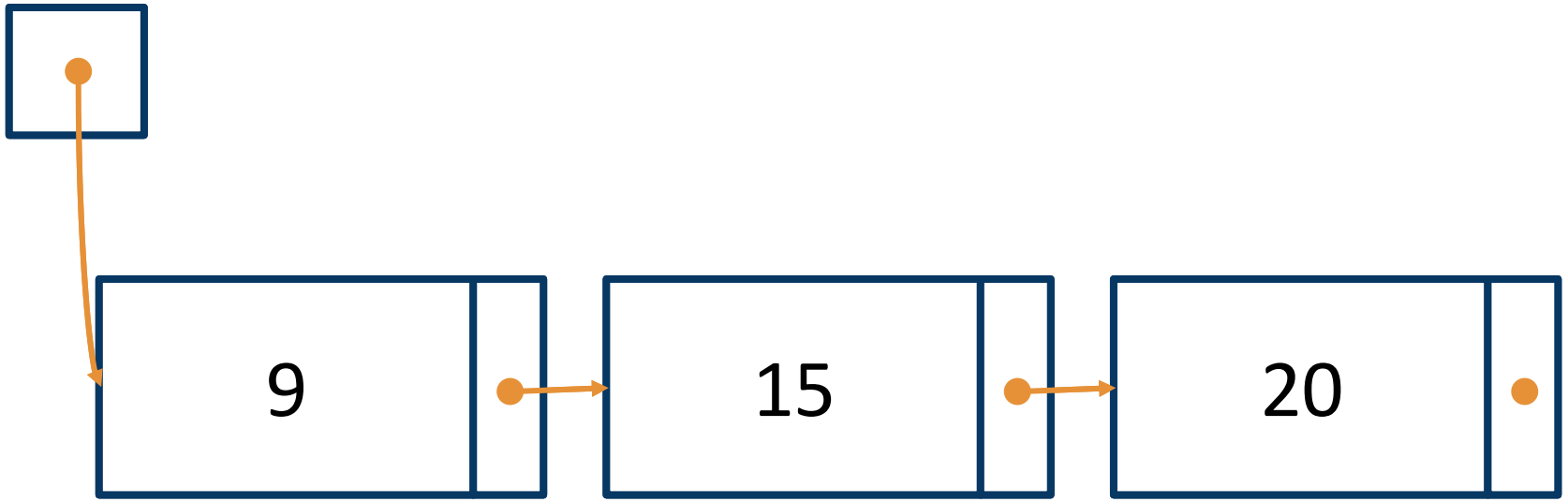
Recap: __contains__(**5**)



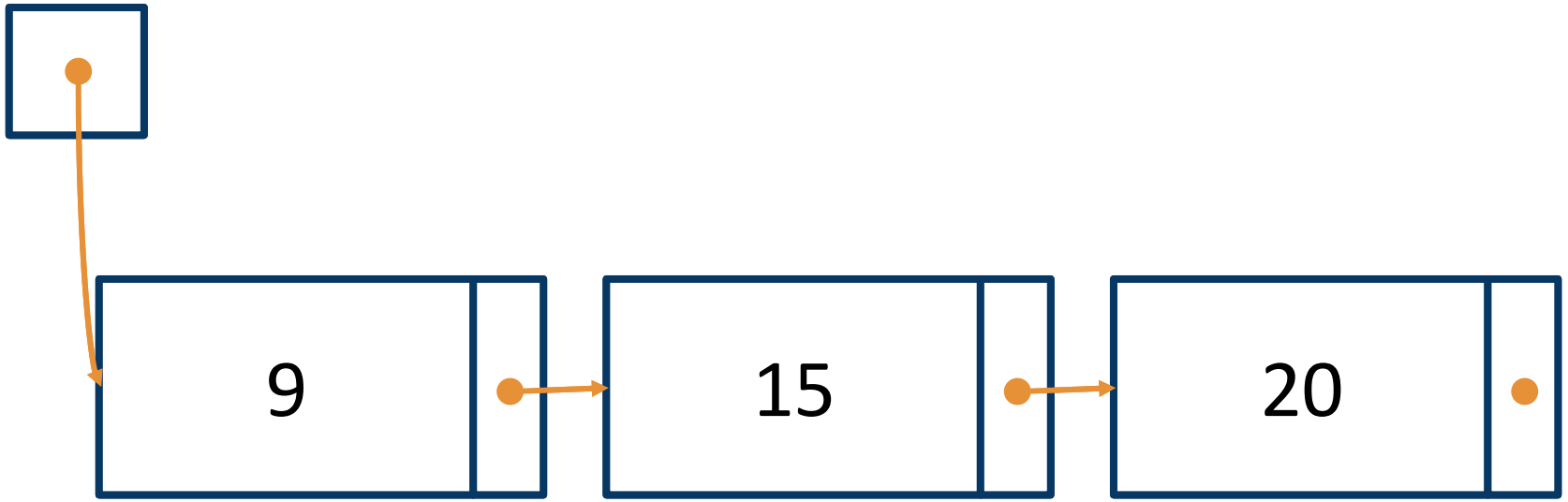
`curr is None`

We've reached the end of our LinkedList and didn't find our item, so return False.

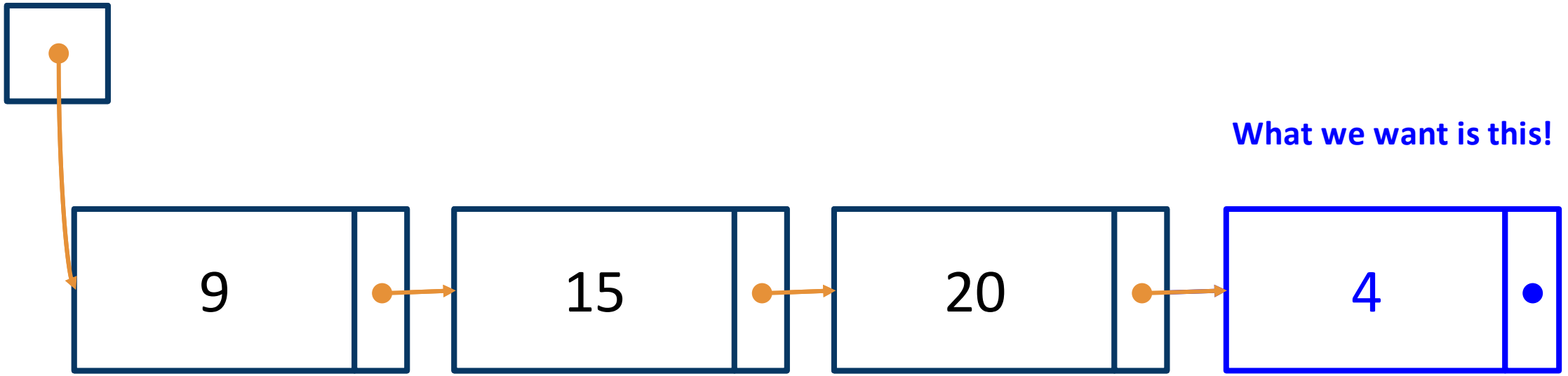
Recap: append



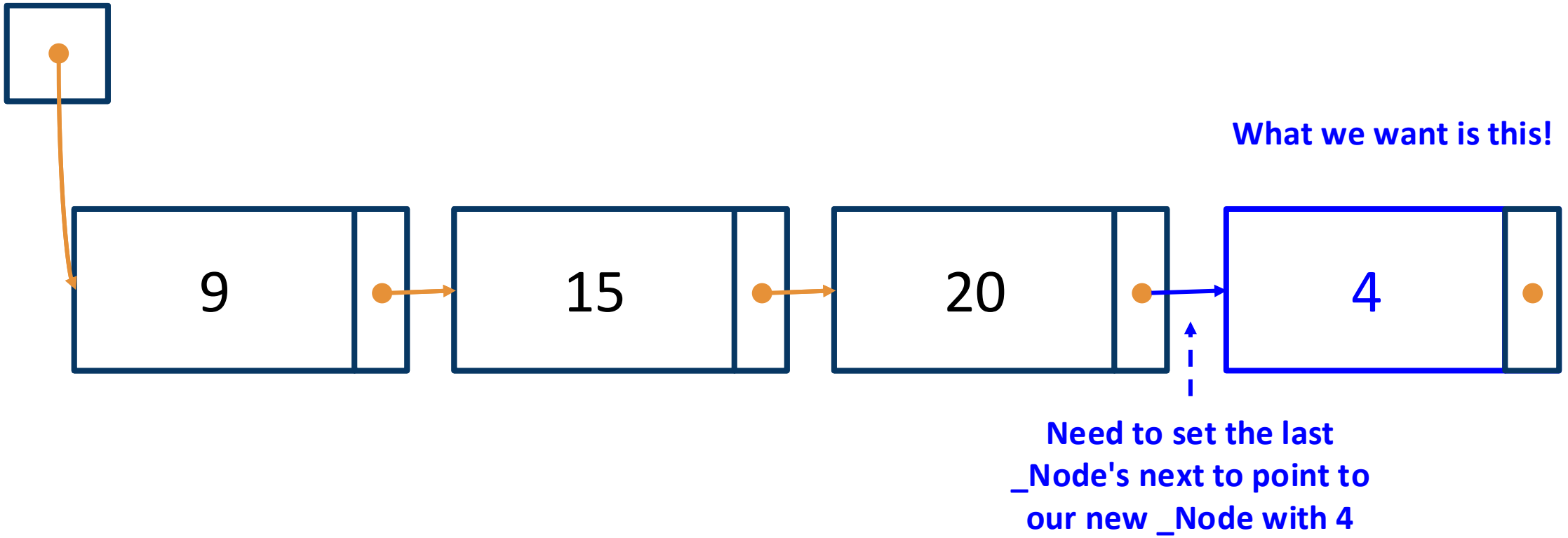
Recap: append(**4**)



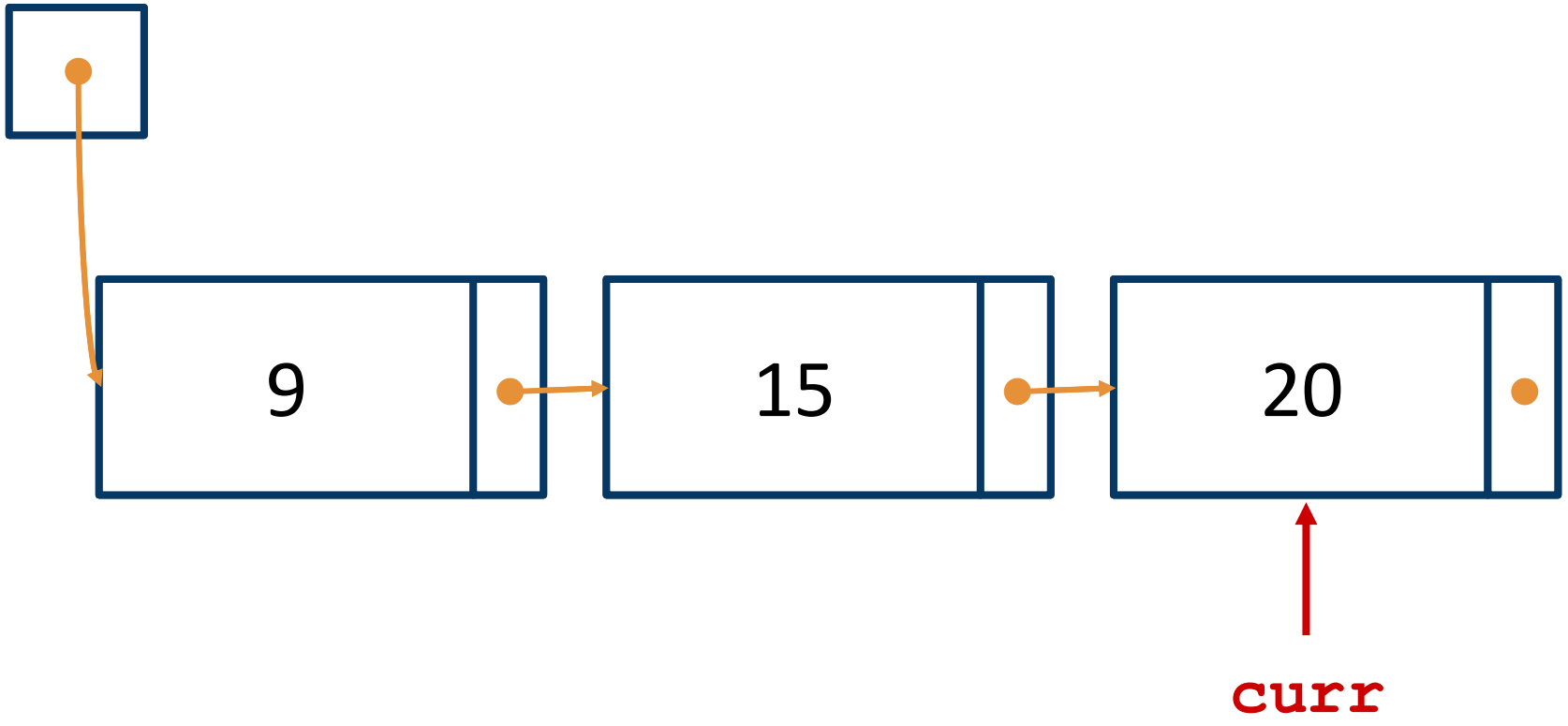
Recap: append(**4**)



Recap: append(**4**)

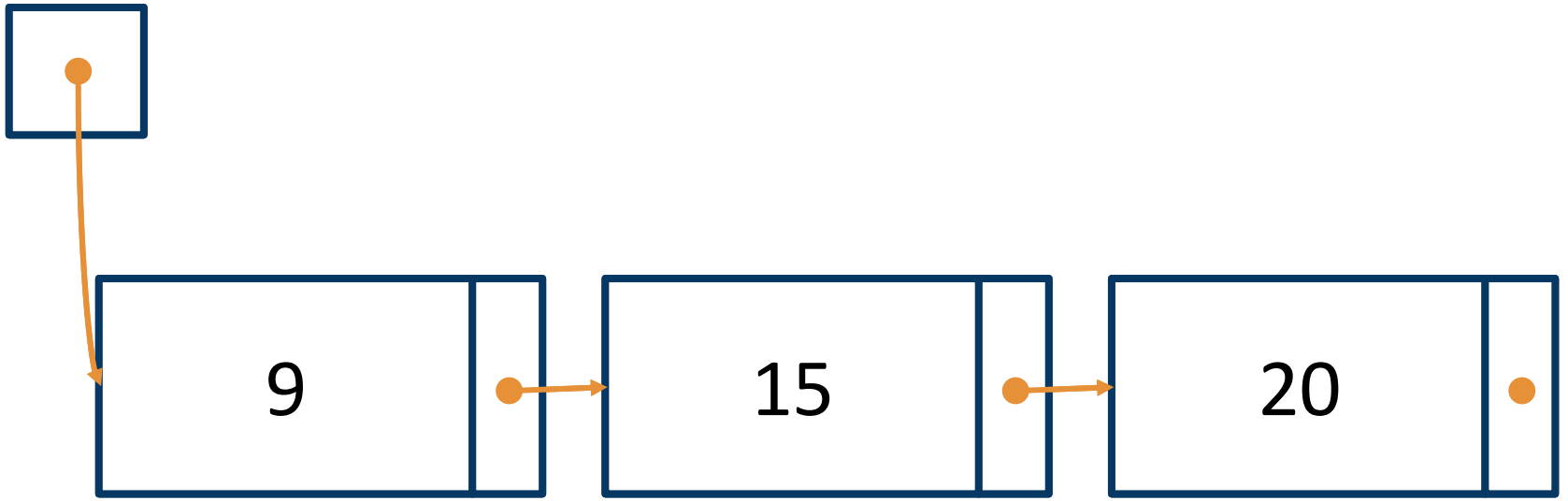


Recap: append(**4**)



If we loop until curr is None . . .

Recap: append(**4**)



`curr`

. . . We have no way to change the last _Node's next attribute!

While-loops

```
curr = self._first
while curr is not None:
    ... curr.item ...
    curr = curr.next

# At this point:
# curr is None
```

```
curr = self._first
while curr.next is not None:
    ... curr.item ...
    curr = curr.next

# At this point:
# curr.next is None
# (curr itself is a Node)
```

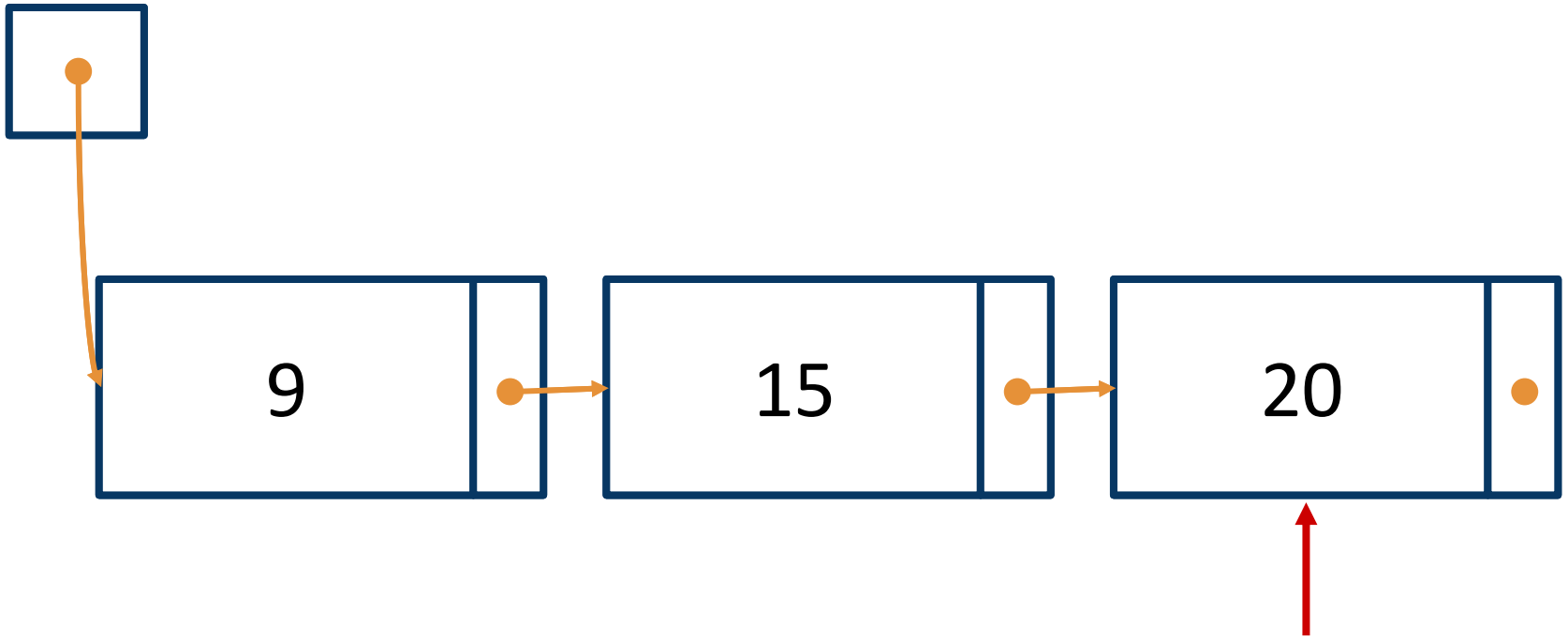
While-loops

Must be sure that curr (i.e. self._first) is not None!
Otherwise AttributeError from trying `curr.next`
(as that's equivalent to `None.next`)

```
curr = self._first
while curr is not None:
    ... curr.item ...
    curr = curr.next

# At this point:
# curr is None
```

```
curr = self._first
while curr.next is not None:
    ... curr.item ...
    curr = curr.next

# At this point:
# curr.next is None
# (curr itself is a Node)
```

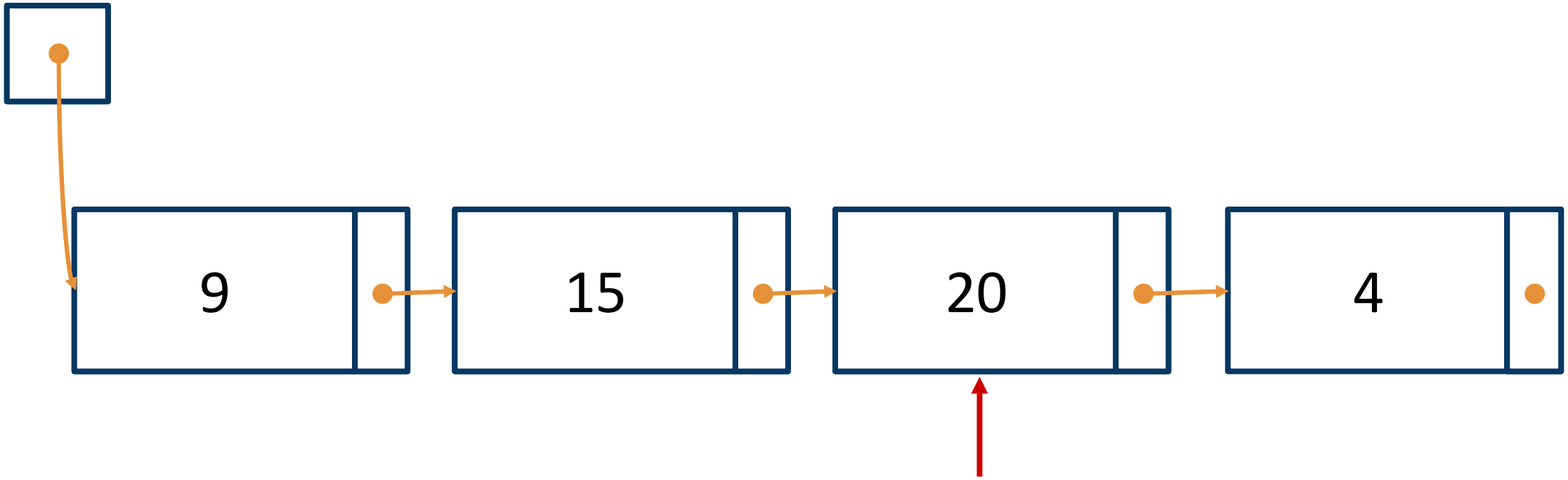
Recap: append(**4**)



`curr.next is None`

But curr is still a _Node!

Recap: append(**4**)




```
curr.next = _Node(4)
```

Takeaways

Code templates are useful.

Code templates aren’t everything.

Writing a stopping condition is often *easier to understand* than writing a loop condition.

Stopping condition

"Stop this loop when…"

- curr is None → our while-loop goes while curr is not None
  - Equivalently: not (curr is None)
- curr.next is None → while curr.next is not None
  - Equivalently: not (curr.next is None)

Write in whatever way is easiest for you!

While-loops

```
curr = self._first
while curr is not None:
    ... curr.item ...
    curr = c

# At this point:
# curr is None
```

```
curr = self._first
while curr.next is not None:
    ... curr.item ...
                    xt

# At this point:
# curr.next is None
# (curr itself is a Node)
```

The opposite of our while-loop condition is always true after the while-loop!

CSC148 - More practice with linked list traversals

Recall the basic Linked List traversal pattern:

```
curr = self._first        # Variable initialization
while curr is not None:   # Traversal loop
    ... curr.item ...
    curr = curr.next
```

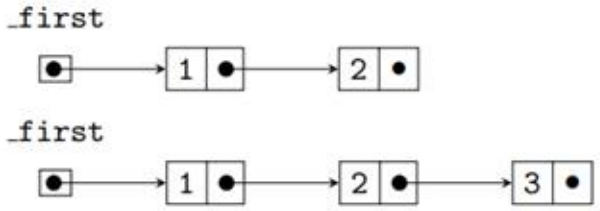
On this worksheet, you'll work on developing a method that modifies this basic pattern. In particular, this will involve changing the *while loop condition*, and you'll practice developing a non-trivial loop condition in a logical way.

1. Here is the docstring and implementation sections for the special method `LinkedList.__eq__`

```
def __eq__(self, other: LinkedList) -> bool:
    """Return whether this list and the other list are equal.

    Two lists are equal when each one has the same number of items,
    and each corresponding pair of items are equal (using == to compare).

    >>> lst1 = LinkedList([1, 2, 3])
    >>> lst2 = LinkedList([1, 2])
    >>> lst1 == lst2
    False
    """
    # (1) Variable initialization




    # (2) Traversal loop




    # (3) Post-loop code
```

Our goal is to implement this method. The second side of this worksheet walks you through how to develop the code for each step. As you complete each step, fill it into the method body above to complete the implementation.

You may find the following linked list diagrams useful when developing your code. These correspond to the doctest example.



Obviously, using just one variable `curr` is not enough to iterate through both lists. Instead, use two variables: `curr1` and `curr2`, one for each list. Show how to initialize these variables in the space below:

```
# (1) Variable initialization

curr1 =


curr2 =
```

Next, let's work on `(2)`, the traversal loop.

(a) First, let's think about when we want the loop to *stop*. This should be when we reach the end of `self` or the end of `other`. Write down a Python expression involving `curr1` and `curr2` that expresses this *stopping condition*. (By "expresses", we mean your expression should evaluate to `True` when the condition is true, and `False` otherwise.)

(b) The while loop condition will then be the *negation* of the stopping condition. Write down a Python expression for the while loop condition.

Using this, write down the while loop you would use to traverse the two lists. At each iteration, you should compare `curr1.item` against `curr2.item`; if this is False, you should be able to stop and return, without checking any other values!

```
# (2) Traversal loop
```

Finally, suppose we reach the end of the loop. We need some code to handle this case as well.

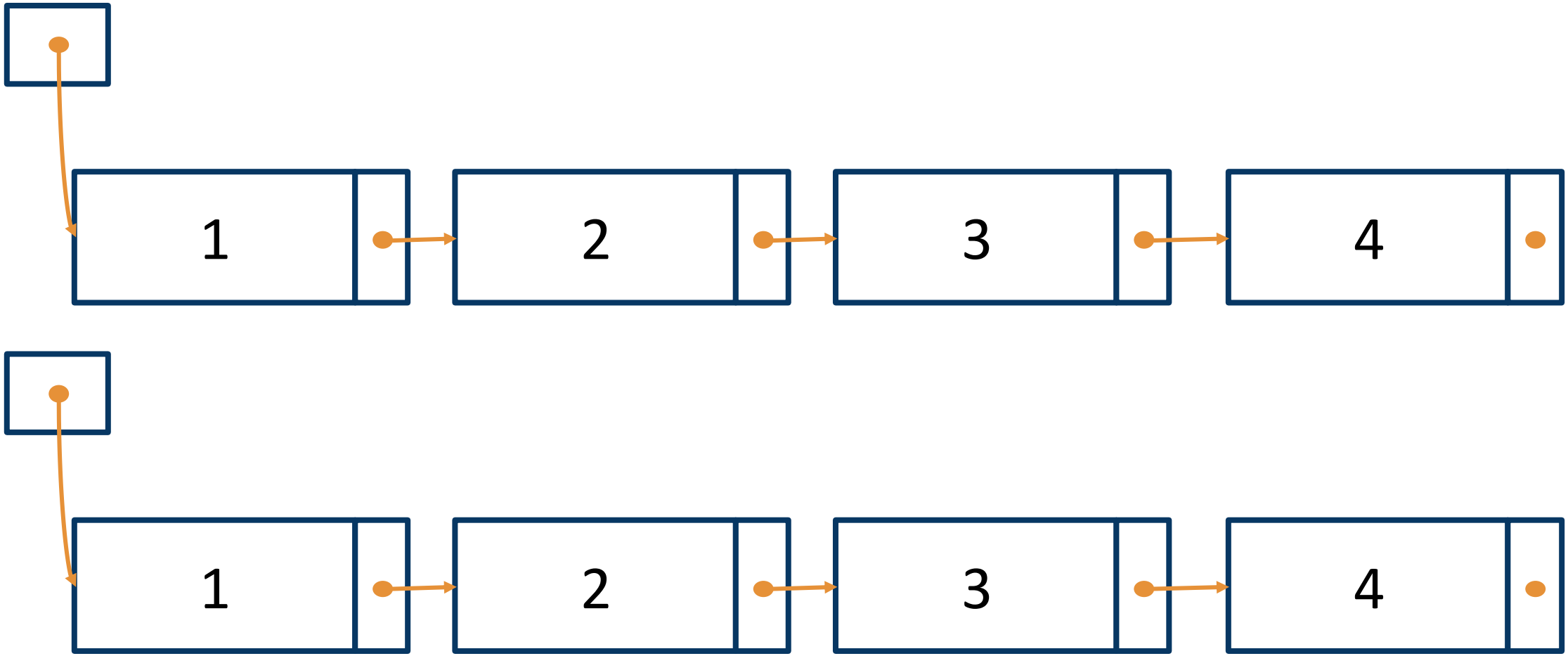
(a) What do we know about `curr1` and/or `curr2` after the loop ends? (Hint: look at your stopping condition.)

(b) How can we use the values of `curr1` and `curr2` to check whether the lists have the same length?

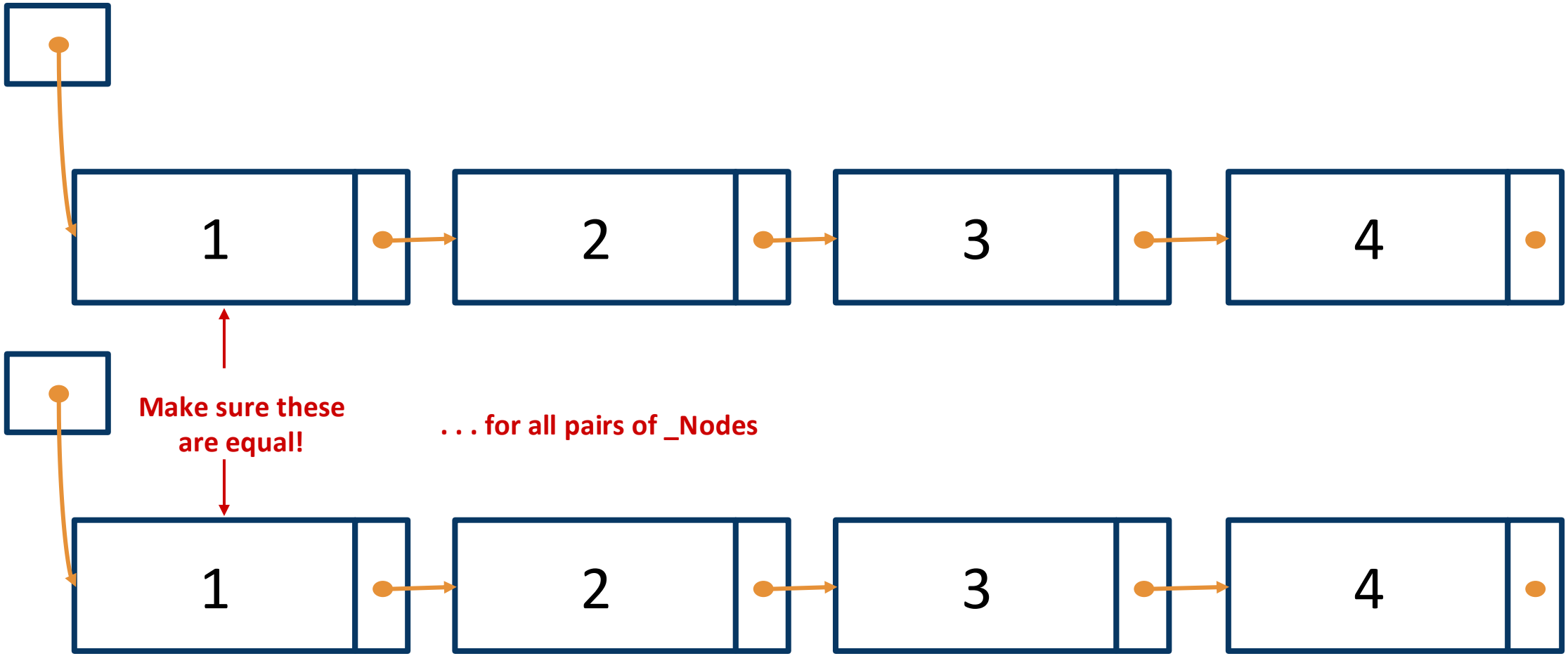
(c) Write the code that should go after the end of our loop. Remember that it should return `True` or `False`.

```
# (3) Post-loop code
```

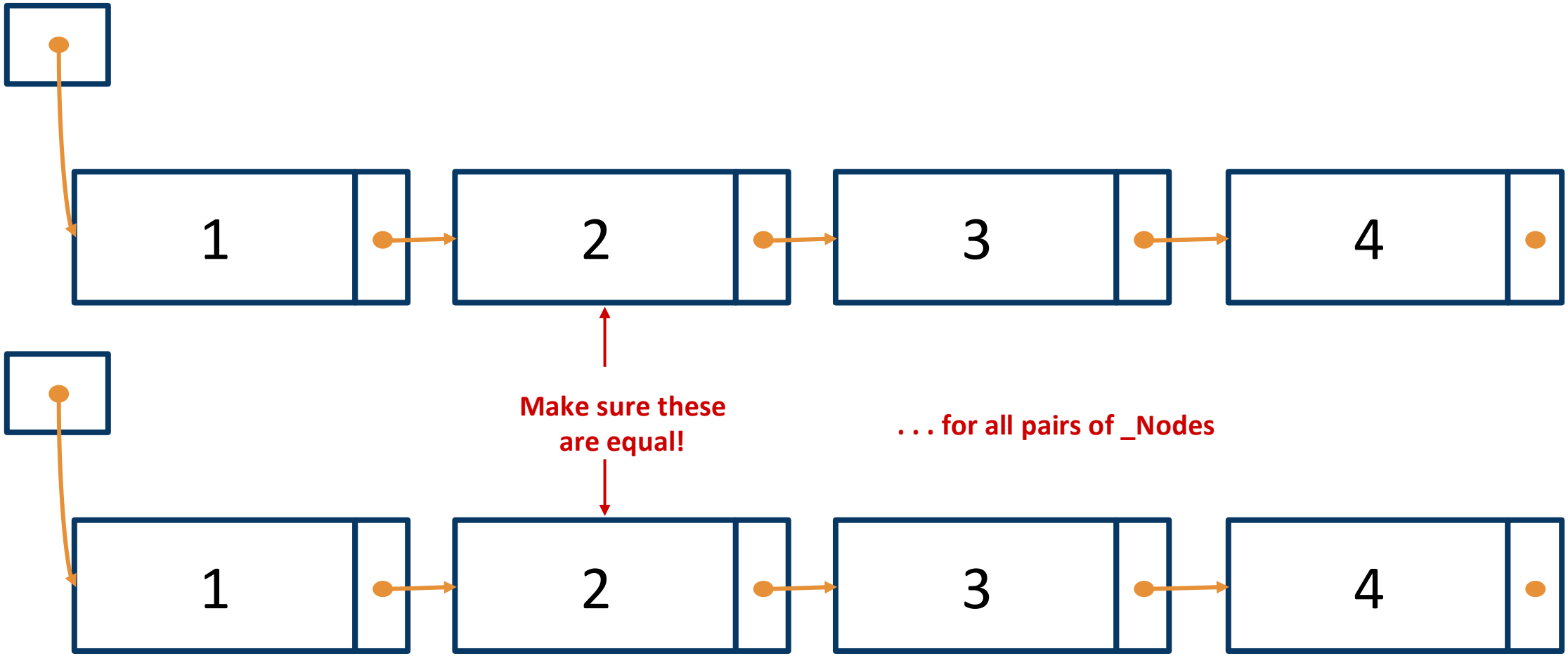
__eq__ concept



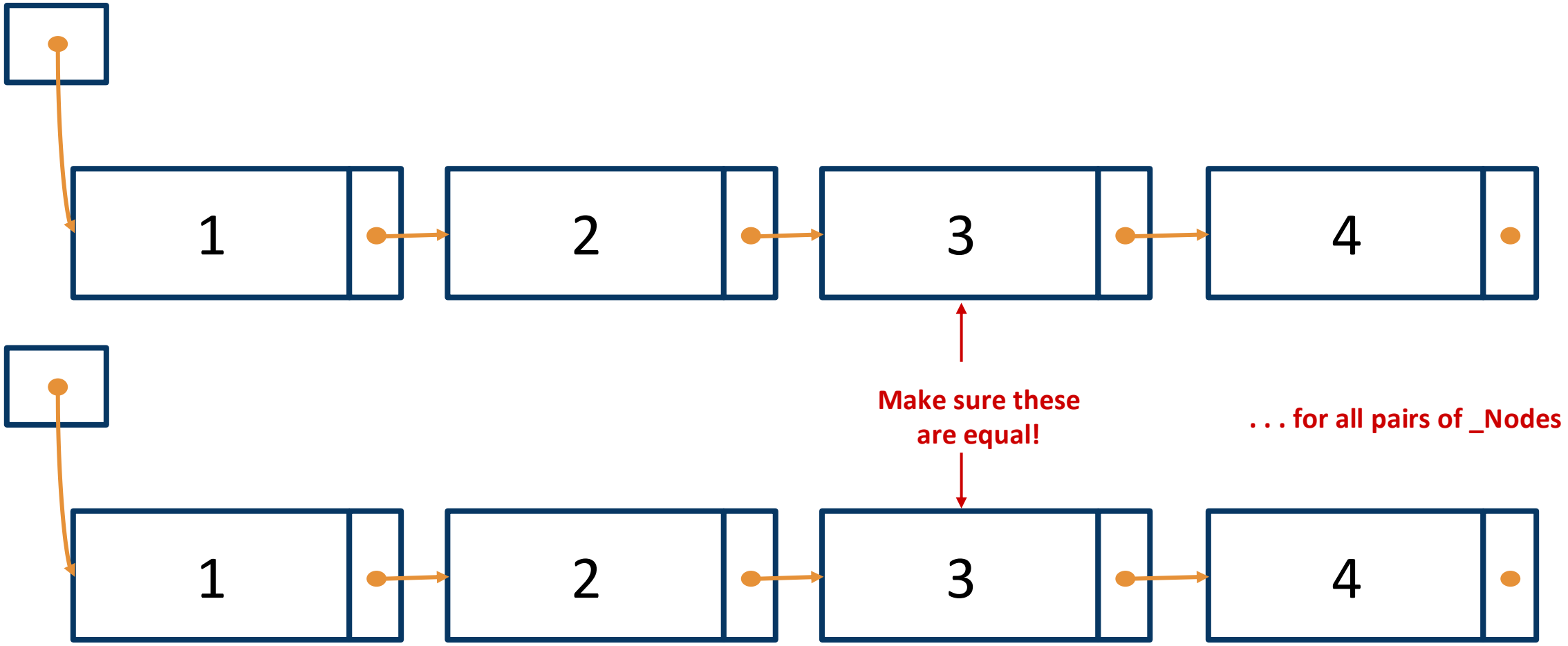
__eq__ concept



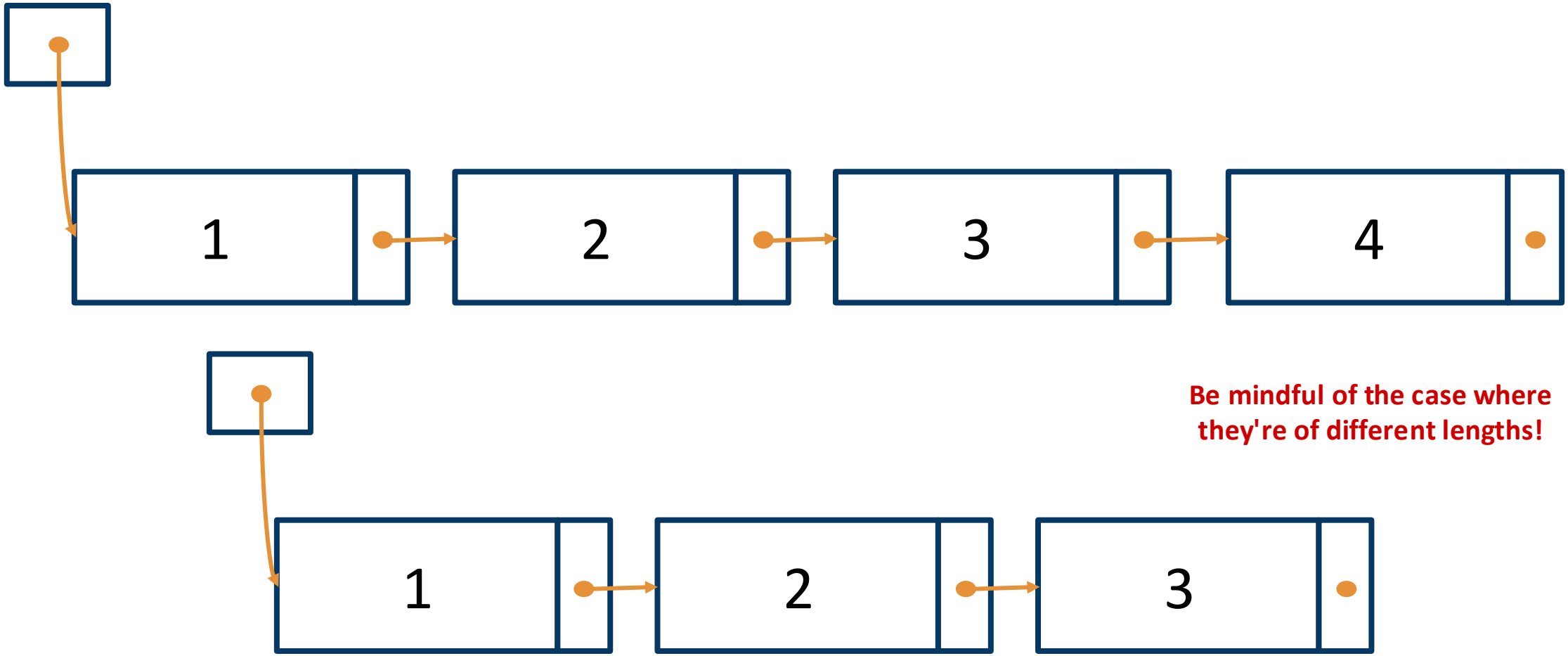
__eq__ concept



__eq__ concept



__eq__ concept



Be mindful of the case where they're of different lengths!

CSC148 - More practice with linked list traversals

Recall the basic Linked List traversal pattern:

```
curr = self._first        # Variable initialization
while curr is not None:   # Traversal loop
    ... curr.item ...
    curr = curr.next
```

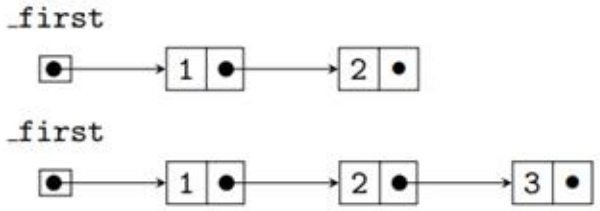
On this worksheet, you'll work on developing a method that modifies this basic pattern. In particular, this will involve changing the *while loop condition*, and you'll practice developing a non-trivial loop condition in a logical way.

1. Here is the docstring and implementation sections for the special method `LinkedList.__eq__`

```
def __eq__(self, other: LinkedList) -> bool:
    """Return whether this list and the other list are equal.

    Two lists are equal when each one has the same number of items,
    and each corresponding pair of items are equal (using == to compare).

    >>> lst1 = LinkedList([1, 2, 3])
    >>> lst2 = LinkedList([1, 2])
    >>> lst1 == lst2
    False
    """
    # (1) Variable initialization




    # (2) Traversal loop




    # (3) Post-loop code
```

Our goal is to implement this method. The second side of this worksheet walks you through how to develop the code for each step. As you complete each step, fill it into the method body above to complete the implementation.

You may find the following linked list diagrams useful when developing your code. These correspond to the doctest example.



Obviously, using just one variable `curr` is not enough to iterate through both lists. Instead, use two variables: `curr1` and `curr2`, one for each list. Show how to initialize these variables in the space below:

```
# (1) Variable initialization

curr1 =


curr2 =
```

Next, let's work on `(2)`, the traversal loop.

(a) First, let's think about when we want the loop to *stop*. This should be when we reach the end of `self` or the end of `other`. Write down a Python expression involving `curr1` and `curr2` that expresses this *stopping condition*. (By "expresses", we mean your expression should evaluate to `True` when the condition is true, and `False` otherwise.)

(b) The while loop condition will then be the *negation* of the stopping condition. Write down a Python expression for the while loop condition.

Using this, write down the while loop you would use to traverse the two lists. At each iteration, you should compare `curr1.item` against `curr2.item`; if this is False, you should be able to stop and return, without checking any other values!

```
# (2) Traversal loop
```

Finally, suppose we reach the end of the loop. We need some code to handle this case as well.

(a) What do we know about `curr1` and/or `curr2` after the loop ends? (Hint: look at your stopping condition.)

(b) How can we use the values of `curr1` and `curr2` to check whether the lists have the same length?

(c) Write the code that should go after the end of our loop. Remember that it should return `True` or `False`.

```
# (3) Post-loop code
```

Linked list insertion and deletion

Related Reading: 6.3 Linked List Mutation

```
def insert(self, index: int, item: Any) -> None:
    """Insert the given item at the given index.

    Raise IndexError if index > len(self)
    or index < 0.
    Adding to the end of the list is okay.
    """
```

CSC148 - Linked Lists

Here is the specification of the `LinkedList` and `_Node` classes (attribute descriptions omitted).

```
from __future__ import annotations
from typing import Any

class _Node:
    """A node in a linked list."""
    item: Any
    next: _Node | None
```

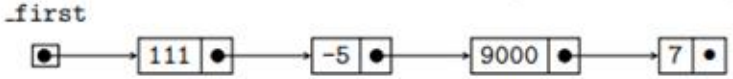
```
class LinkedList:
    """A linked list implementation of
    the List ADT.
    """
    _first: _Node | None

    def __init__(self) -> None:
        """Initialize an empty linked list."""
```

Part 1: Index-based traversal

When traversing a linked list, we may also want to keep track of the current index as shown by the code template below:

```
curr = self._first
curr_index = 0

while ... curr is not None ... :
    ... curr.item ...  # Do something with curr.item and/or curr_index

    curr = curr.next
    curr_index += 1
```

Using this pattern, implement the following `LinkedList` method. Hint: Recall the ideas from the last worksheet to develop your stopping condition. Do not call any other linked list methods (e.g., `__len__`).
You may find it useful to refer to this linked list diagram to trace through the logic of your code.




```
class LinkedList:
    def __getitem__(self, i: int) -> Any:
        """Return the item stored at index i in this linked list.
        Raise an IndexError if index i is out of bounds.

        Preconditions:
        - i >= 0
        """
```

Part 2: Index-based insertion

Our next goal is to extend `LinkedList` by implementing a standard mutating List method: inserting into a list by index.

```
class LinkedList:
    def insert(self, i: int, item: Any) -> None:
        """Insert the given item at index i in this list.
        Raise IndexError if i > len(self). Adding to the end of the list (i == len(self)) is okay.

        Preconditions:
        - i >= 0
        """
```

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

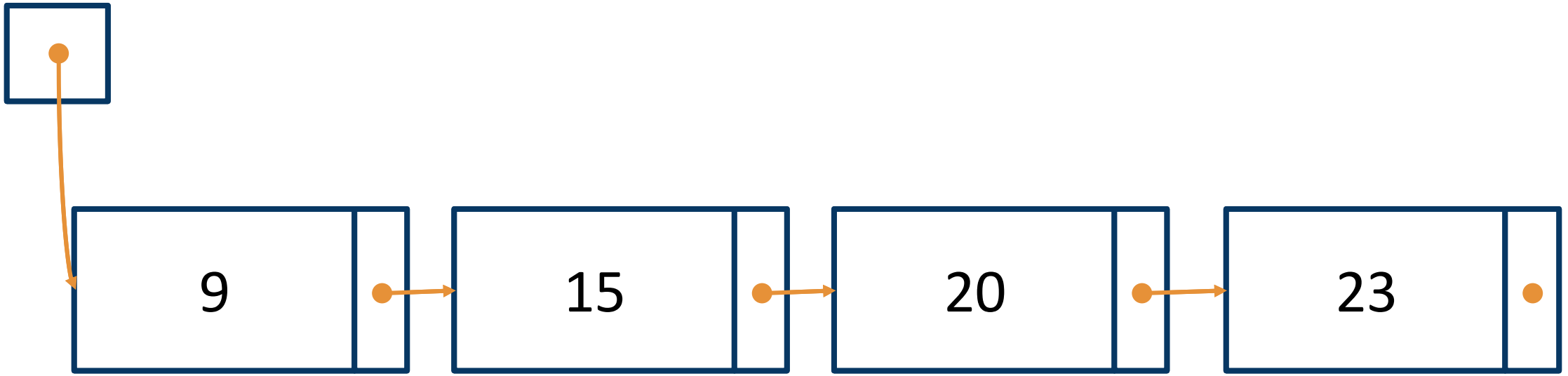
len(self)	i	self
0	0	_first → ⊡
1	0	_first → 111
1	1	_first → 111
4	0	_first → 111 → -5 → 9000 → 7
4	2	_first → 111 → -5 → 9000 → 7
4	4	_first → 111 → -5 → 9000 → 7

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.
   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.
2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:
   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

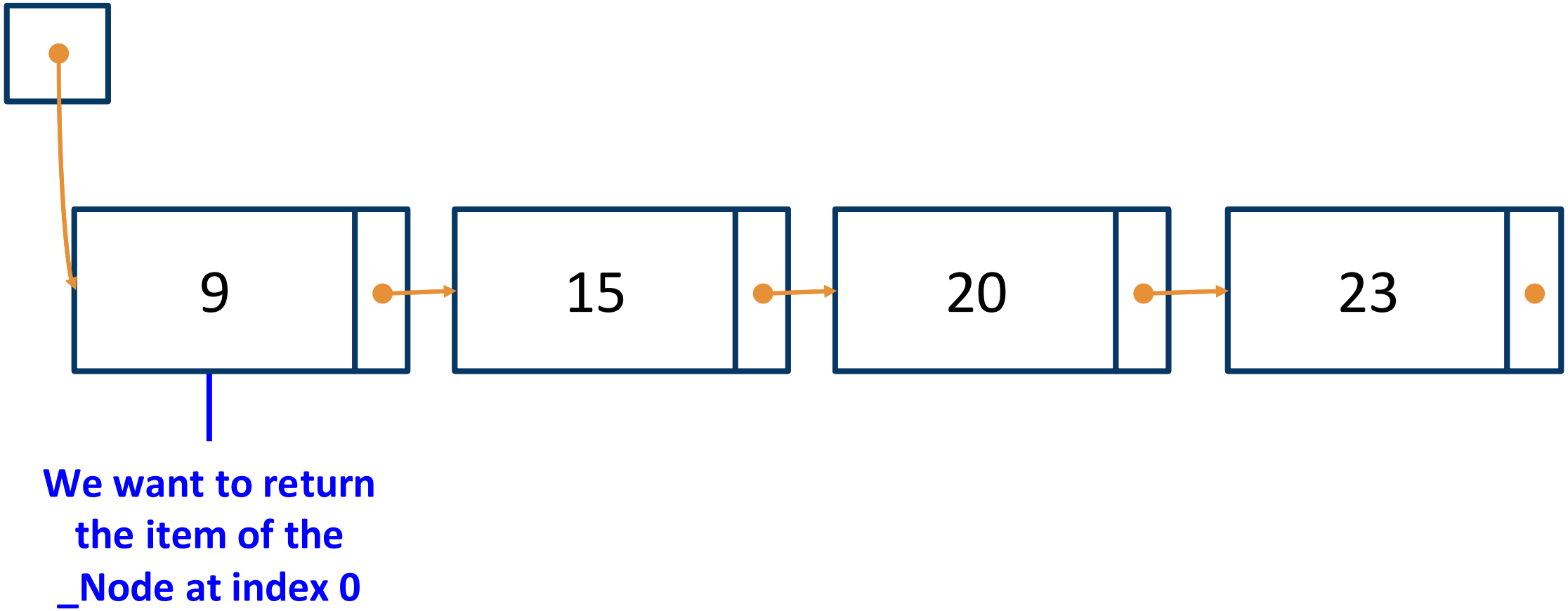
   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

3. Finally, using these ideas, implement the `LinkedList.insert` method on a separate piece of paper. You should have two cases: one for when you need to reassign `self._first`, and one where you don't.

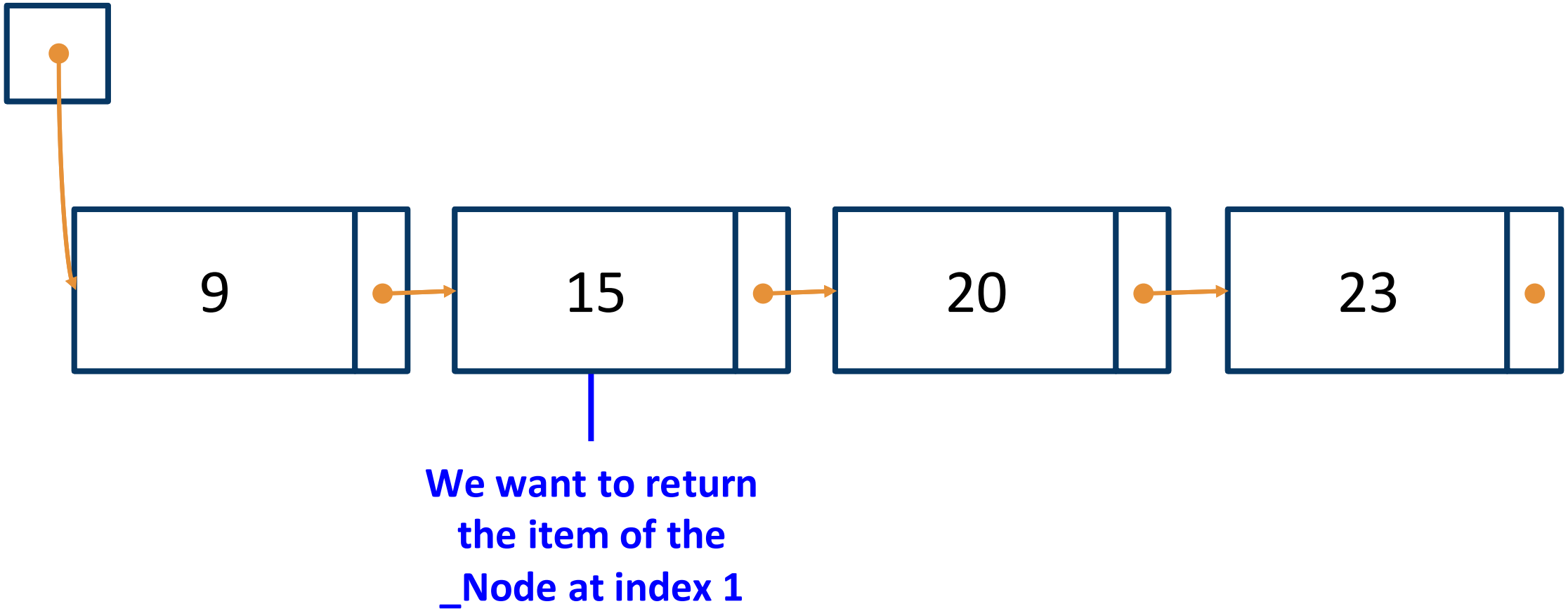
__getitem__ concept



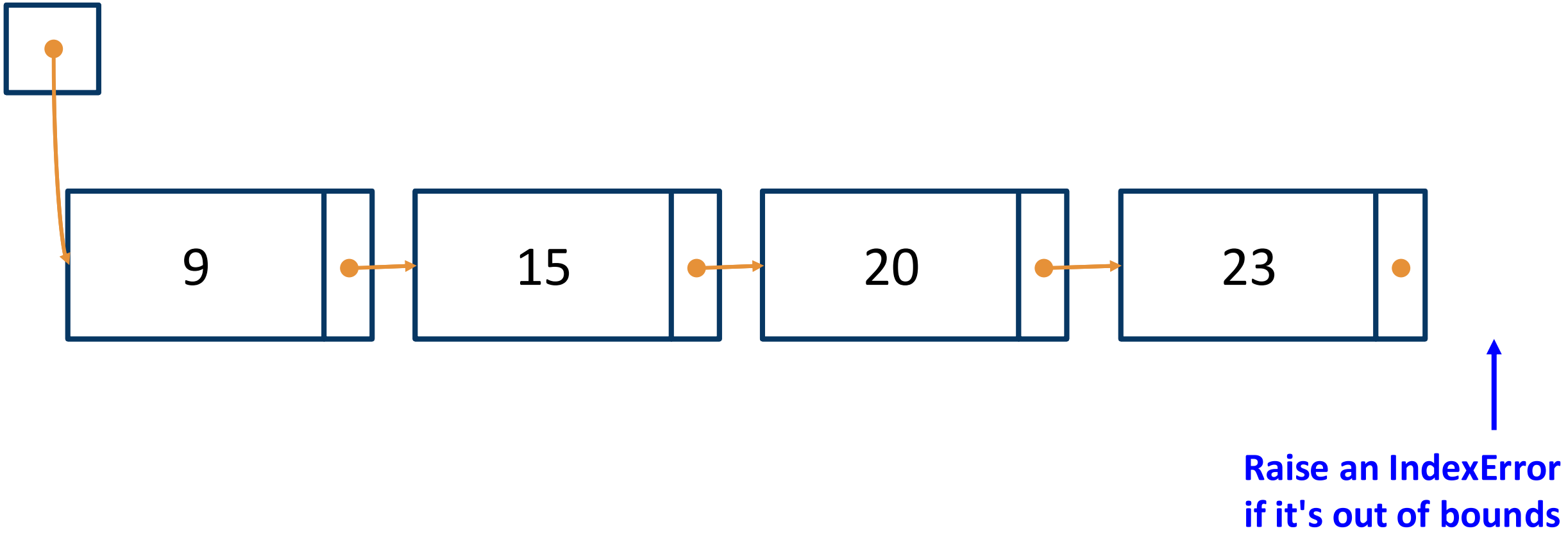
__getitem__(**0**) concept



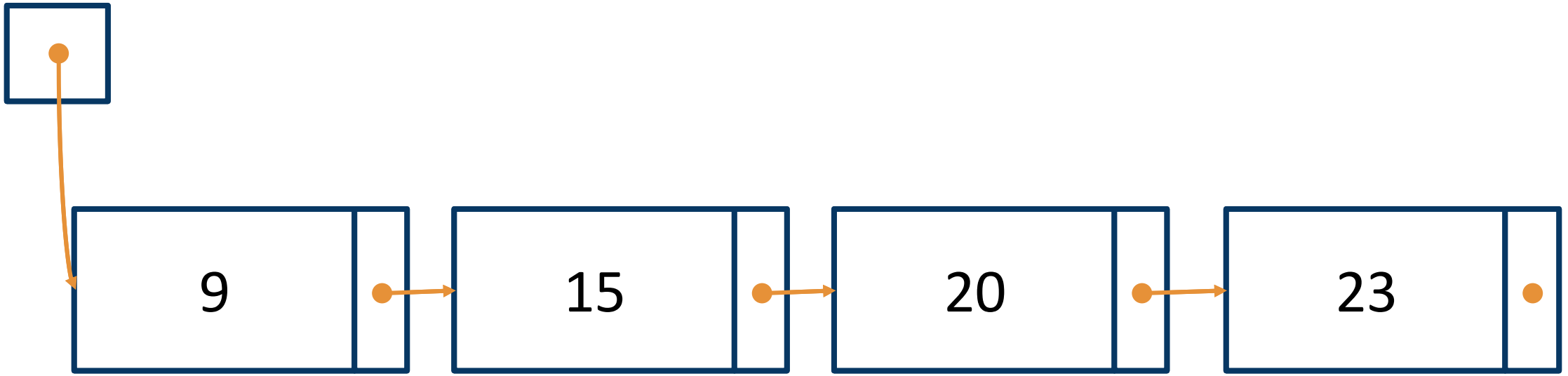
__getitem__(**1**) concept



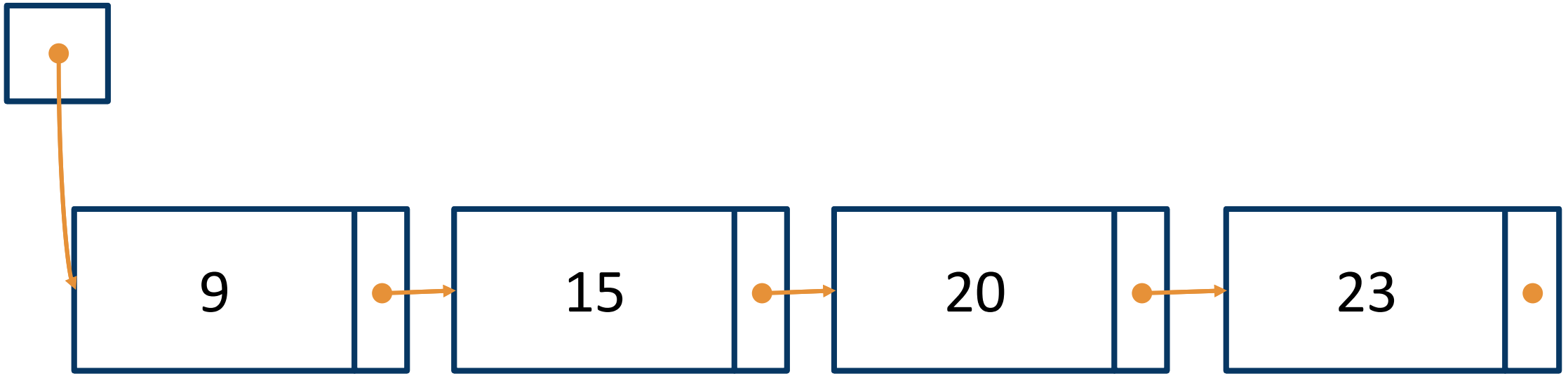
__getitem__(7) concept



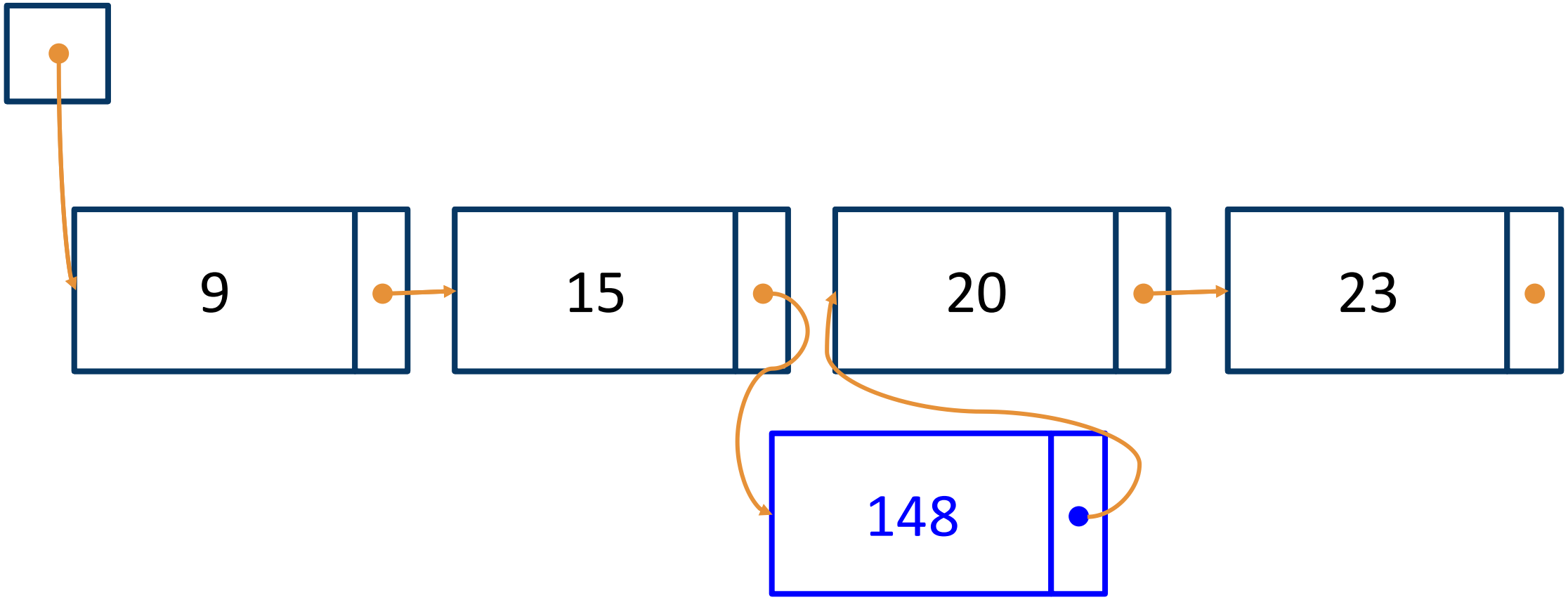
insert concept



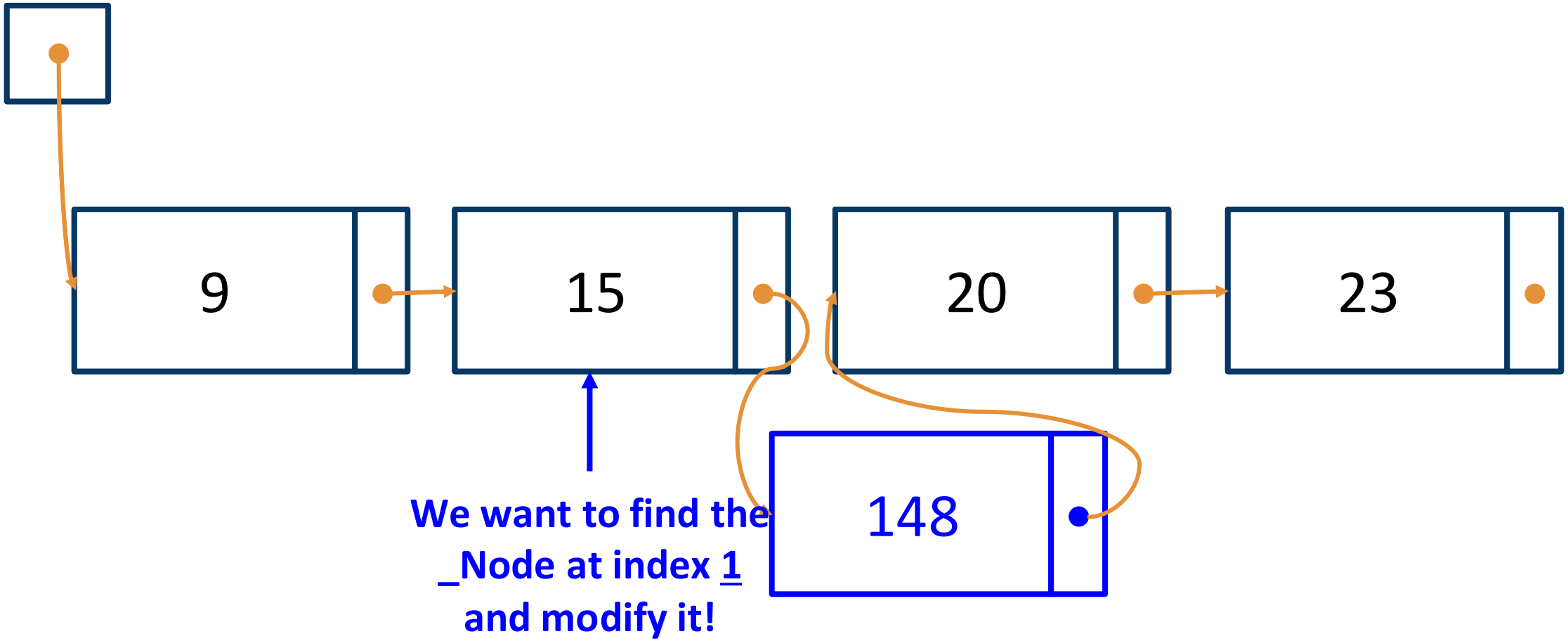
insert(2, 148) concept



insert(2, 148) concept



insert(2, 148) concept



CSC148 - Linked Lists

Here is the specification of the `LinkedList` and `_Node` classes (attribute descriptions omitted).

```
from __future__ import annotations
from typing import Any

class _Node:
    """A node in a linked list."""
    item: Any
    next: _Node | None
```

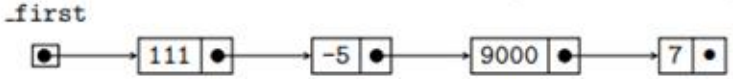
```
class LinkedList:
    """A linked list implementation of
    the List ADT.
    """
    _first: _Node | None

    def __init__(self) -> None:
        """Initialize an empty linked list."""
```

Part 1: Index-based traversal

When traversing a linked list, we may also want to keep track of the current index as shown by the code template below:

```
curr = self._first
curr_index = 0

while ... curr is not None ... :
    ... curr.item ...  # Do something with curr.item and/or curr_index

    curr = curr.next
    curr_index += 1
```

Using this pattern, implement the following `LinkedList` method. Hint: Recall the ideas from the last worksheet to develop your stopping condition. Do not call any other linked list methods (e.g., `__len__`).
You may find it useful to refer to this linked list diagram to trace through the logic of your code.




```
class LinkedList:
    def __getitem__(self, i: int) -> Any:
        """Return the item stored at index i in this linked list.
        Raise an IndexError if index i is out of bounds.

        Preconditions:
        - i >= 0
        """
```

Part 2: Index-based insertion

Our next goal is to extend `LinkedList` by implementing a standard mutating List method: inserting into a list by index.

```
class LinkedList:
    def insert(self, i: int, item: Any) -> None:
        """Insert the given item at index i in this list.
        Raise IndexError if i > len(self). Adding to the end of the list (i == len(self)) is okay.

        Preconditions:
        - i >= 0
        """
```

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	_first → ⊡
1	0	_first → 111
1	1	_first → 111
4	0	_first → 111 → -5 → 9000 → 7
4	2	_first → 111 → -5 → 9000 → 7
4	4	_first → 111 → -5 → 9000 → 7

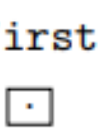
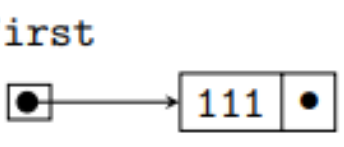
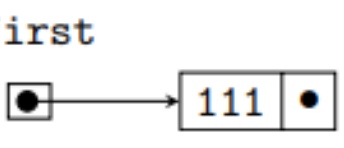
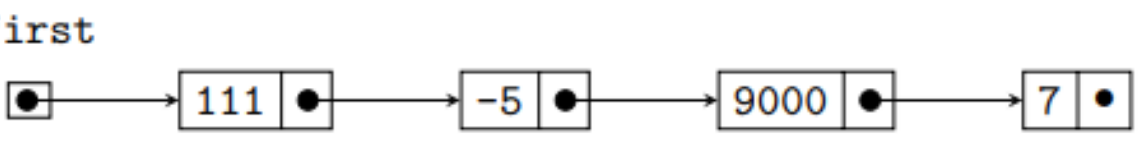
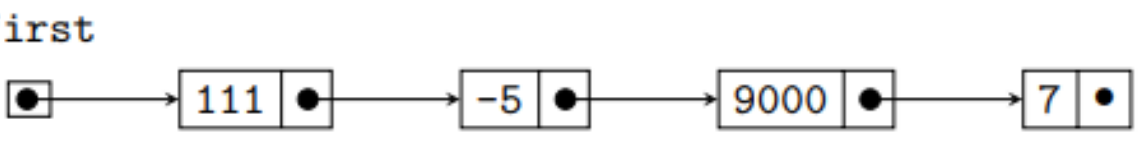
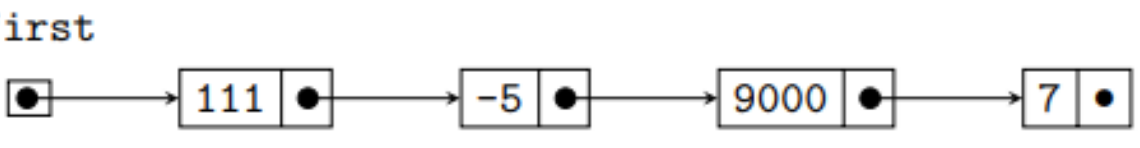
1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.
   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.
2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:
   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

3. Finally, using these ideas, implement the `LinkedList.insert` method on a separate piece of paper. You should have two cases: one for when you need to reassign `self._first`, and one where you don't.

Taking up __getitem__ in Pycharm

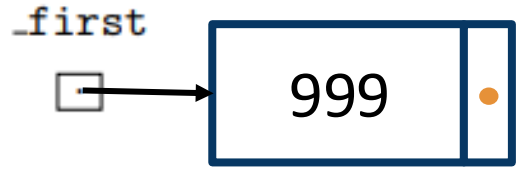
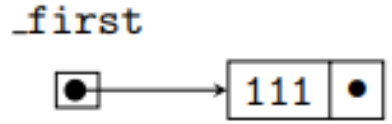
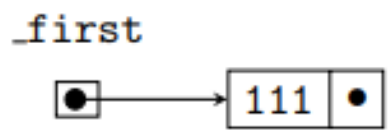
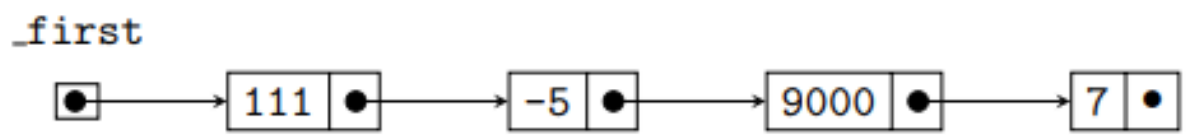
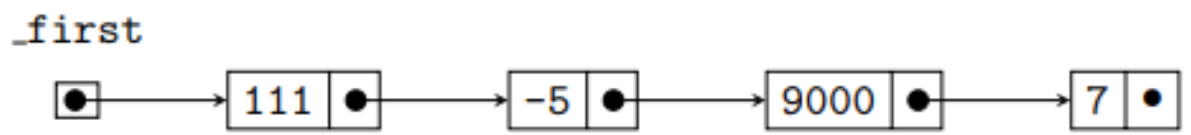
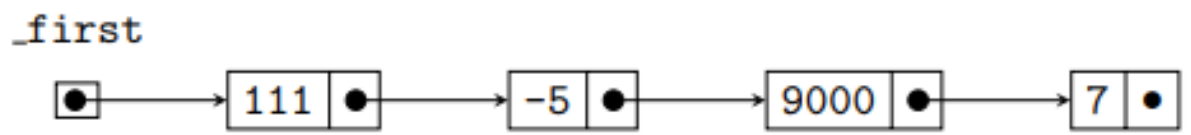
Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

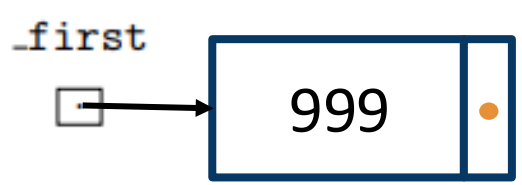
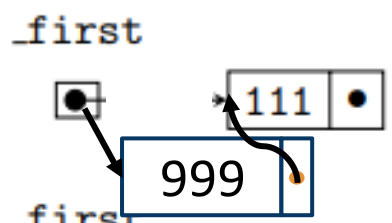
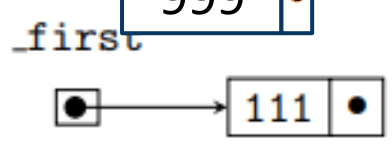
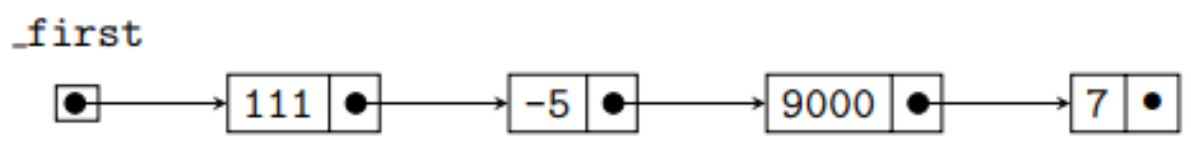
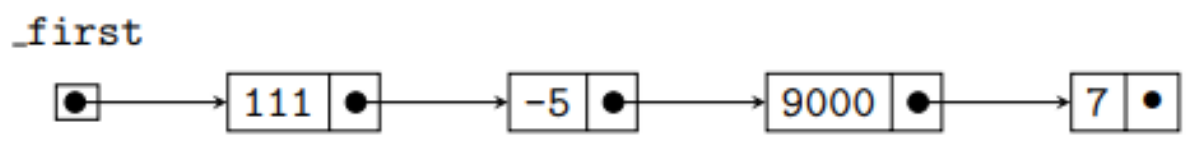
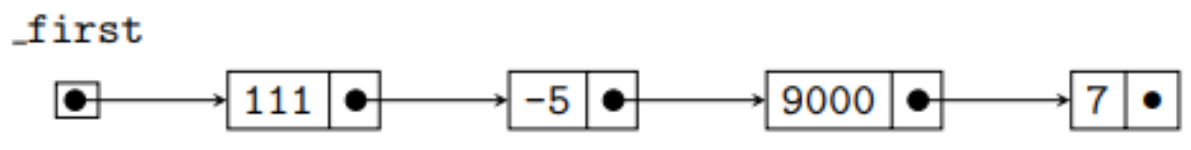
Consider the operation of inserting the value 999 into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after 999 is inserted.

   You should draw a new node containing 999. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

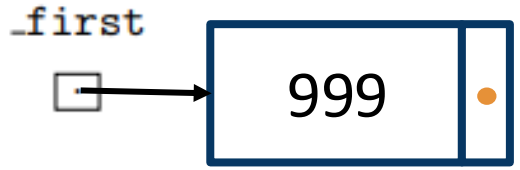
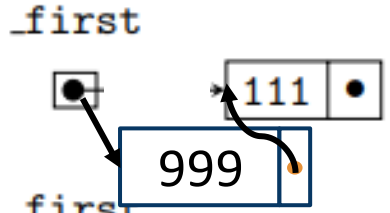
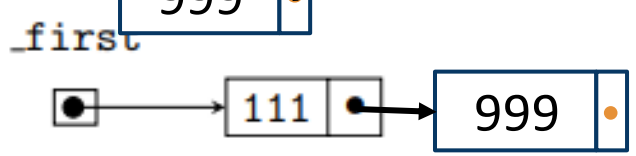
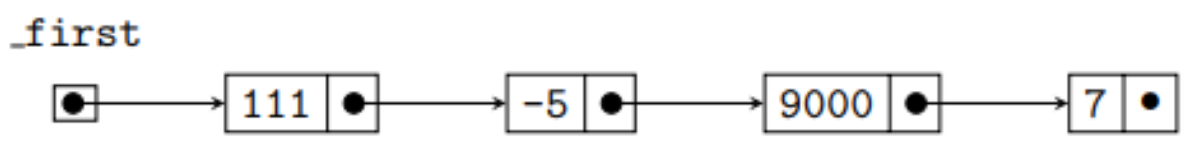
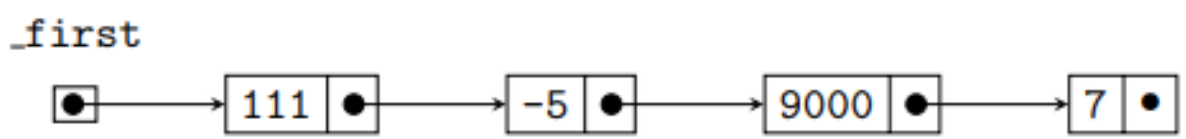
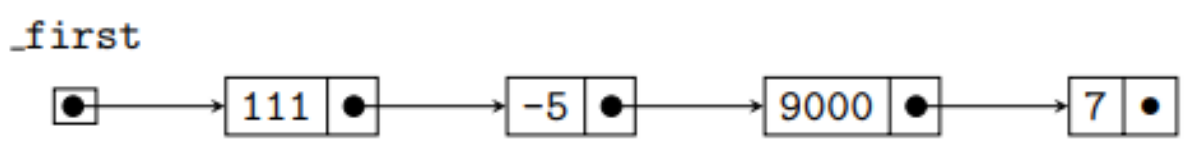
Consider the operation of inserting the value 999 into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after 999 is inserted.

   You should draw a new node containing 999. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

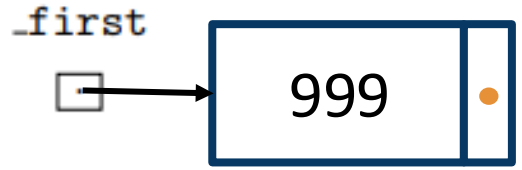
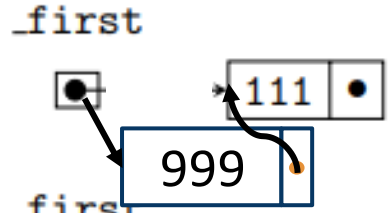
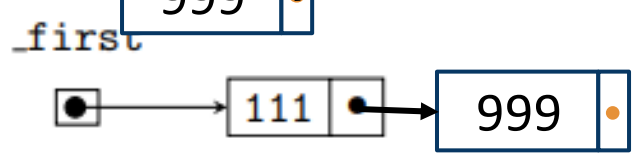
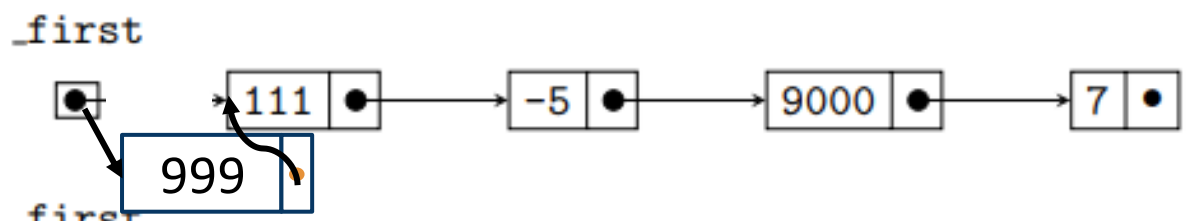
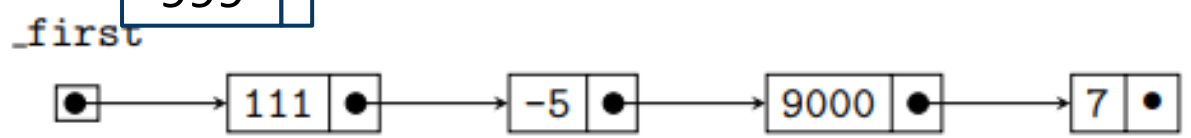
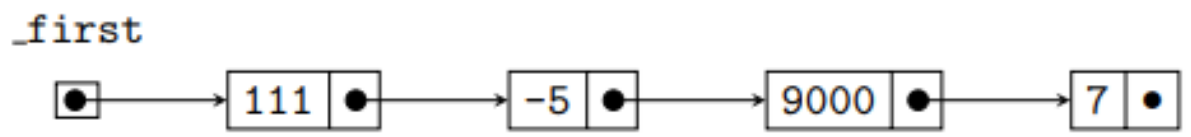
Consider the operation of inserting the value 999 into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after 999 is inserted.

   You should draw a new node containing 999. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

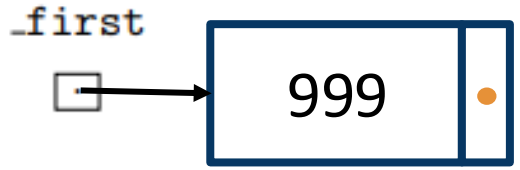
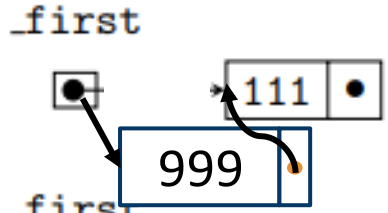
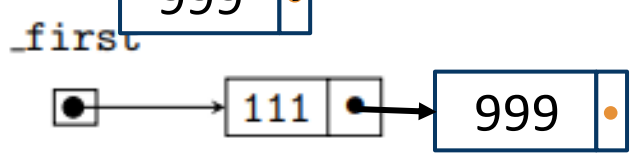
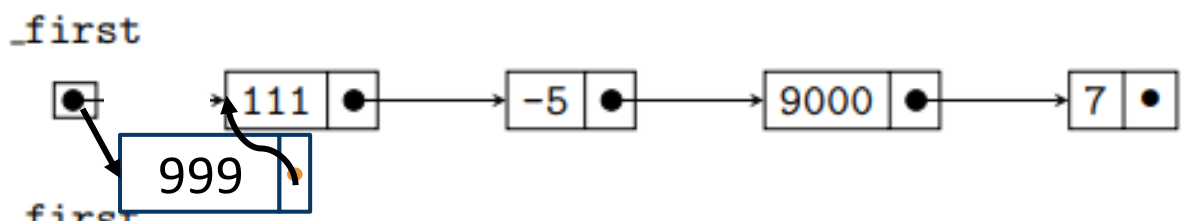
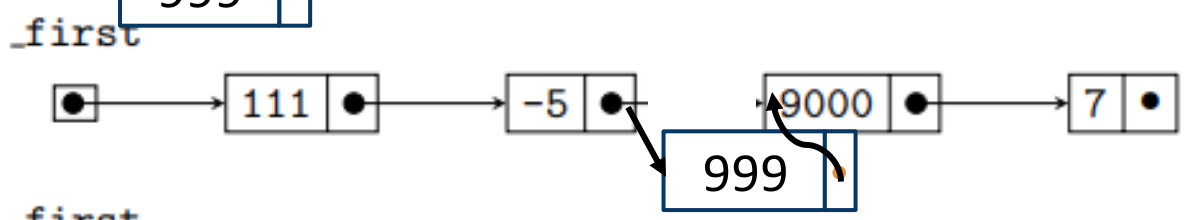
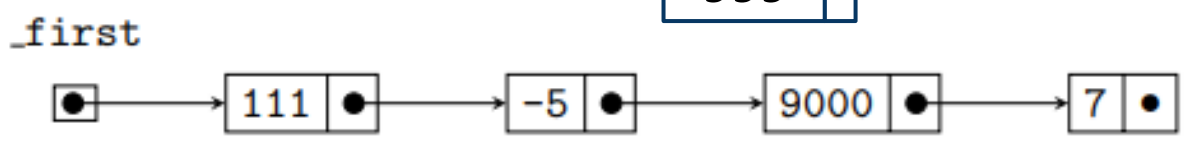
Consider the operation of inserting the value 999 into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after 999 is inserted.

   You should draw a new node containing 999. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

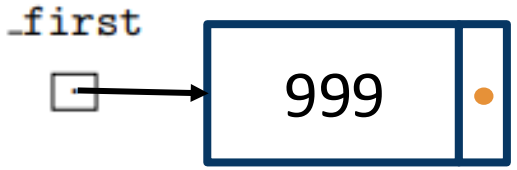
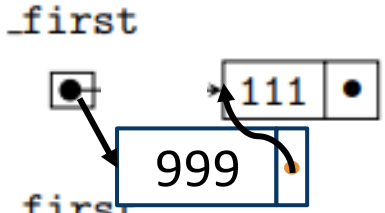
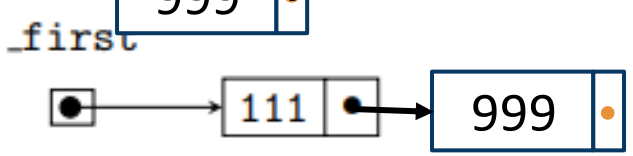
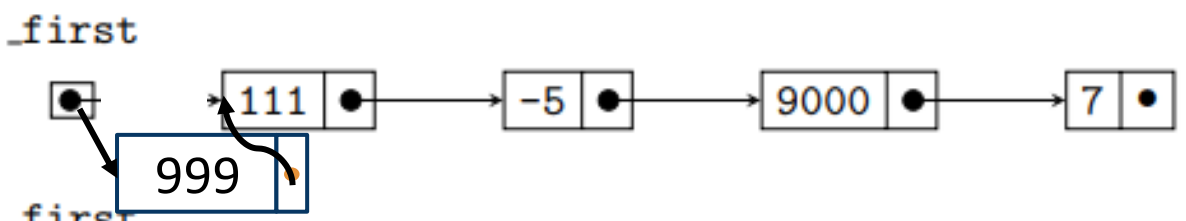
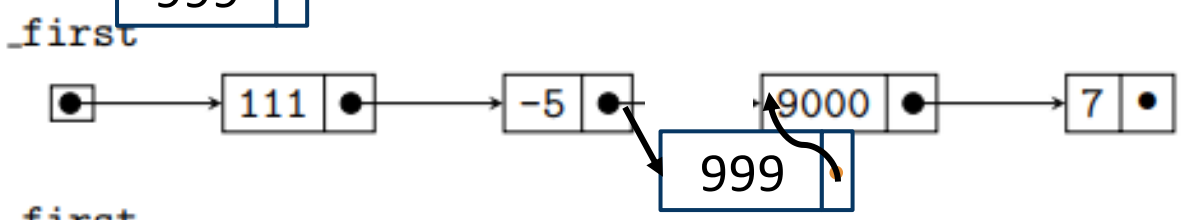
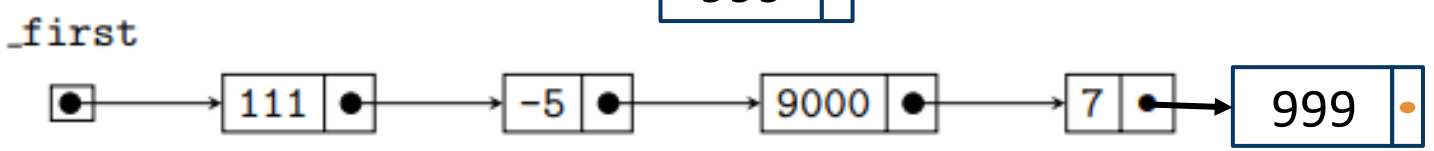
Consider the operation of inserting the value 999 into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after 999 is inserted.

   You should draw a new node containing 999. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

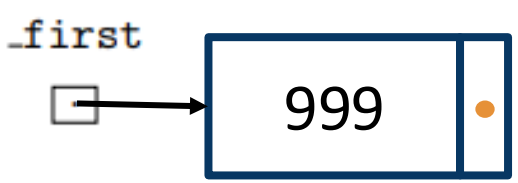
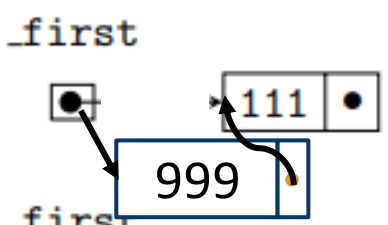
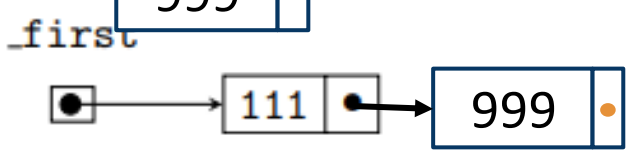
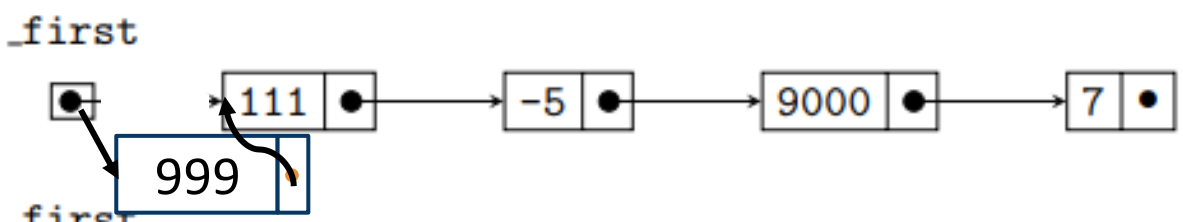
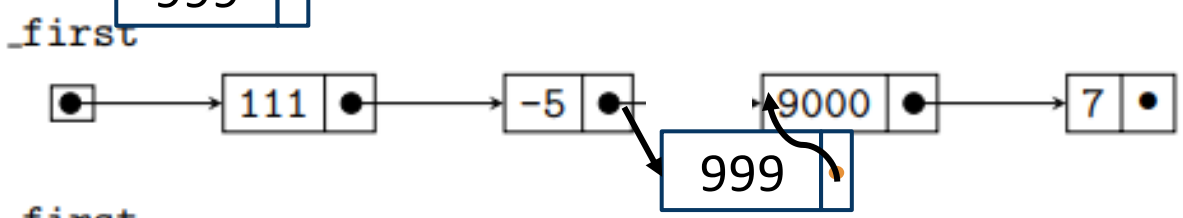
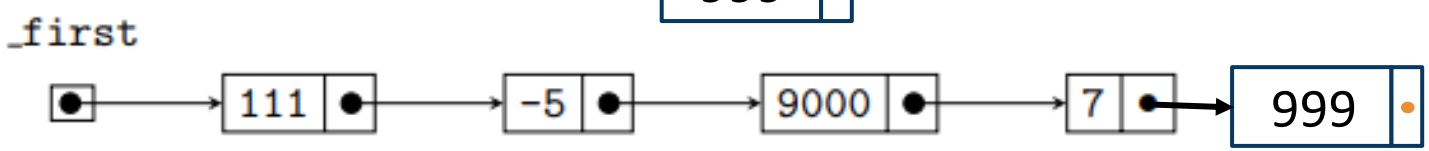
Consider the operation of inserting the value 999 into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after 999 is inserted.

   You should draw a new node containing 999. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

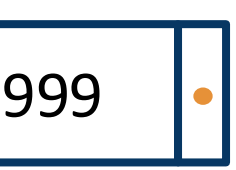
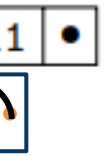
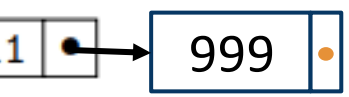
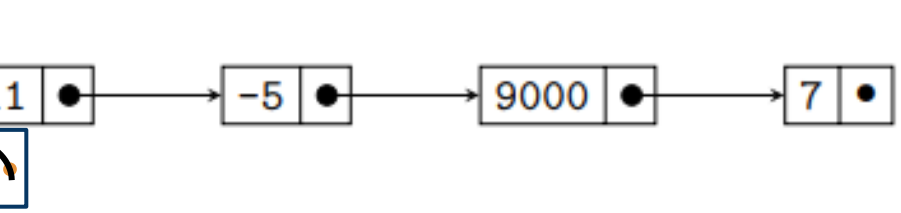
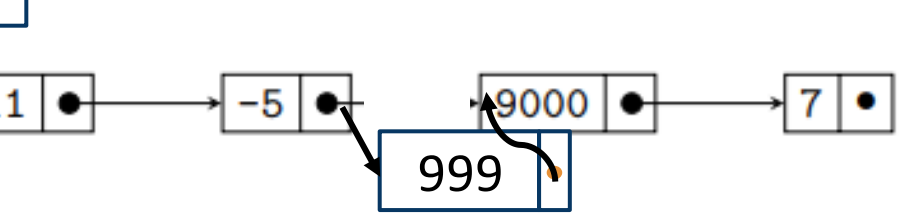
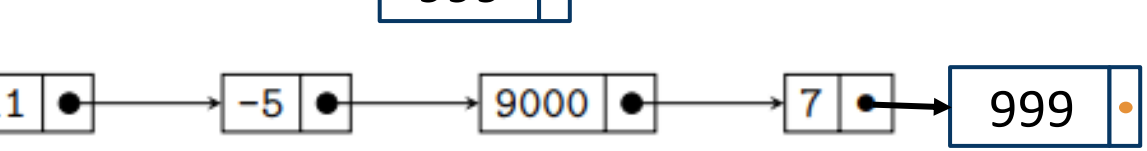
   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:

   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:

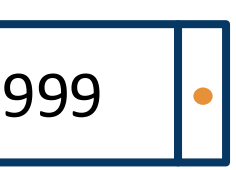
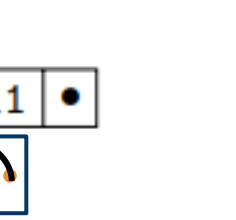
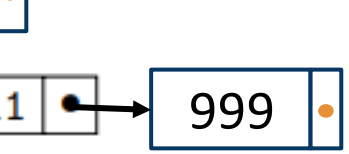
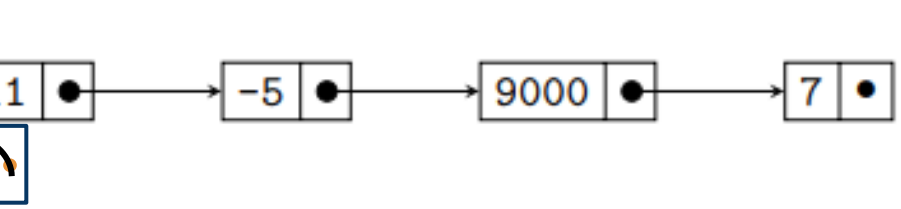
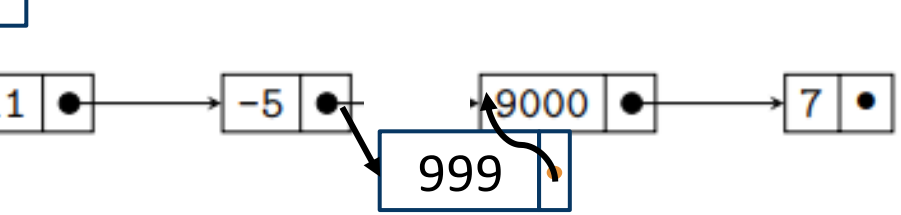
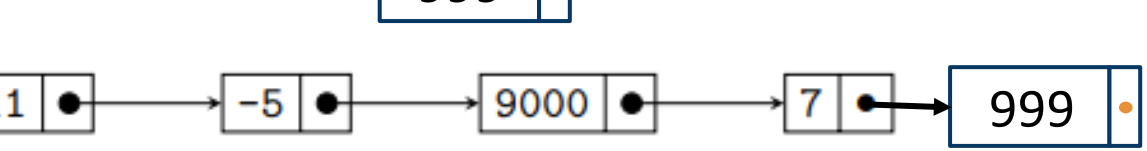
   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:

   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

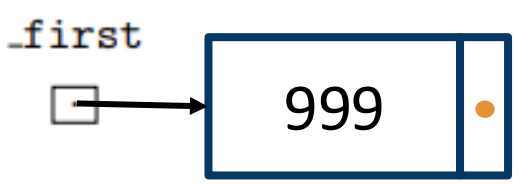
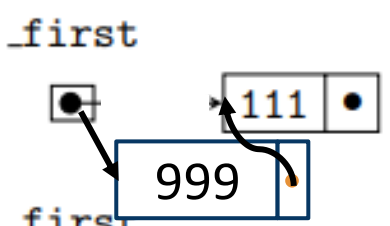
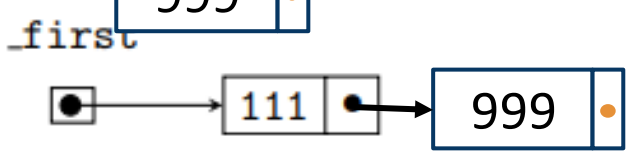
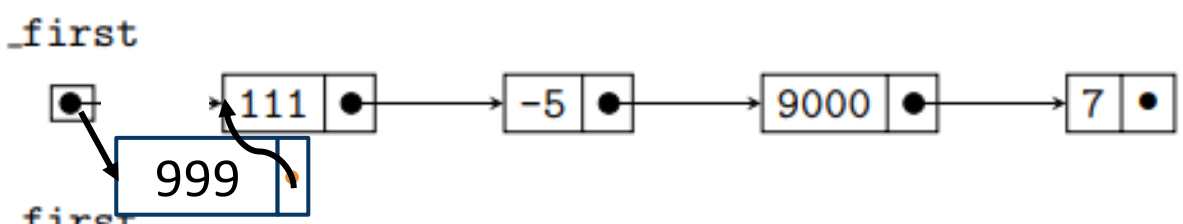
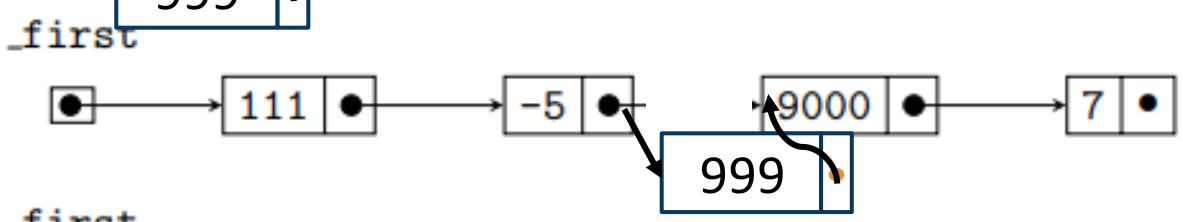
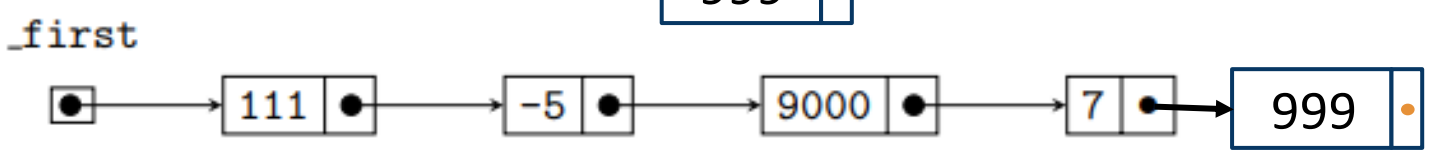
   i = 0

   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:

   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

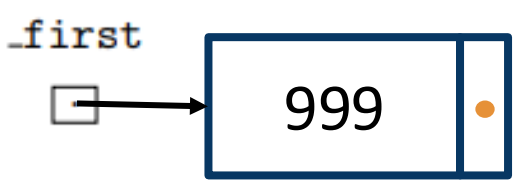
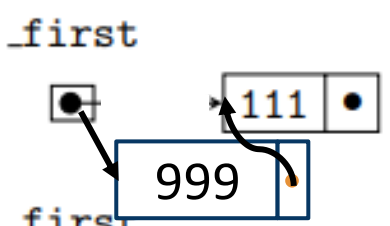
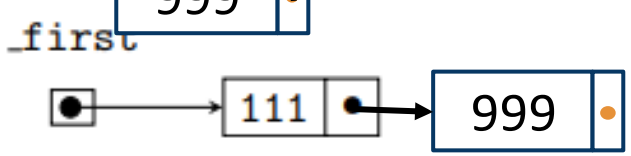
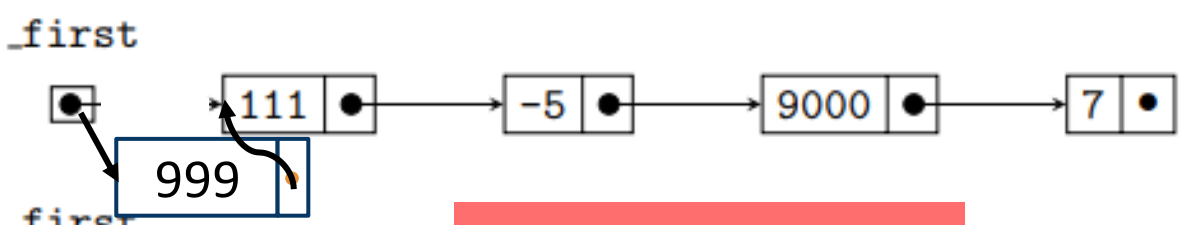
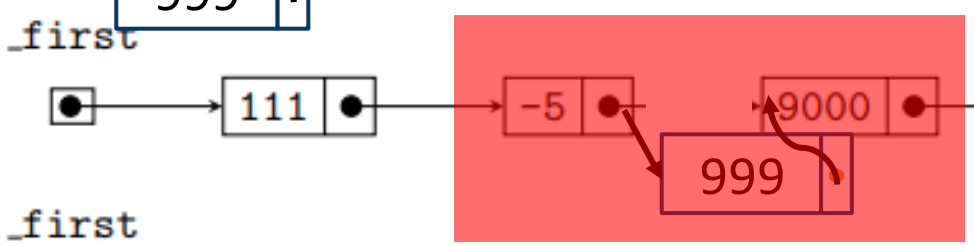
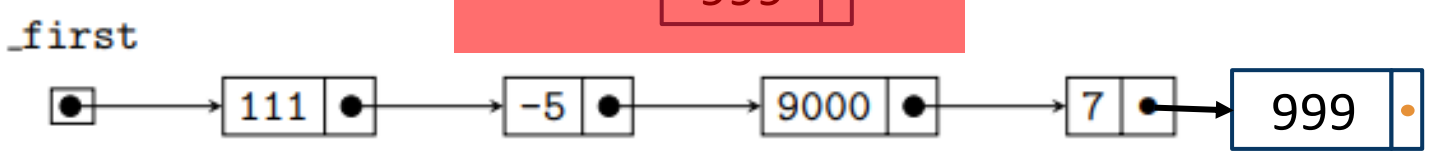
   i = 0

   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:

   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

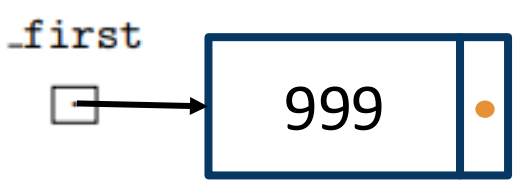
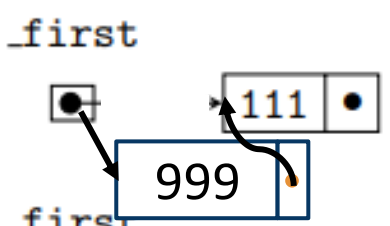
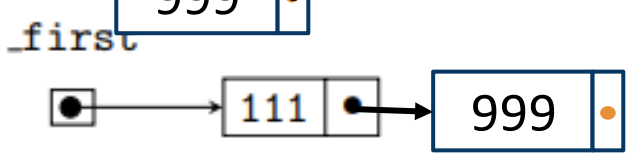
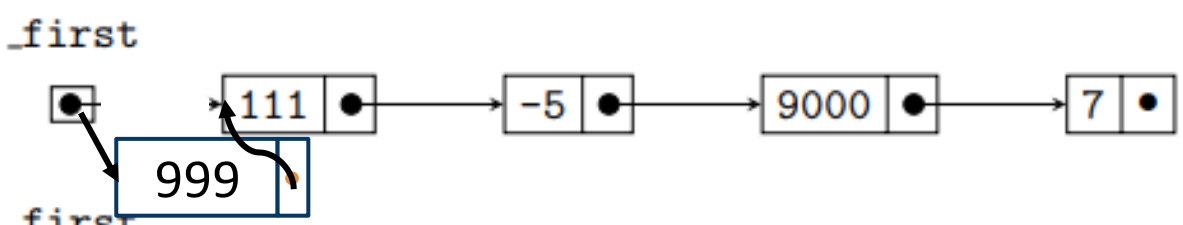
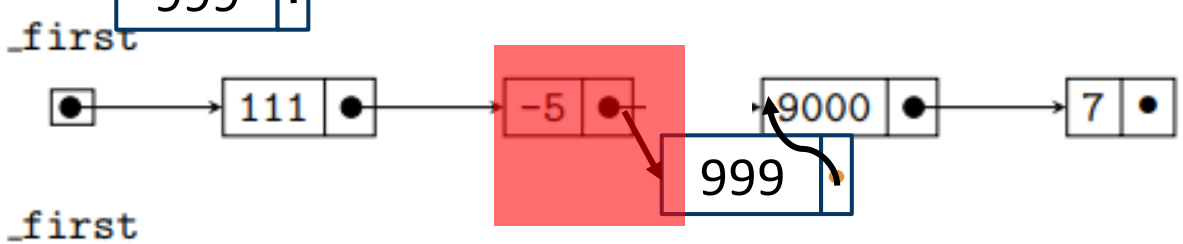
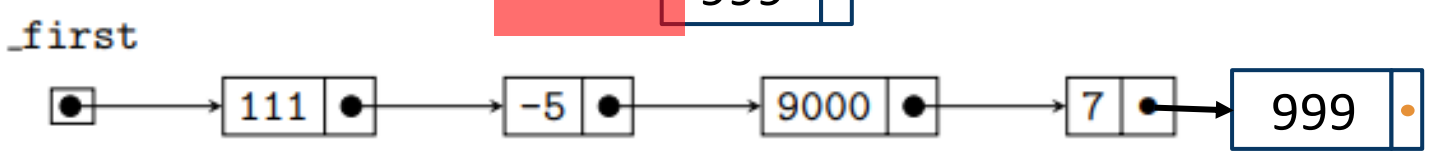
   i = 0

   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:

   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

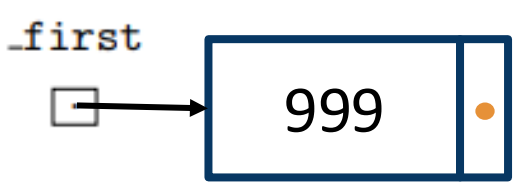
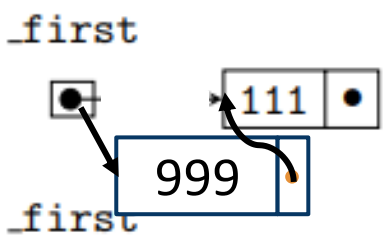
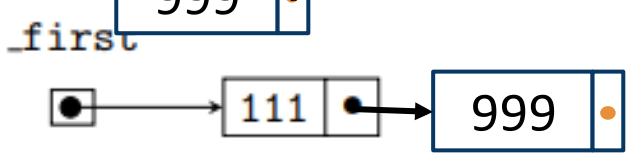
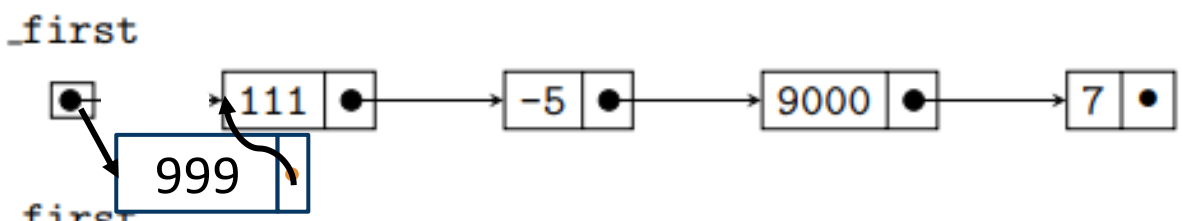
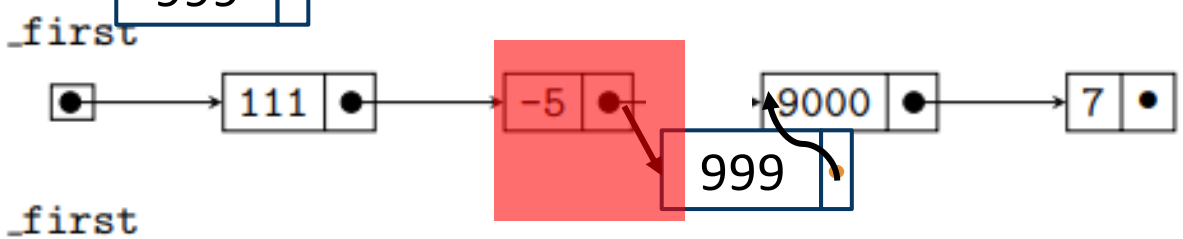
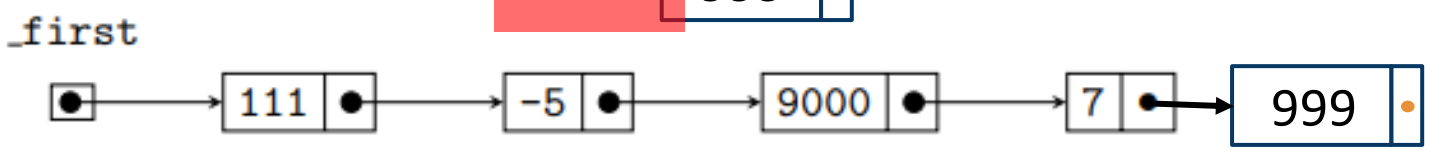
   i = 0

   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

Consider the operation of inserting the value `999` into a linked list. Below is a table of some relevant test cases to consider:

len(self)	i	self
0	0	
1	0	
1	1	
4	0	
4	2	
4	4	

2. Now let's start thinking about some code. Using your diagrams as a guide, answer the following questions:

   (a) For what values of `len(self)` and/or `i` would we need to reassign `self._first` to something new?

   i = 0

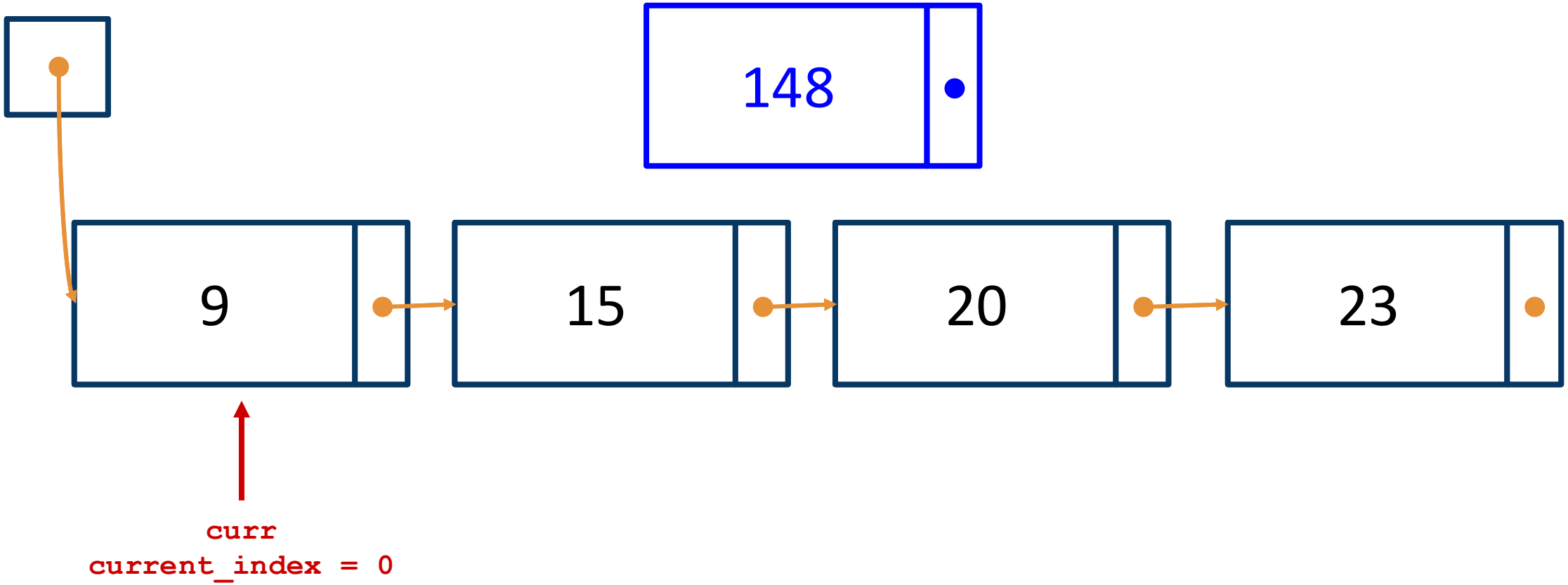
   (b) In the `len(self) == 4, i == 2` case, which *existing* node was actually mutated? Write down the index of this node in the list. (Hint: it's `not` the node at index 2!)

   index 1
   (i.e., **i - 1**)

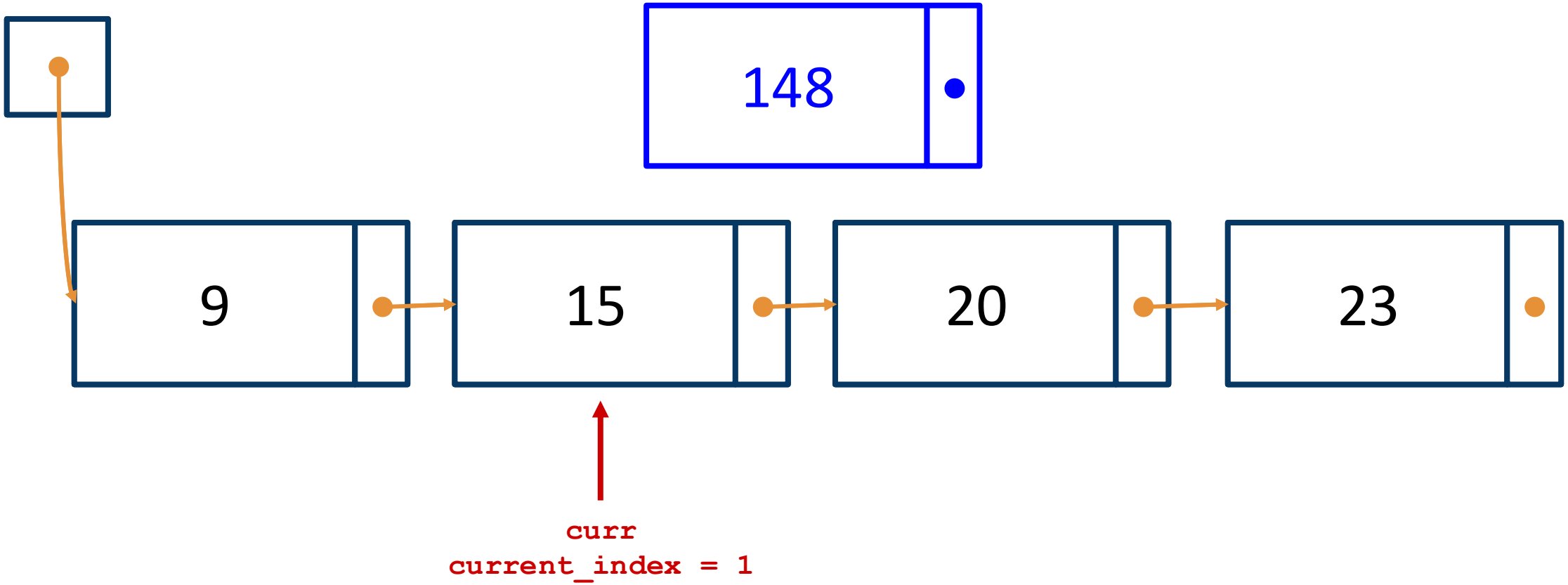
1. In the table above, modify each `self` diagram to show the state of the linked list after `999` is inserted.

   You should draw a new node containing `999`. In each case, you need to determine which arrow(s) to modify to insert the new node into the correct location in the list.

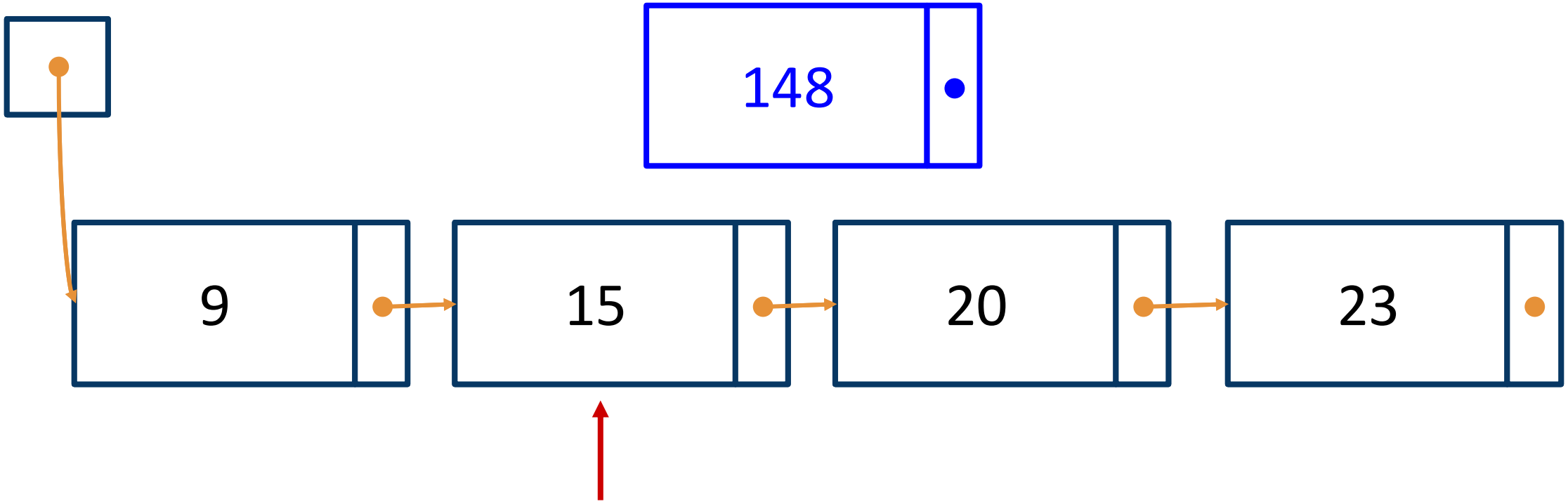
insert(2, 148) concept



insert(2, 148) concept

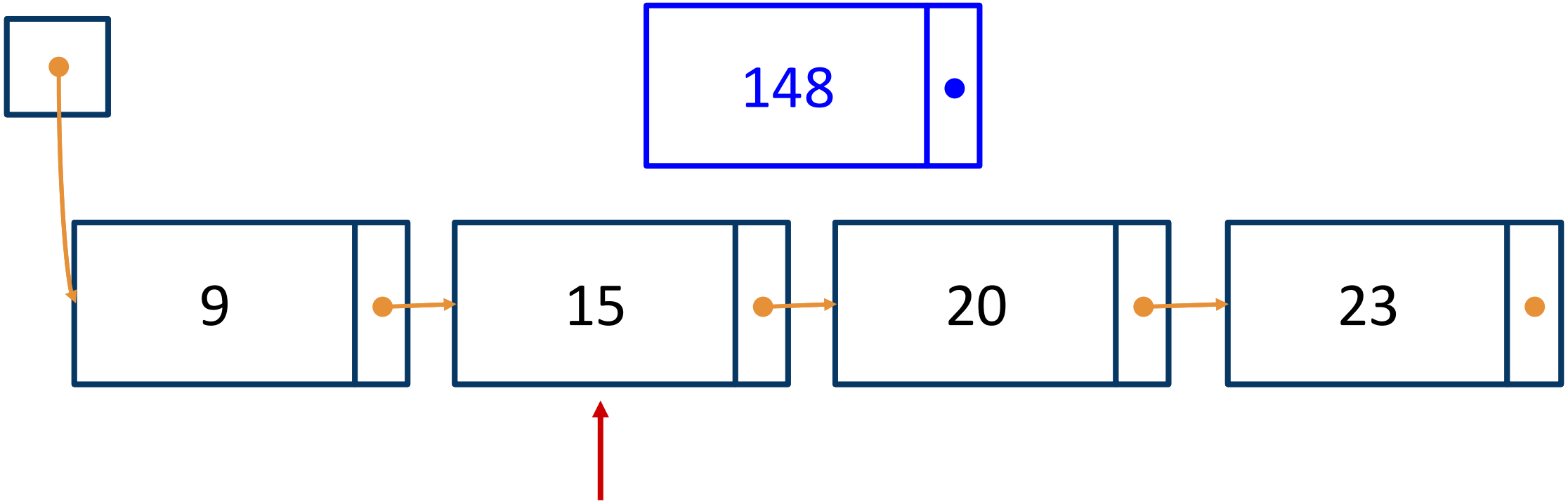


insert(2, 148) concept



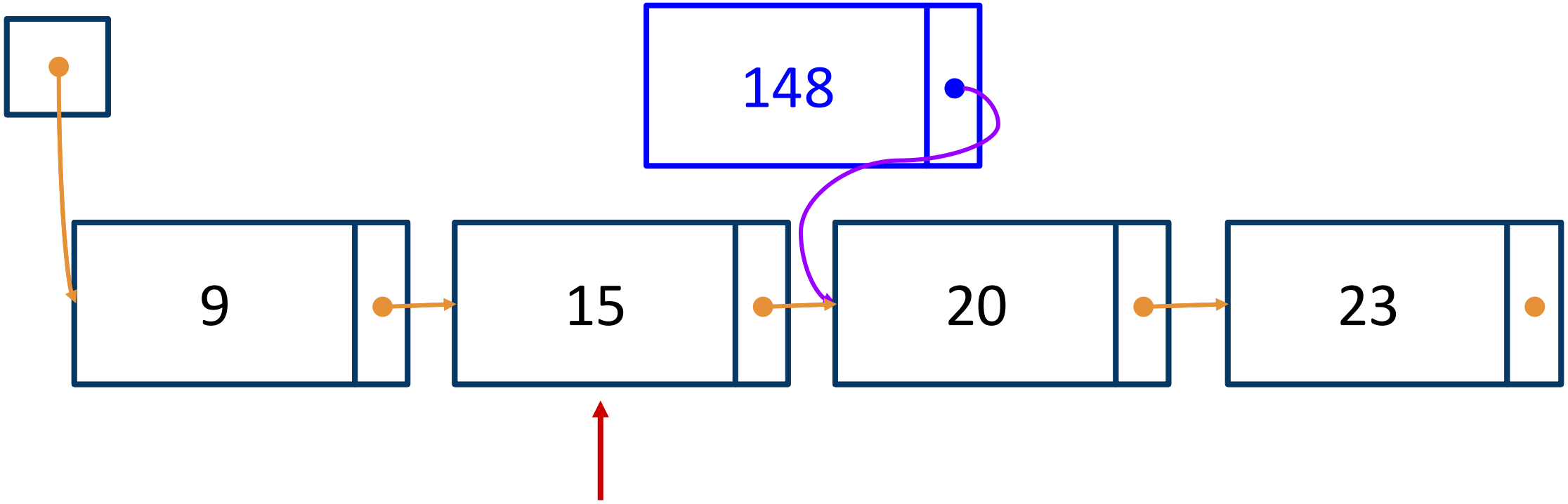
Since `current_index + 1 == index`,
we can stop searching!

insert(2, 148) concept



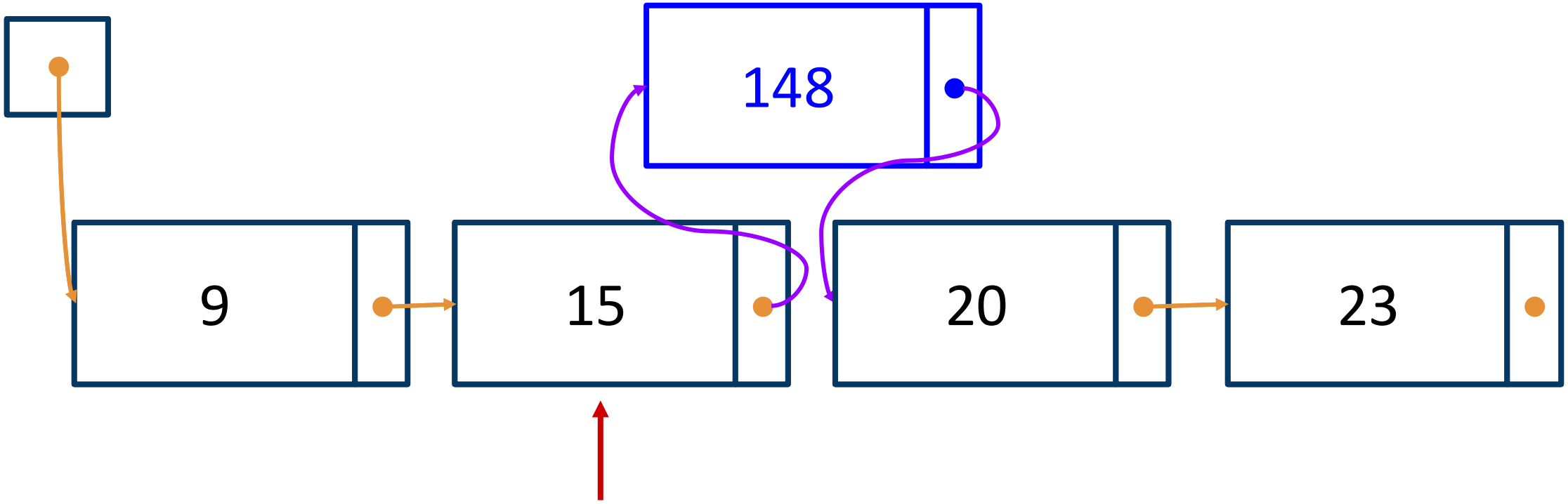

```
new_node.next = curr.next
curr.next = new_node
```

insert(2, 148) concept




```
new_node.next = curr.next
curr.next = new_node
```

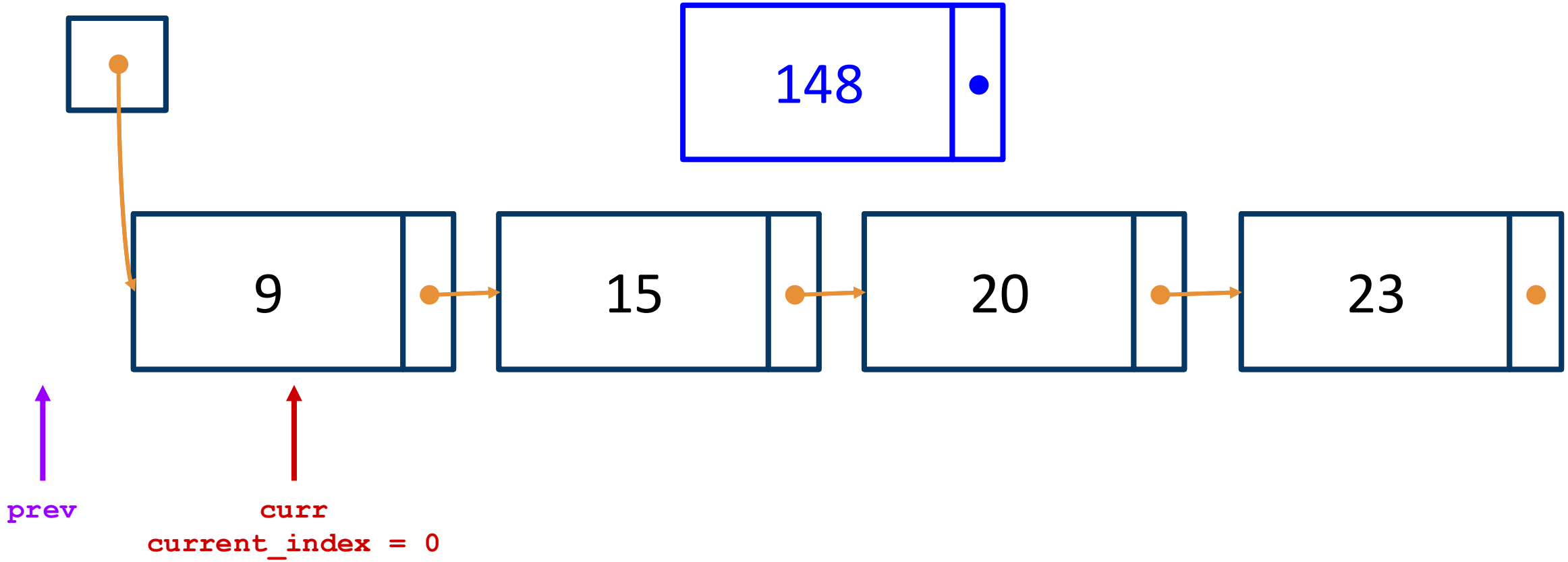
insert(2, 148) concept



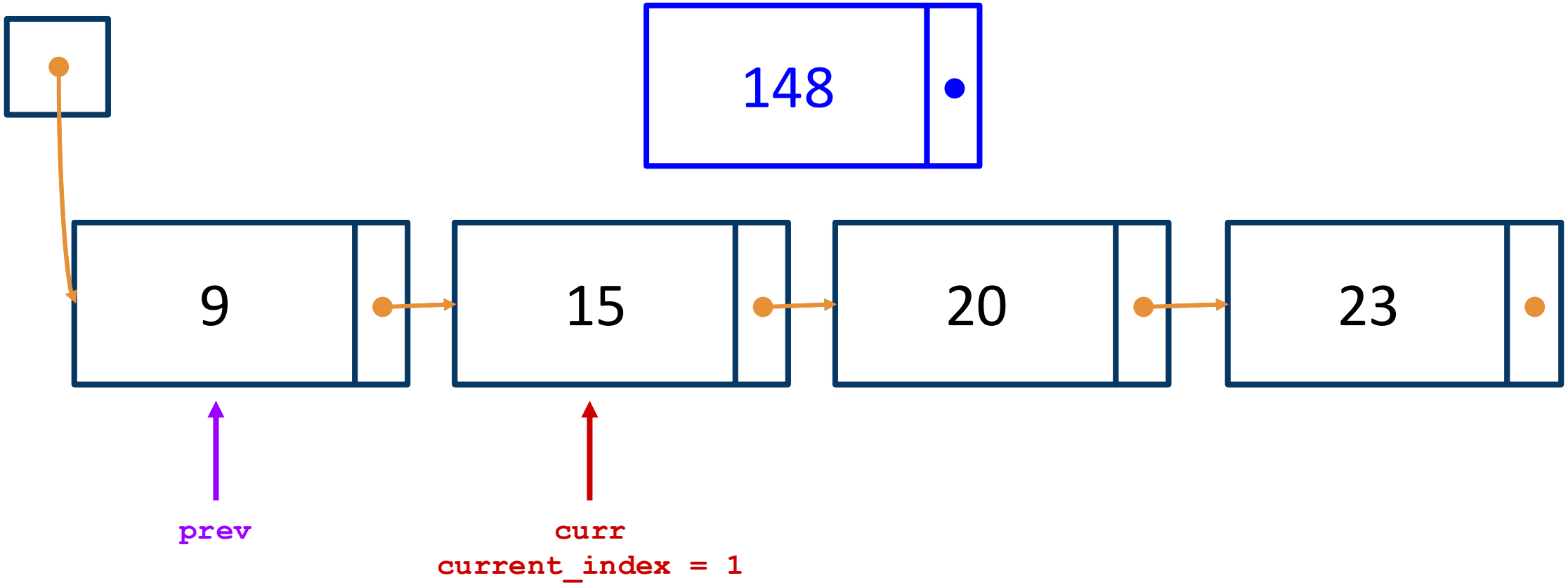

```
new_node.next = curr.next
curr.next = new_node
```

Alternatively…

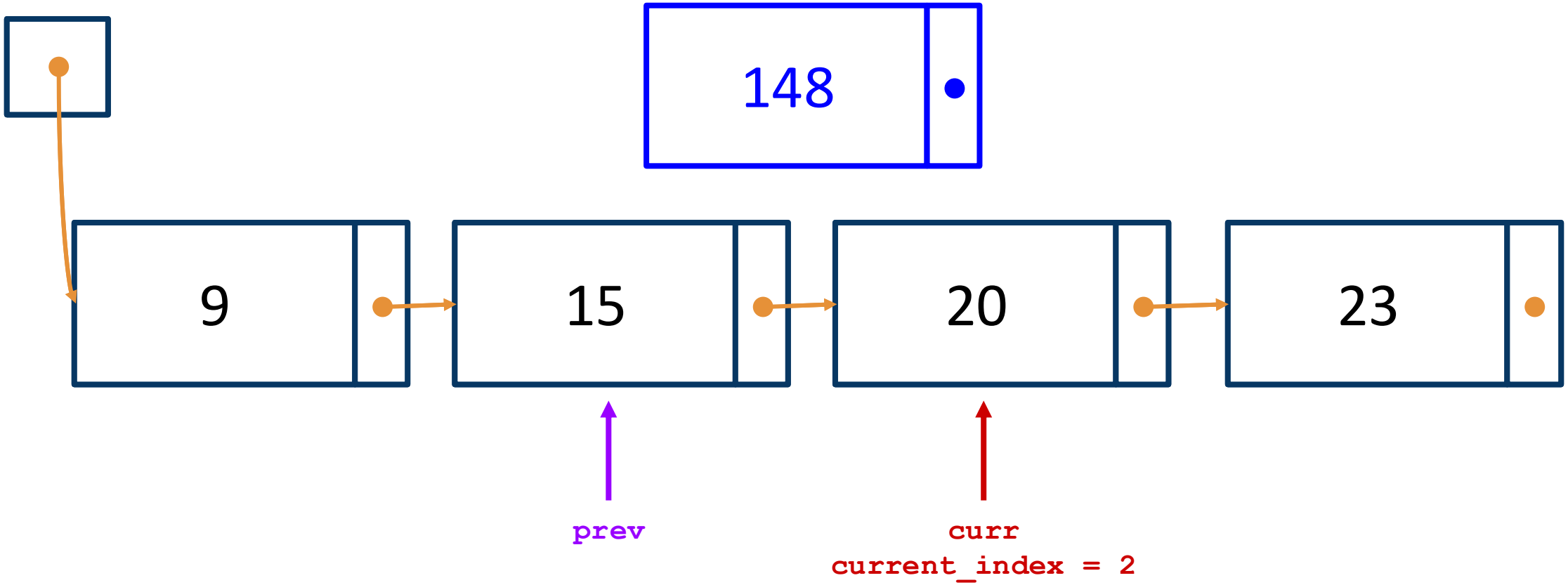
insert(2, 148) concept



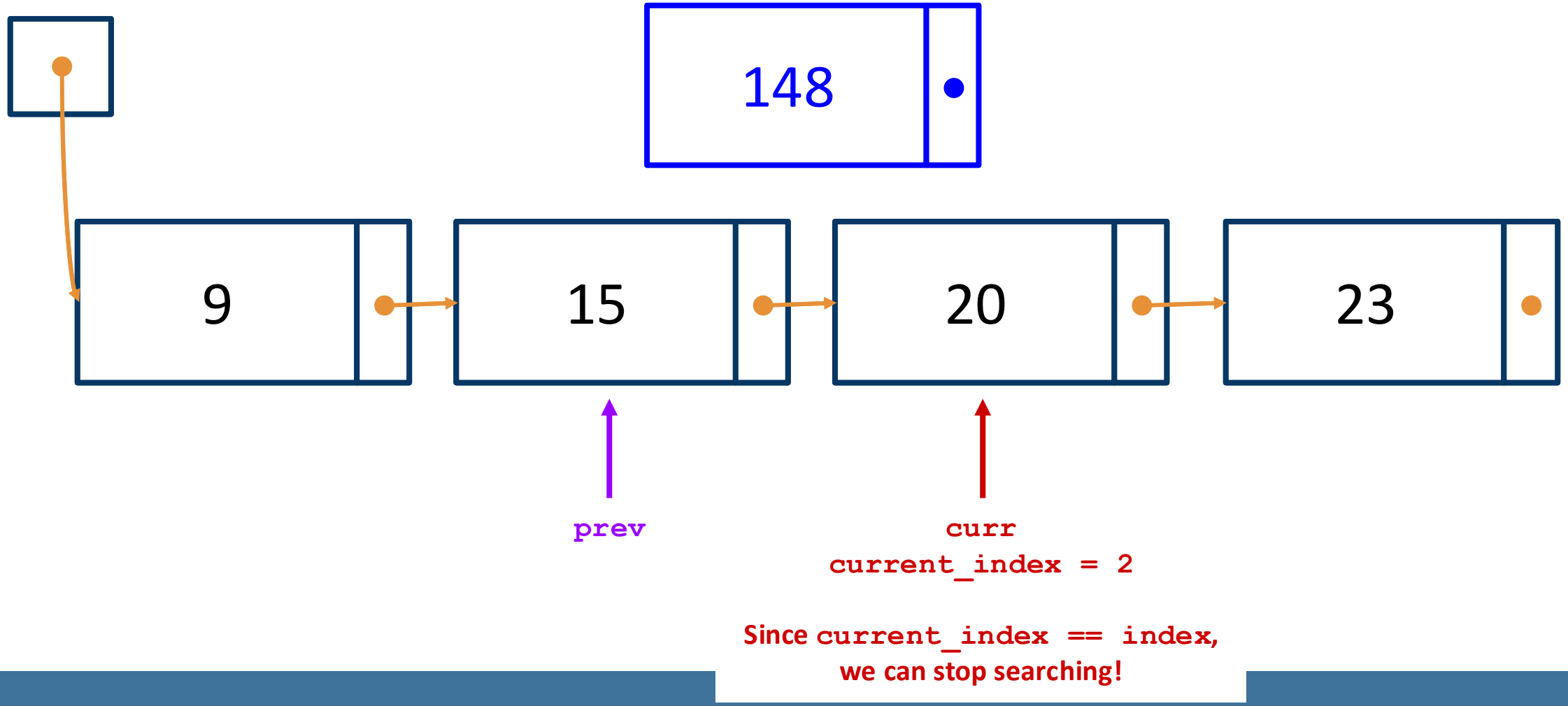
insert(2, 148) concept



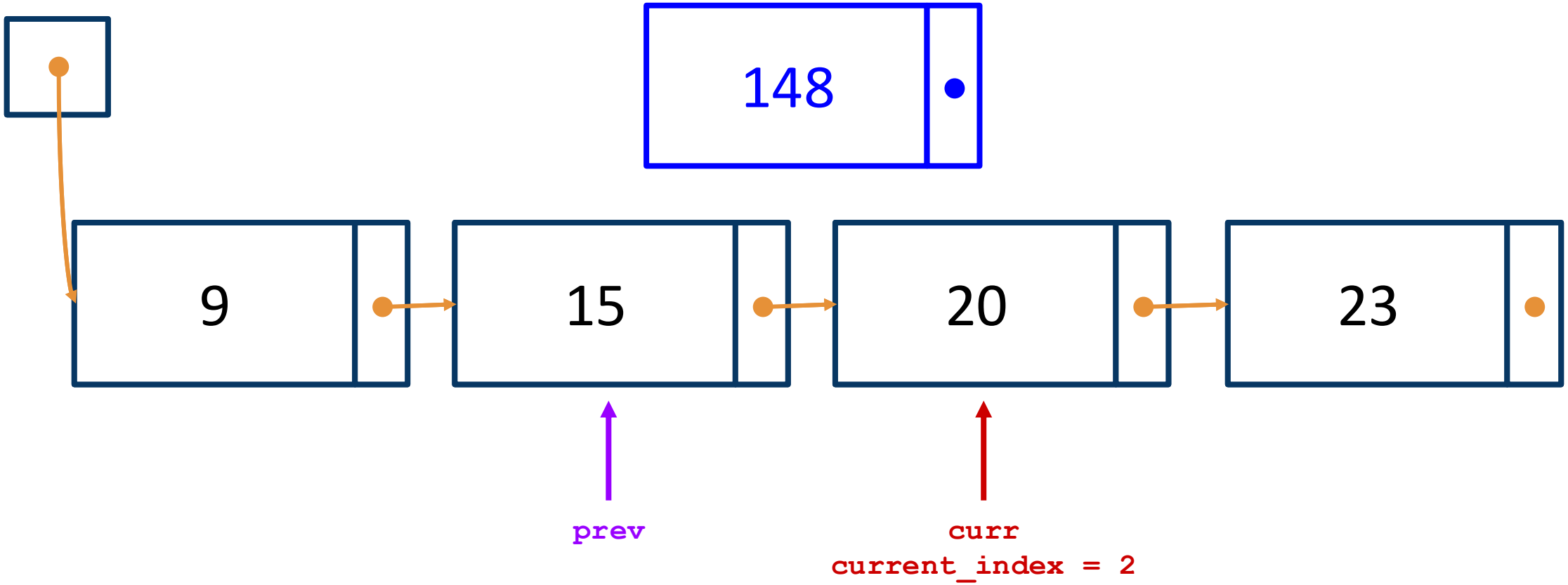
insert(2, 148) concept



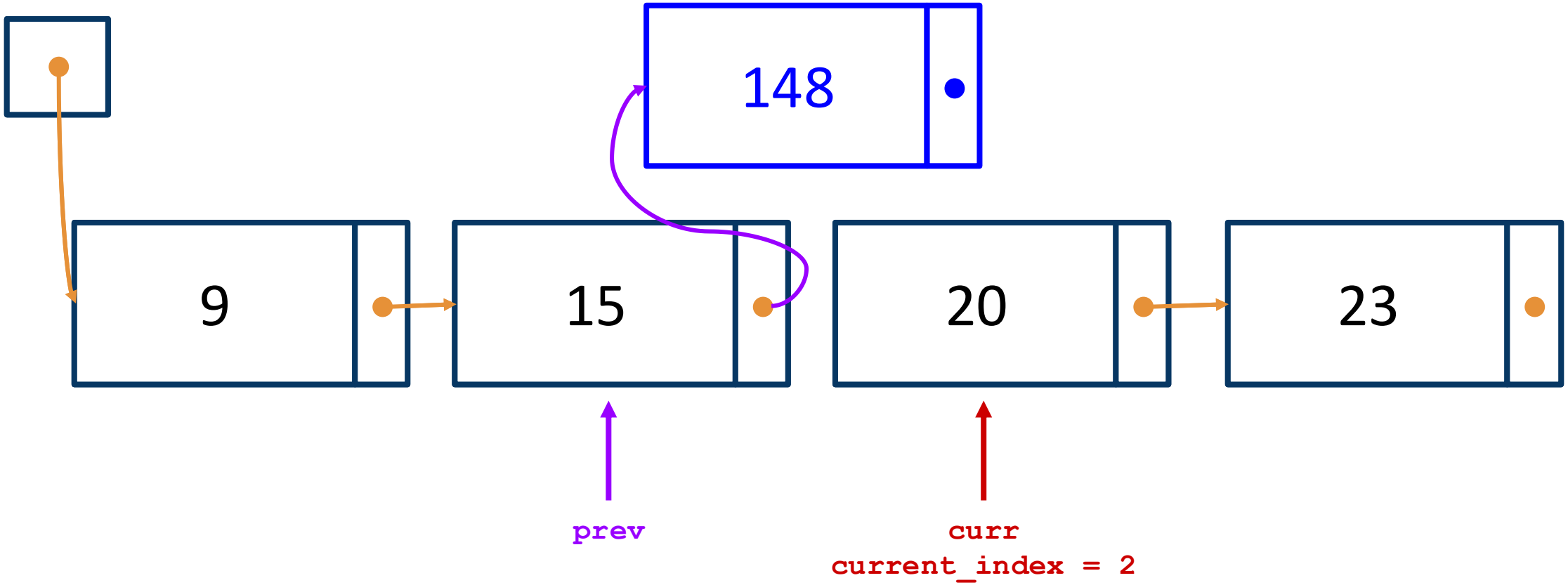
insert(2, 148) concept



insert(2, 148) concept

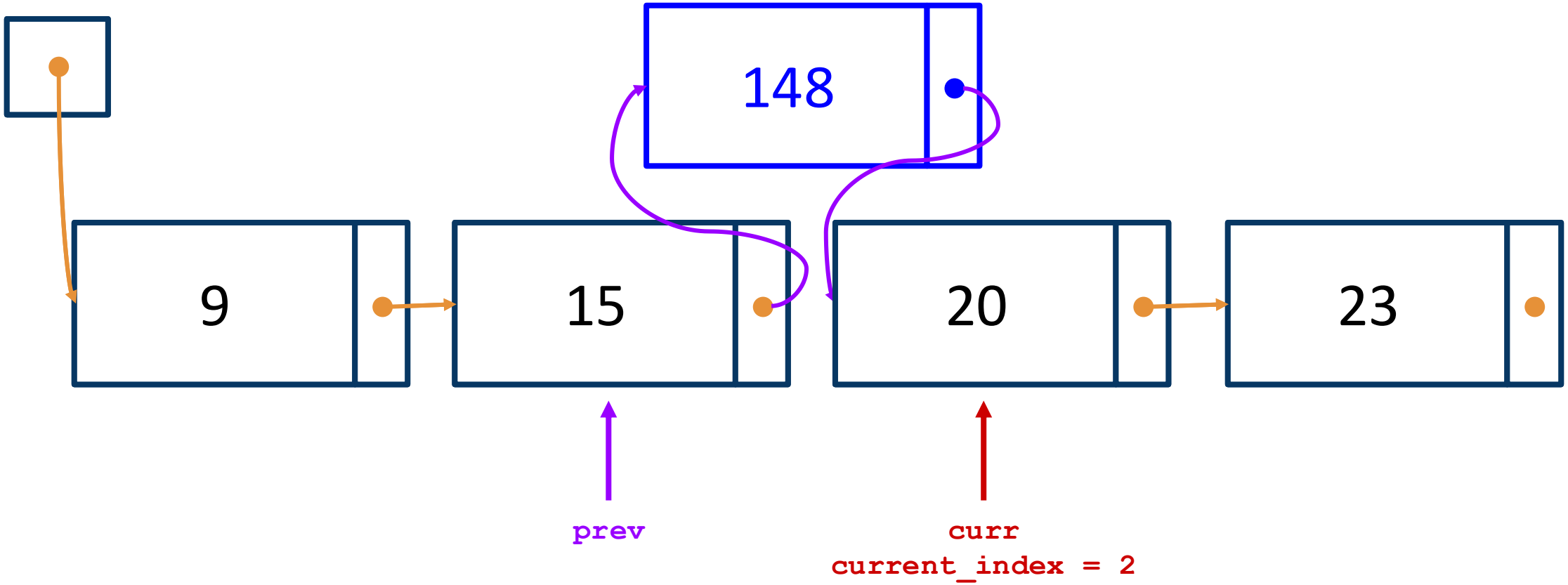


insert(2, 148) concept




```
prev.next = new_node
new_node.next = curr
```

insert(2, 148) concept




```
prev.next = new_node
new_node.next = curr
```

The “problem of previous”

Strategy #1: iterate to the node *before* the desired position.

```
i = 0
curr = self._first
while not (curr is None or i == index – 1):
    curr = curr.next
    i += 1
```

The “problem of previous”

Strategy #2: track the previous node explicitly

```
i = 0
prev = None
curr = self._first
while not (curr is None or i == index):
    prev, curr = curr, curr.next
    i += 1
```

Mistakes when updating next

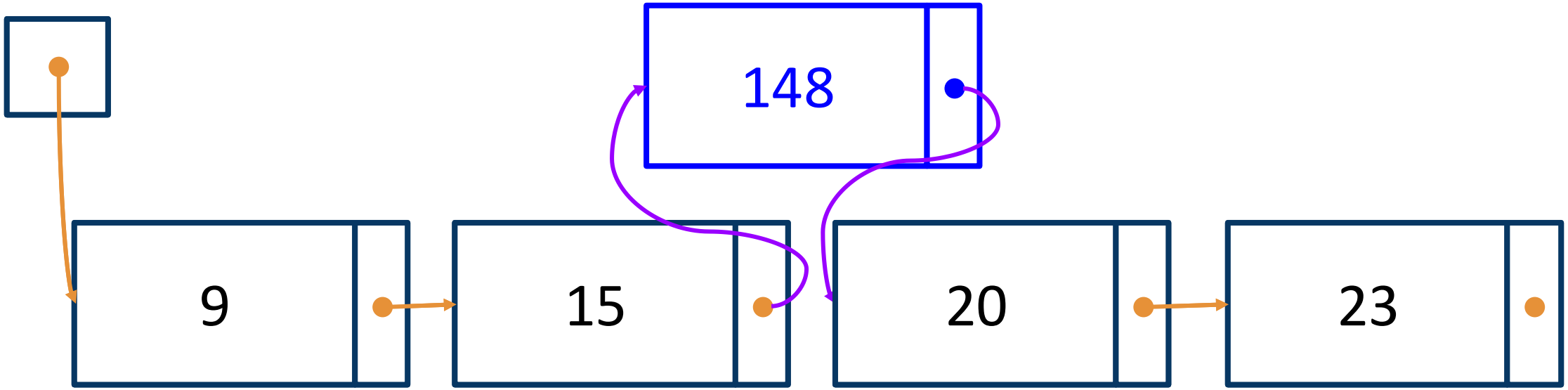
If you don't update .next

```
curr.next = new_node
```

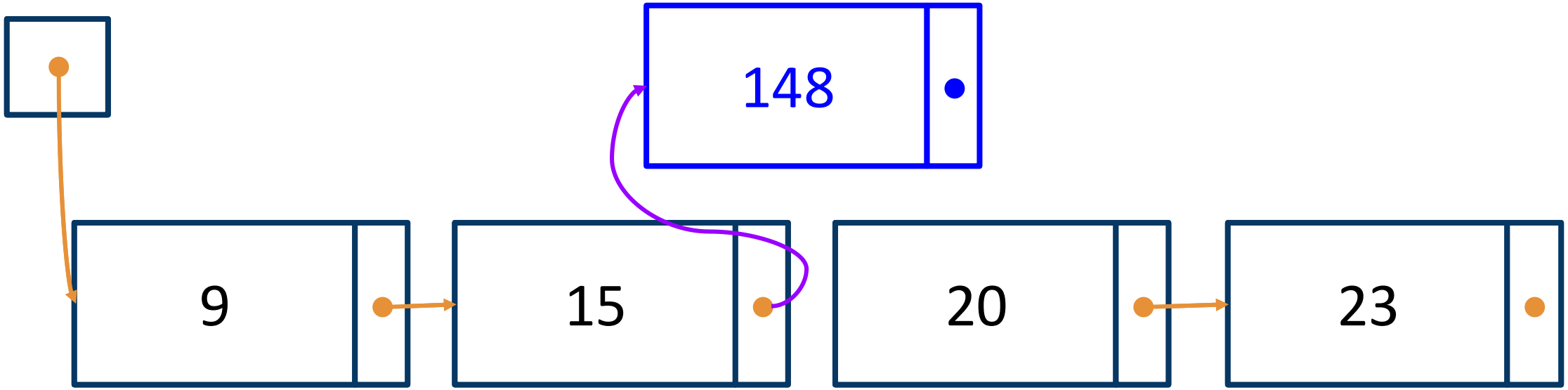
vs.

```
new_node.next = curr.next
curr.next = new_node
```

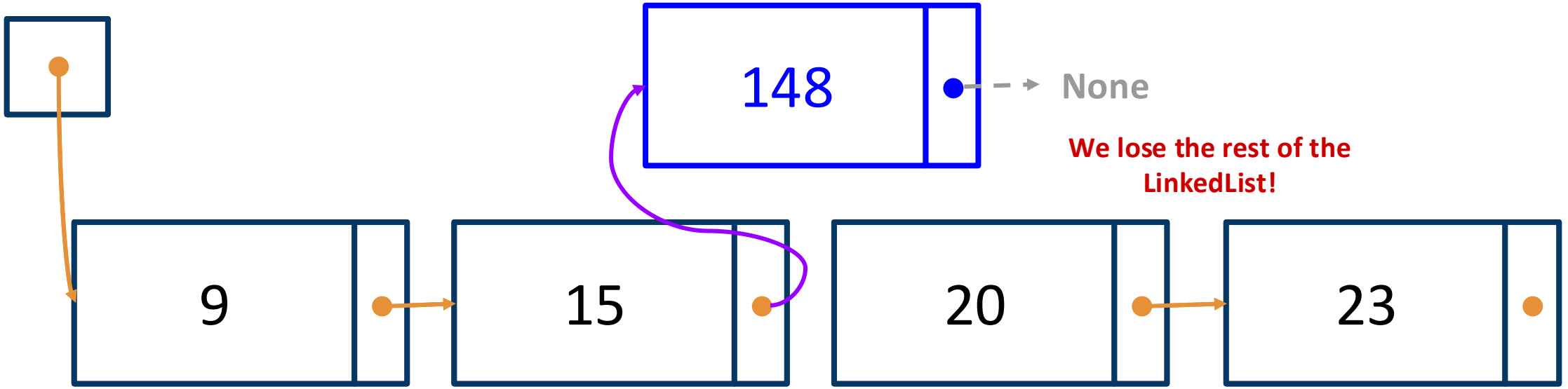
If you don't update .next



If you don't update .next



If you don't update .next



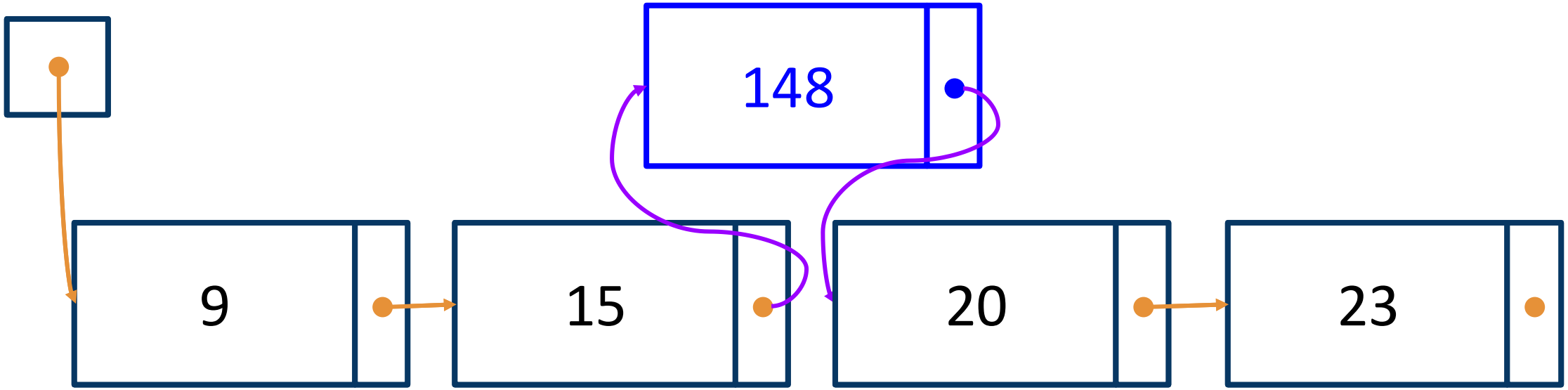
If you don't update .next

```
new_node.next = curr.next
```

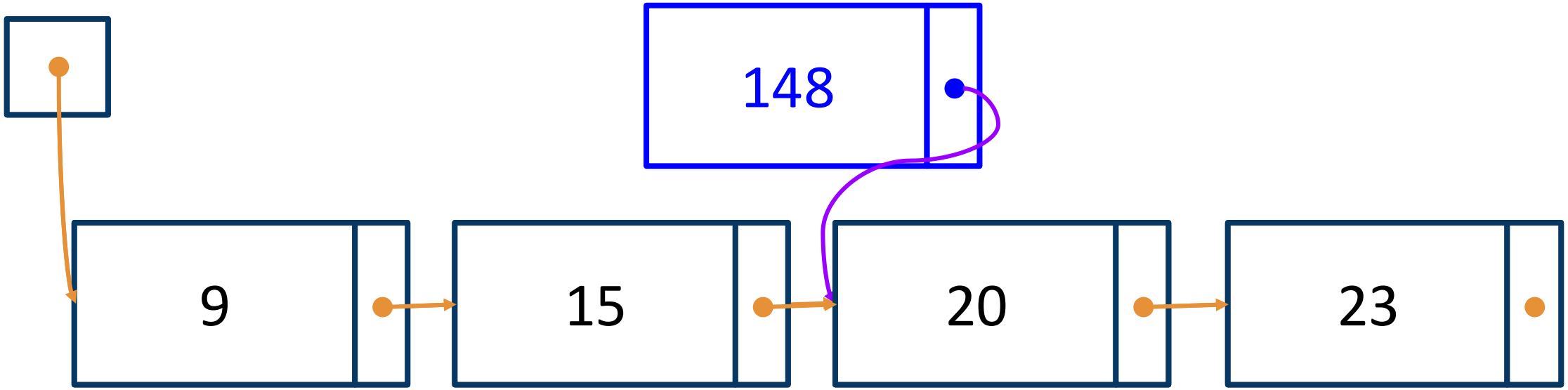
vs.

```
new_node.next = curr.next
curr.next = new_node
```

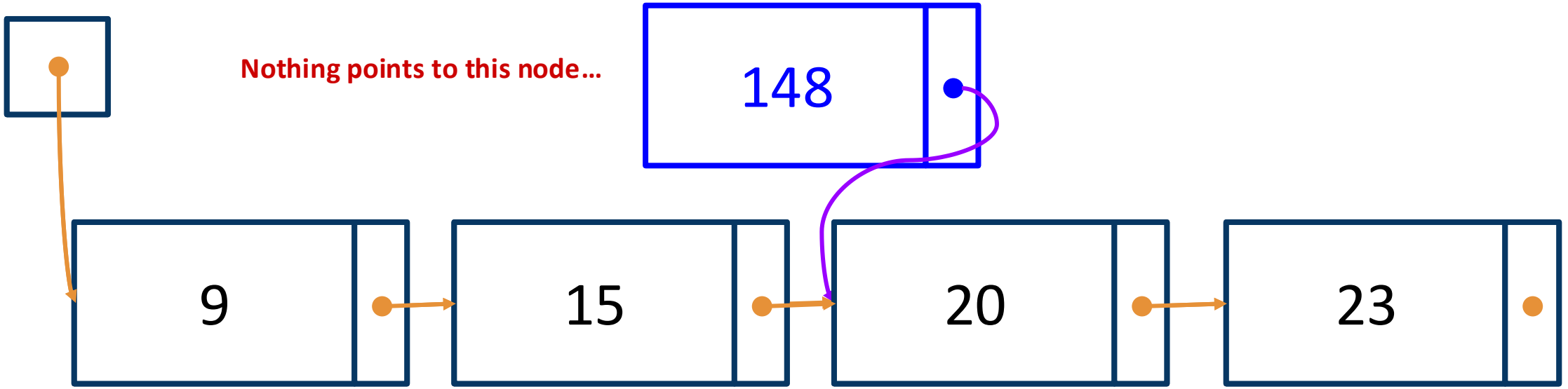
If you don't update .next



If you don't update .next



If you don't update .next



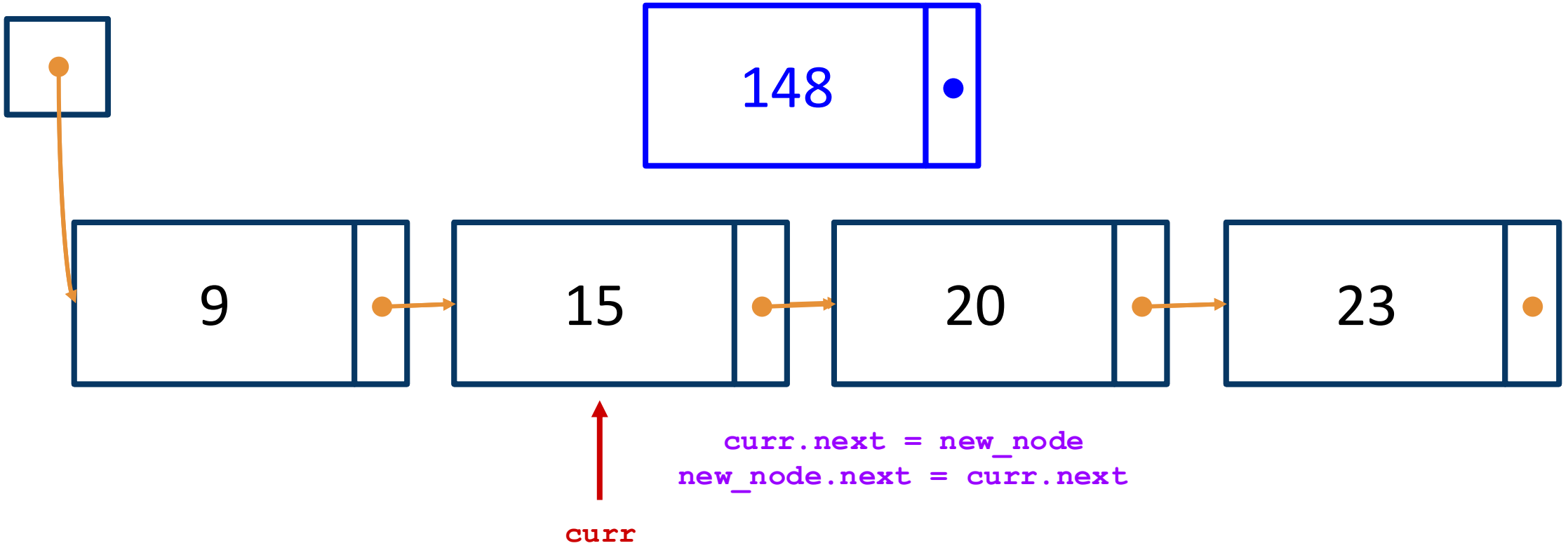
If you update in the wrong order

```
curr.next = new_node
new_node.next = curr.next
```

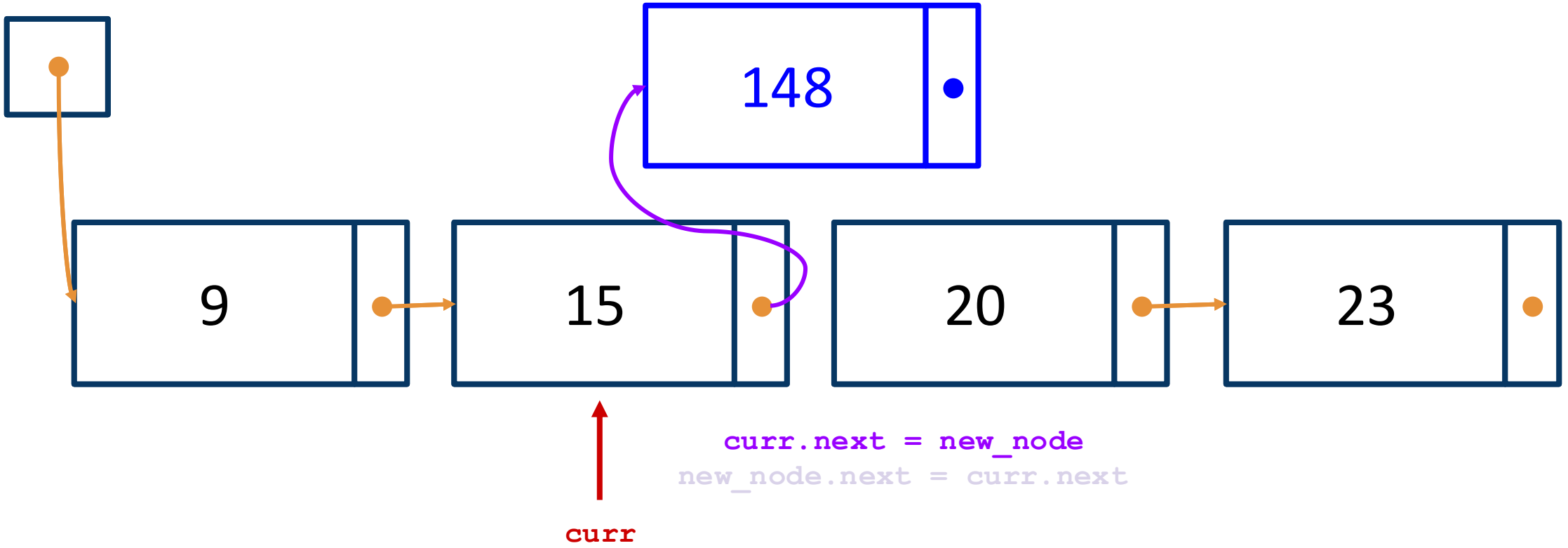
vs.

```
new_node.next = curr.next
curr.next = new_node
```

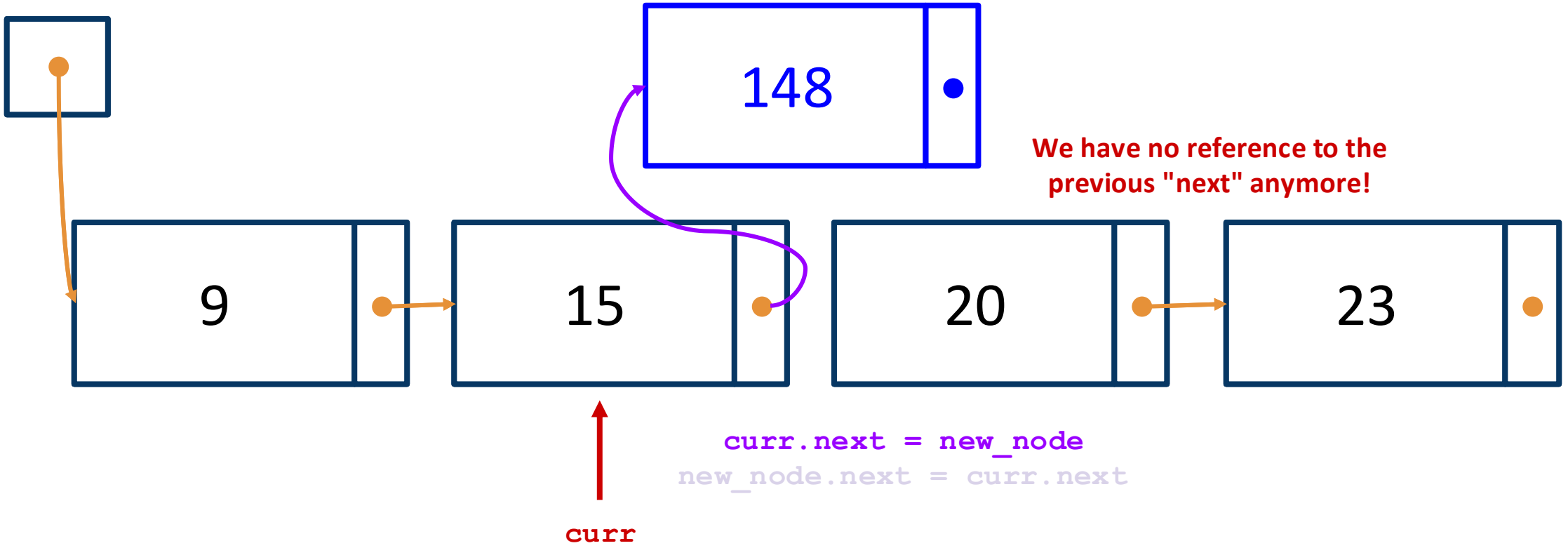
If you update in the wrong order



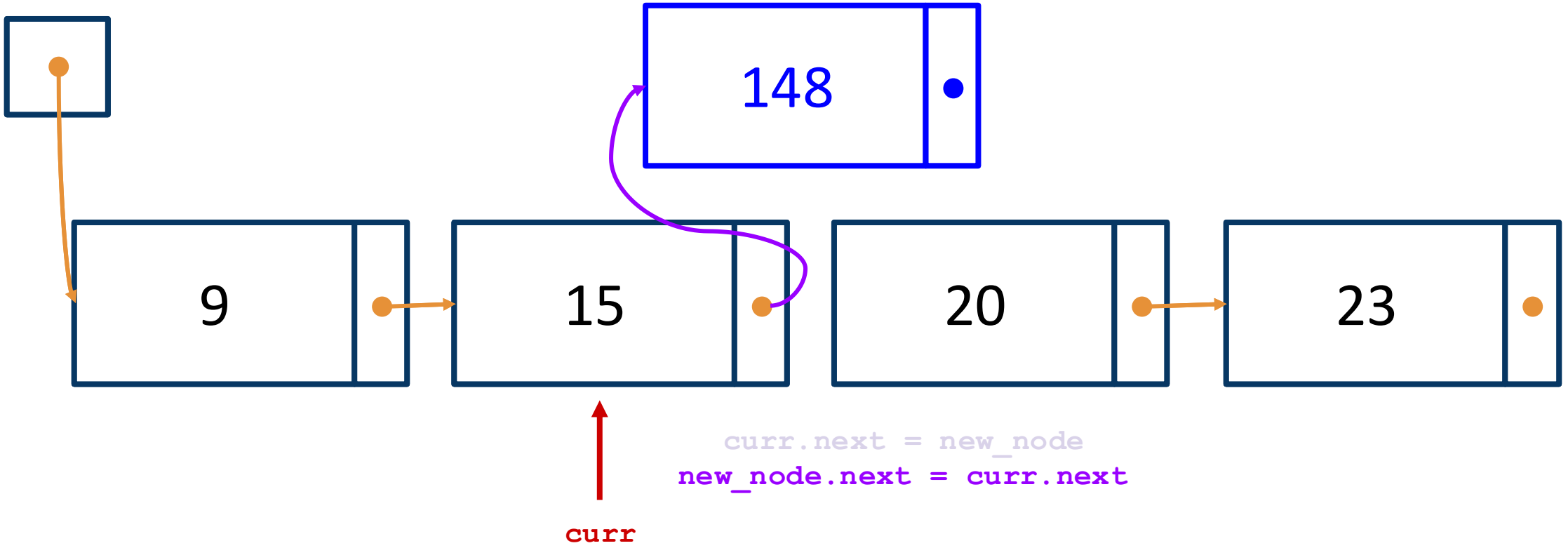
If you update in the wrong order



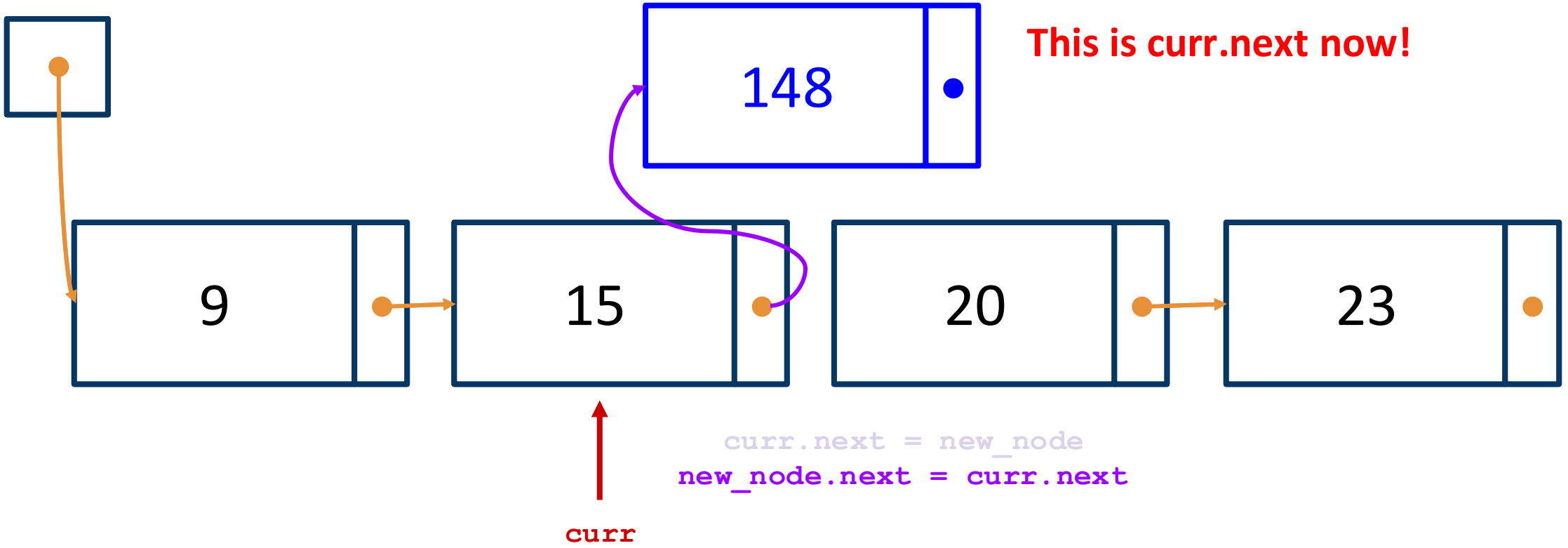
If you update in the wrong order



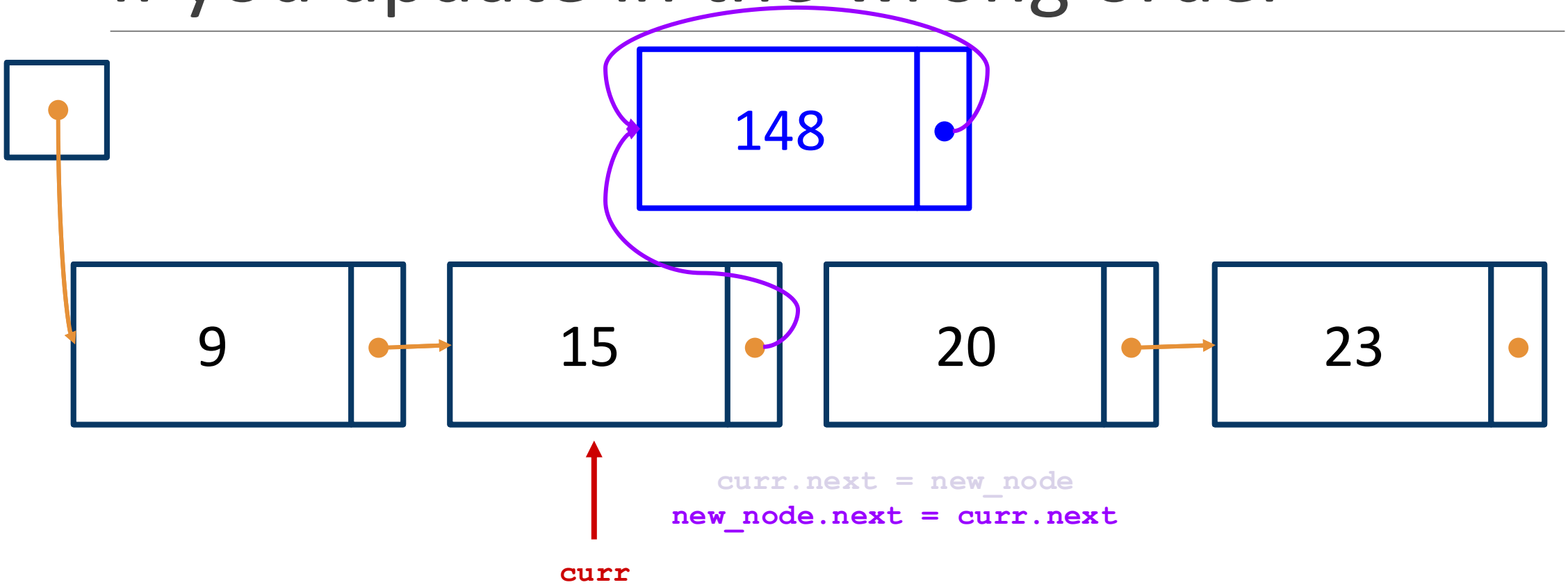
If you update in the wrong order



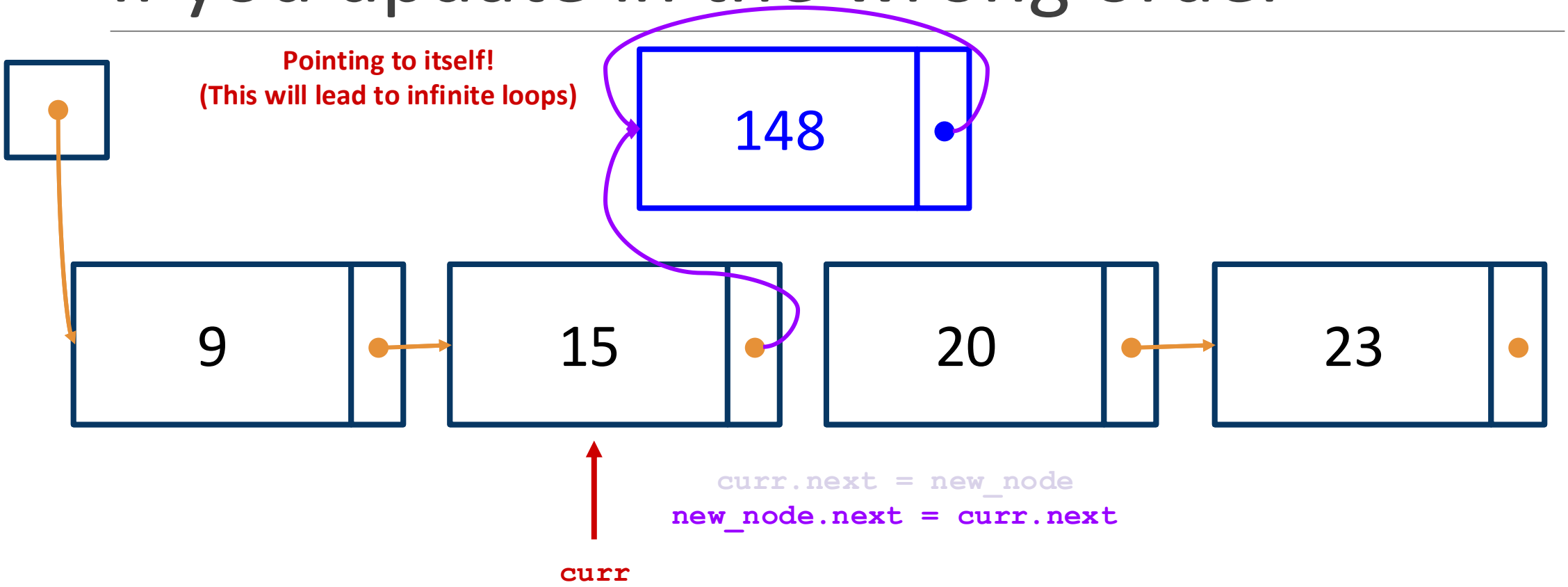
If you update in the wrong order



If you update in the wrong order



If you update in the wrong order



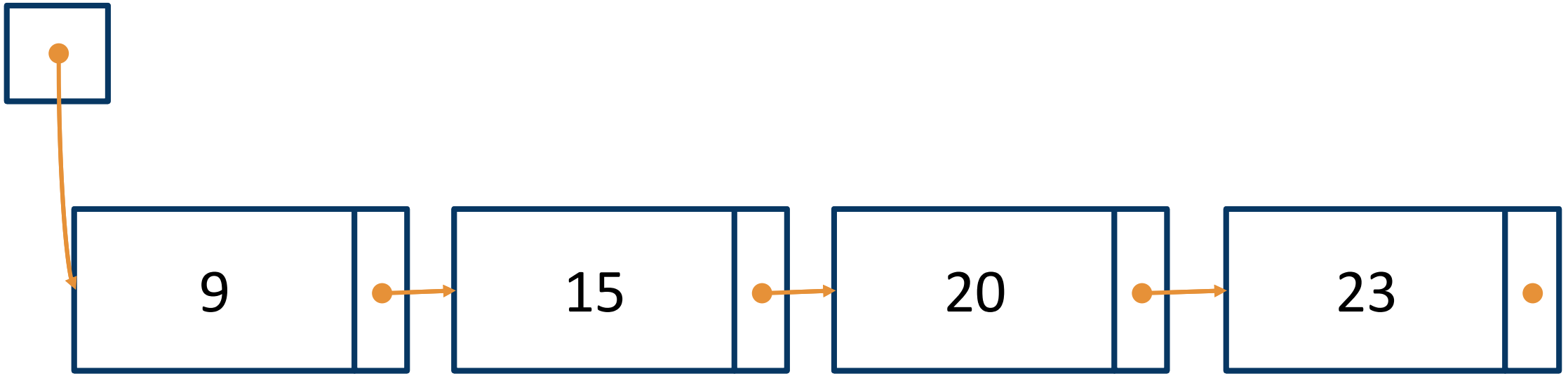
Recapping the key ideas

1. Figure out when we need to modify `self._first` vs. a _Node in the list.

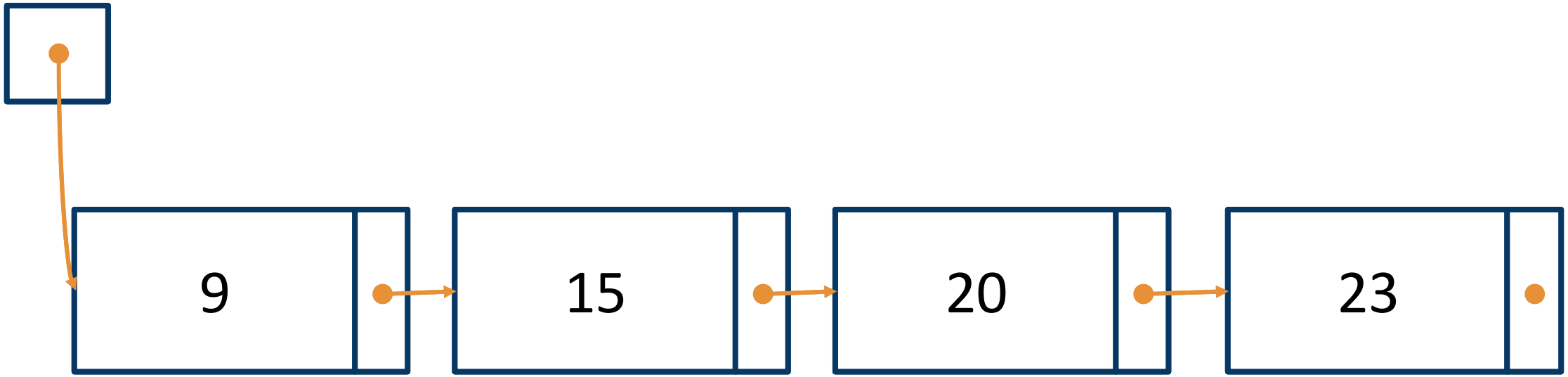
1. When `index` > 0, iterate to the (`index`-1)th node and update links.

```
def pop(self, index: int) -> Any:
    """Remove and return the item at the given
    index.

    Raise IndexError if index >= len(self)
    or index < 0.
    """
```

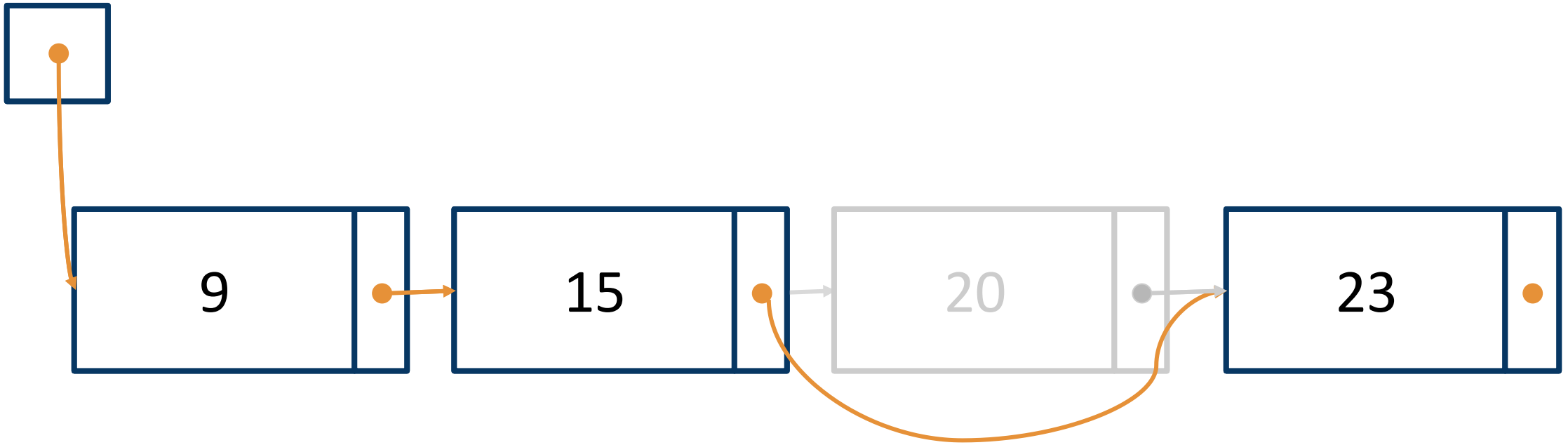
pop concept



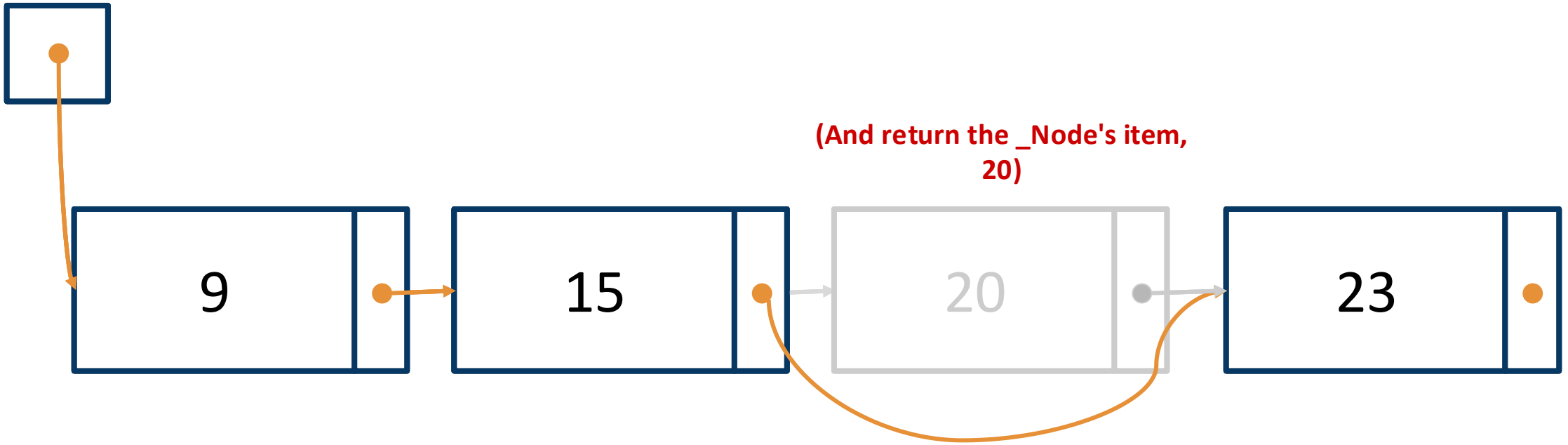
pop(2) concept



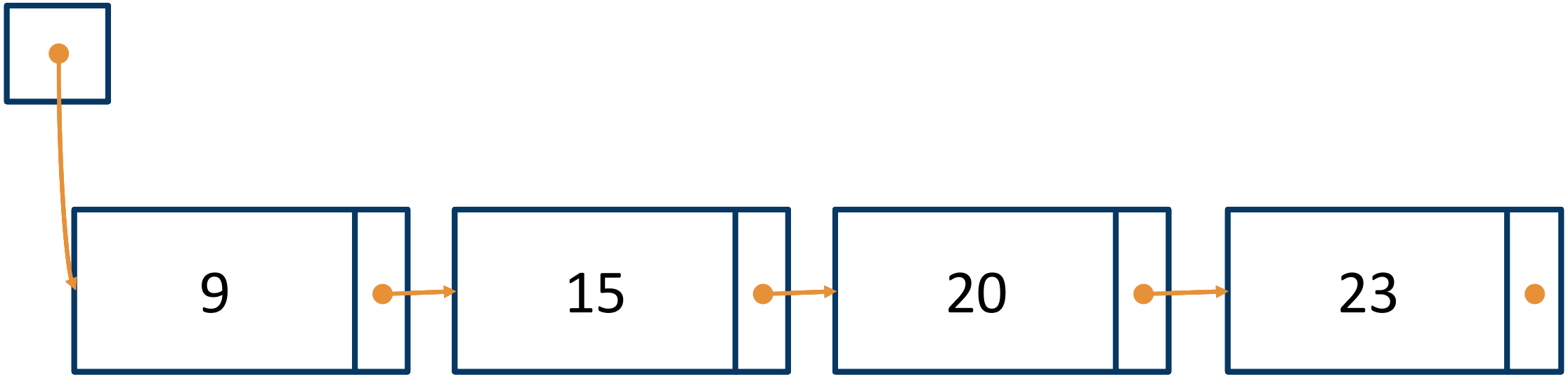
pop(2) concept



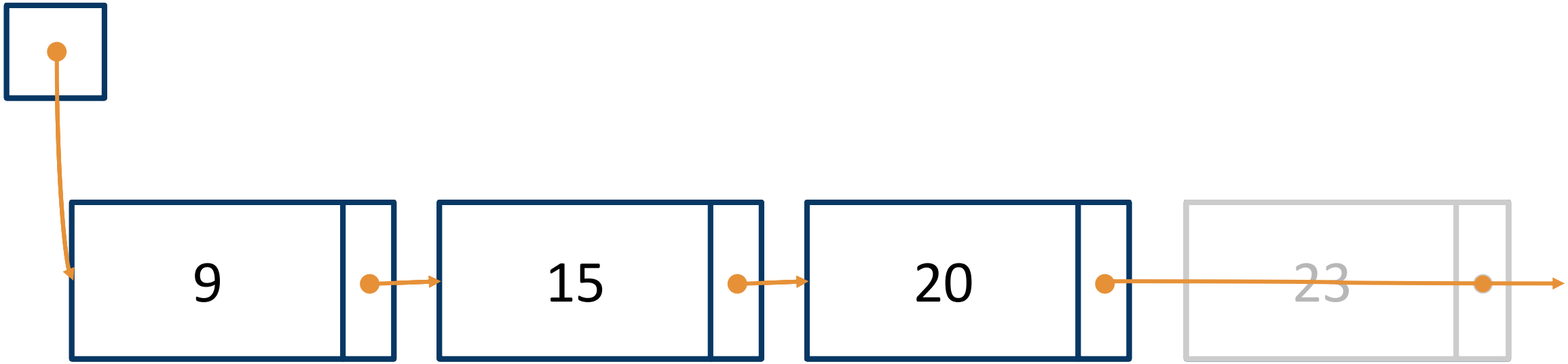
pop(2) concept



pop(3) concept



pop(3) concept

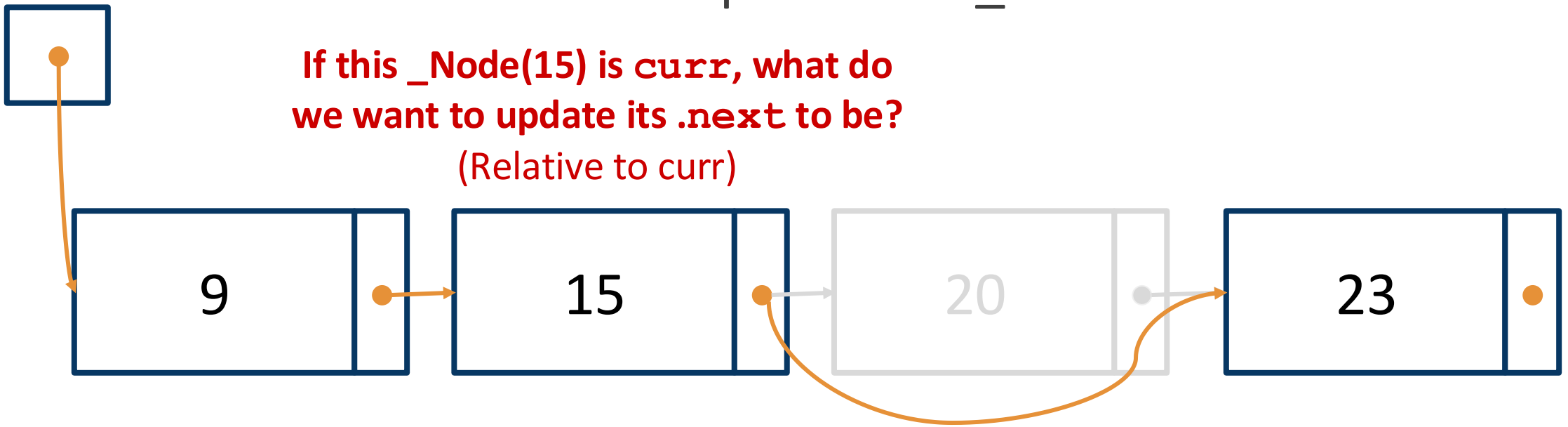


pop

- When do we need to update self._first?

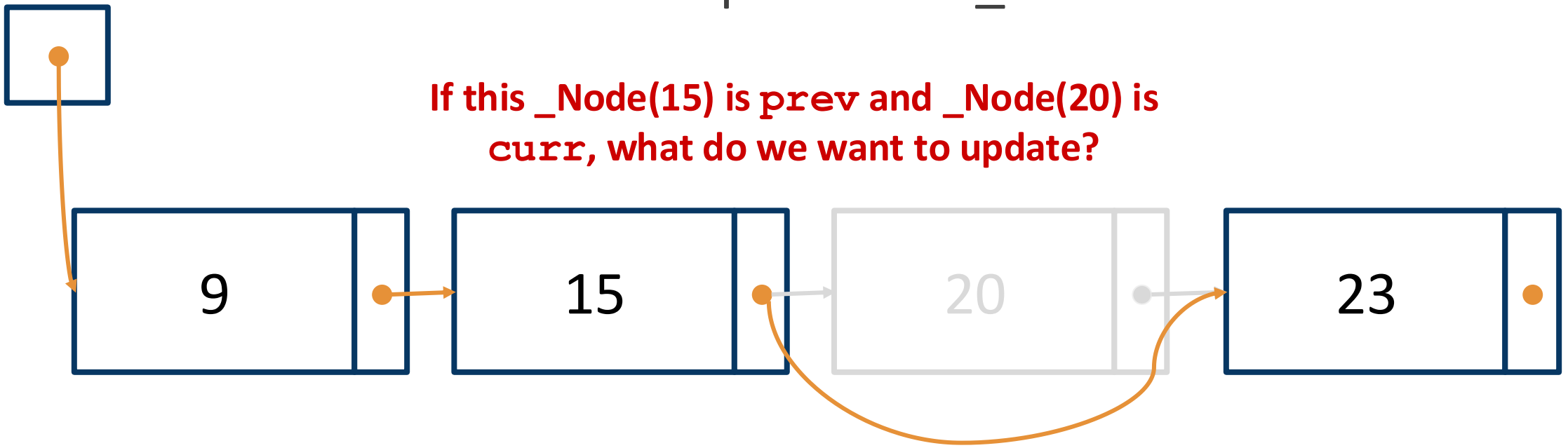
pop

- When do we need to update self._first?



pop

- When do we need to update self._first?



Same key ideas!

1. Figure out when we need to modify `self._first` vs. a _Node in the list.

1. When `index` > 0, iterate to the (`index`-1)th node and update links.

```
def pop(self, index: int) -> Any:
    """Remove and return the item at the given
    index.

    Raise IndexError if index >= len(self)
    or index < 0.
    """
```